

TensorGuard Formal Soundness Appendix

Generated from Lean, proof-footprint, and fragment sources

Deterministic repository artifact

1 Generated Provenance

This appendix is generated by `python -m src.formal_soundness_appendix`. It contains no hand-maintained theorem ledger: Lean declarations are scanned from repository source files, operator evidence is loaded from the live proof-footprint generator, and the accepted program fragment is rendered from the canonical grammar in `src.verifiable_fragment`.

Generated input	Observed value
Lean source files	52 files under <code>lean/</code> , excluding <code>.lake</code> .
Lean declarations	703 theorem statements and 0 lemma statements.
Axiom-audited declarations	459 <code>#print axioms</code> entries in <code>lean/TensorGuard/AxiomAudit.lean</code> .
Operator proof footprint	139 registered transfer functions from <code>src.proof_footprint</code> .
Verifiable fragment	70 nn layers, 25 torchvision v2 transforms, 54 tensor methods, 45 torch functions, 21 F functions.

2 Verifiable Fragment Grammar

The strict soundness mode may report a confident safe verdict only inside this generated fragment. Programs outside it must abstain with UNKNOWN unless a concrete bug has already been refuted.

```
Module      ::=  class C(nn.Module):
                  __init__(self, <params>): <InitBody>
                  forward(self, <inputs>): <FwdBody>

InitBody    ::=  (self.<attr> = <LayerExpr>)*
LayerExpr   ::=  nn.<SupportedLayer>(<literal>*)
                  |  nn.Sequential(<LayerExpr>*)
                  |  nn.ModuleList([<LayerExpr>*])

FwdBody     ::=  <Stmt>*; return <Expr>

Stmt        ::=  <var> = <Expr>
                  |  <var> = self.<attr>(<Expr>*)           # submodule call
                  |  for <var> in self.<modulelist>:         # static iteration
                      <Stmt>*

Expr        ::=  <var>
                  |  self.<attr>(<Expr>*)                   # nn.Module call
                  |  <SupportedFunc>(<Expr>*)              # torch.* / F.*
                  |  <Expr>.<SupportedMethod>(<literal>*)    # tensor method
                  |  <Expr> <BinOp> <Expr>                 # +, *, @
                  |  <literal>
```

2.1 Supported Construct Tables

Nn Layers (70) AdaptiveAvgPool1d, AdaptiveAvgPool2d, AdaptiveMaxPool1d, AdaptiveMaxPool2d, AlphaDropout, AvgPool1d, AvgPool2d, BatchNorm1d, BatchNorm2d, BatchNorm3d, ConstantPad2d, Conv1d, Conv2d, Conv3d, ConvTranspose1d, ConvTranspose2d, ConvTranspose3d, Dropout, Dropout2d, Dropout3d, ELU, Embedding, Flatten, Fold, FractionalMaxPool2d, GELU, GRU, GroupNorm, Hardsigmoid, Hardswish, Identity, InstanceNorm1d, InstanceNorm2d, InstanceNorm3d, LPPool2d, LSTM, LayerNorm, LeakyReLU, Linear, LogSoftmax, MaxPool1d, MaxPool2d, MaxPool3d, Mish, ModuleDict, ModuleList, MultiheadAttention, PReLU, PixelShuffle, PixelUnshuffle, RNN, ReLU, ReLU6, ReflectionPad2d, ReplicationPad2d, SELU, Sequential, SiLU, Sigmoid, Softmax, SyncBatchNorm, Tanh, TransformerDecoder, TransformerDecoderLayer, TransformerEncoder, TransformerEncoderLayer, Unflatten, Unfold, Upsample, ZeroPad2d

Torchvision V2 Transforms (25) CenterCrop, ColorJitter, Compose, FiveCrop, GaussianBlur, Identity, Normalize, PILToTensor, Pad, RandomAdjustSharpness, RandomAutocontrast, RandomCrop, RandomEqualize, RandomGrayscale, RandomHorizontalFlip, RandomInvert, RandomPosterize, RandomResizedCrop, RandomSolarize, RandomVerticalFlip, Resize, TenCrop, ToDtype, ToPILImage, ToPureTensor

Tensor Methods (54) add, add_, bmm, chunk, clone, contiguous, cpu, cuda, detach, dim, double, expand, flatten, flip, float, gather, half, index_select, matmul, mean, mm, moveaxis, movedim, mul, mul_, narrow, numel, permute, relu, repeat, repeat_interleave, reshape, roll, rot90, scatter, scatter_, scatter_add, scatter_add_, select, shape, sigmoid, size, softmax, split, squeeze, sum, swapaxes, swapdims, tanh, tile, to, transpose, unsqueeze, view

Torch Functions (45) torch.add, torch.arange, torch.bmm, torch.broadcast_shapes, torch.broadcast_tensors, torch.cat, torch.chunk, torch.column_stack, torch.dropout, torch.dstack, torch.einsum, torch.flatten, torch.flip, torch.gather, torch.hstack, torch.index_select, torch.linspace, torch.matmul, torch.mm, torch.moveaxis, torch.movedim, torch.mul, torch.ones, torch.ones_like, torch.relu, torch.repeat_interleave, torch.roll, torch.rot90, torch.row_stack, torch.scatter, torch.scatter_add, torch.sigmoid, torch.softmax, torch.split, torch.squeeze, torch.stack, torch.swapaxes, torch.swapdims, torch.tanh, torch.tile, torch.unsqueeze, torch.vstack, torch.where, torch.zeros, torch.zeros_like

F Functions (21) F.adaptive_avg_pool2d, F.avg_pool2d, F.batch_norm, F.conv1d, F.conv2d, F.dropout, F.elu, F.gelu, F.group_norm, F.interpolate, F.layer_norm, F.leaky_relu, F.linear, F.log_softmax, F.max_pool2d, F.pad, F.relu, F.sigmoid, F.silu, F.softmax, F.tanh

3 Proof-Footprint Summary

The following table is a direct projection of the live `tenorguard.proof_footprint/v1` payload. Only rows that name concrete Lean declarations claim the `lean_theorem` tier; other transfer functions are labeled as paper rules, tested-only rules, or heuristics.

Evidence tier	Count	Meaning
lean_theorem	16	A manifest row names imported Lean theorem evidence.
pen_and_paper_rule	102	A static mathematical rule is specified and tested, but not per-operator Lean-backed.
tested_only_rule	18	The transfer is validated by registry or differential tests without a compact proof rule.
heuristic	3	The output shape is data-dependent or intentionally approximate.

3.1 Lean-Backed Operator Rows

Operator	Lean module(s)	Lean theorem(s)
----------	----------------	-----------------

F.relu	TensorGuard.Extended	TensorGuard.relu_shape_preserving	
torch.bmm	TensorGuard.MatmulSound, TensorGuard.SmtEncoding	TensorGuard.MatmulSound.matmul_contraction_sound, TensorGuard.SmtEncoding.dtype_smt_matches_dtMatmulBug	
torch.broadcast_shapes	TensorGuard.BroadcastChain, TensorGuard.SmtEncoding	TensorGuard.BroadcastChain.bcDim_none_iff, TensorGuard.BroadcastChain.bcRun_append, TensorGuard.SmtEncoding.broadcast_smt_sat_iff_bcDim	some
torch.broadcast_tensors	TensorGuard.BroadcastChain, TensorGuard.SmtEncoding	TensorGuard.BroadcastChain.bcDim_none_iff, TensorGuard.BroadcastChain.bcRun_append, TensorGuard.SmtEncoding.broadcast_smt_sat_iff_bcDim	some
torch.clamp	TensorGuard.SoundnessV5	TensorGuard.V5.applyOp_sound_clamp	
torch.gather	TensorGuard.SoundnessV5, TensorGuard.V5OperatorRules	TensorGuard.V5.applyOp_sound_gather, TensorGuard.V5.gather_eq	
torch.index_select	TensorGuard.SoundnessV5	TensorGuard.V5.applyOp_sound_index_select	
torch.matmul	TensorGuard.MatmulSound	TensorGuard.MatmulSound.applyOpExt_sound_matmul, TensorGuard.MatmulSound.matmul_contraction_sound	
torch.mean	TensorGuard.AssumeGuarant, TensorGuard.RankTransfer	TensorGuard.applyOpExt_sound_mean_reduce, TensorGuard.RankTransfer.rankRun_le	
torch.mm	TensorGuard.MatmulSound, TensorGuard.SmtEncoding	TensorGuard.MatmulSound.matmul_contraction_sound, TensorGuard.SmtEncoding.dtype_smt_matches_dtMatmulBug	
torch.relu	TensorGuard.Extended	TensorGuard.relu_shape_preserving	
torch.scatter	TensorGuard.SoundnessV5, TensorGuard.V5OperatorRules	TensorGuard.V5.applyOp_sound_scatter, TensorGuard.V5.scatter_eq	
torch.squeeze	TensorGuard.SoundnessV5	TensorGuard.V5.applyOp_sound_squeeze	
torch.stack	TensorGuard.SoundnessV5, TensorGuard.V5OperatorRules	TensorGuard.V5.applyOp_sound_stack, TensorGuard.V5.stackOp_iface	
torch.sum	TensorGuard.AssumeGuarant, TensorGuard.RankTransfer	TensorGuard.applyOpExt_sound_sum_reduce, TensorGuard.RankTransfer.rankRun_le, TensorGuard.RankTransfer.rankRun_eq_sub_countDrop	
torch.unsqueeze	TensorGuard.SoundnessV5	TensorGuard.V5.applyOp_sound_unsqueeze	

3.2 Full Operator Evidence Ledger

Operator	Proof status	Confidence	Rule / Lean theorem evidence
F.celu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.elu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.gelu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.hardshrink	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.hardsigmoid	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.hardswish	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.leaky_relu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape

F.logsigmoid	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.mish	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.prelu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.relu	lean_theorem	complete	TensorGuard.relu_shape_preserving
F.rrelu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.selu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.sigmoid	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.silu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.softplus	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.softshrink	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.softsign	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.tanh	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
F.tanhshrink	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.abs	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.acos	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.all	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim
torch.amax	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim
torch.amin	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim
torch.any	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim
torch.argsort	tested_only_rule	sound	sort/topk/argsort preserve or statically set selected output axes
torch.asin	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.atan	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.bernoulli	tested_only_rule	sound	stochastic samplers return statically known tensor shapes
torch.bmm	lean_theorem	sound	TensorGuard.MatmulSound.matmul_contraction_soun TensorGuard.SmtEncoding.dtype_smt_matches_dtMa
torch.broadcast_shapes	lean_theorem	sound	TensorGuard.BroadcastChain.bcDim_none_iff, TensorGuard.BroadcastChain.bcRun_append, TensorGuard.SmtEncoding.broadcast_smt_sat_iff_bc
torch.broadcast_tensors	lean_theorem	sound	TensorGuard.BroadcastChain.bcDim_none_iff, TensorGuard.BroadcastChain.bcRun_append, TensorGuard.SmtEncoding.broadcast_smt_sat_iff_bc
torch.cdist	tested_only_rule	sound	cdist returns pairwise-distance matrix shapes from the two input point axes

torch.ceil	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.celu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.clamp	lean_theorem	complete	TensorGuard.V5.applyOp_sound_clamp
torch.clip	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.column_stack	pen_and_paper_rule	sound	stack-family operators combine equal-shaped tensors along statically determined axes
torch.cos	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.cosh	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.cross	pen_and_paper_rule	sound	linear/tensor contraction rule determined by ranks, broadcast prefixes, and static axes
torch.dstack	pen_and_paper_rule	sound	stack-family operators combine equal-shaped tensors along statically determined axes
torch.einsum	heuristic	heuristic	no proof rule claimed for data-dependent or approximated output shape
torch.elu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.eq	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.equal	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.erf	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.erfc	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.exp	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.fft.fft	tested_only_rule	sound	FFT contracts preserve transformed ranks and real/complex endpoint sizes
torch.fft.fft2	tested_only_rule	sound	FFT contracts preserve transformed ranks and real/complex endpoint sizes
torch.fft.ifft	tested_only_rule	sound	FFT contracts preserve transformed ranks and real/complex endpoint sizes
torch.fft.ifft2	tested_only_rule	sound	FFT contracts preserve transformed ranks and real/complex endpoint sizes
torch.fft.irfft	tested_only_rule	sound	FFT contracts preserve transformed ranks and real/complex endpoint sizes
torch.fft.rfft	tested_only_rule	sound	FFT contracts preserve transformed ranks and real/complex endpoint sizes
torch.flip	pen_and_paper_rule	sound	structural dimension transform with output shape determined by input shape and static args
torch.floor	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.gather	lean_theorem	sound	TensorGuard.V5.applyOp_sound_gather, TensorGuard.V5.gather_eq
torch.ge	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape

torch.gelu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.gt	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.hardshrink	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.hardsigmoid	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.hardswish	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.hstack	pen_and_paper_rule	sound	stack-family operators combine equal- shaped tensors along statically determined axes
torch.index_select	lean_theorem	sound	TensorGuard.V5.applyOp_sound_index_select
torch.isfinite	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.isinf	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.isnan	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.kron	pen_and_paper_rule	sound	linear/tensor contraction rule determined by ranks, broadcast prefixes, and static axes
torch.le	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.leaky_relu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.linalg.cholesky	tested_only_rule	sound	linear-algebra contracts enforce square/broadcast/multi-output shapes
torch.linalg.eig	tested_only_rule	sound	linear-algebra contracts enforce square/broadcast/multi-output shapes
torch.linalg.inv	tested_only_rule	sound	linear-algebra contracts enforce square/broadcast/multi-output shapes
torch.linalg.qr	tested_only_rule	sound	linear-algebra contracts enforce square/broadcast/multi-output shapes
torch.linalg.solve	tested_only_rule	sound	linear-algebra contracts enforce square/broadcast/multi-output shapes
torch.linalg.svd	tested_only_rule	sound	linear-algebra contracts enforce square/broadcast/multi-output shapes
torch.log	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.log10	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.log2	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.logsigmoid	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.logsumexp	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim
torch.lt	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.matmul	lean_theorem	sound	TensorGuard.MatmulSound.applyOpExt_sound_matmul TensorGuard.MatmulSound.matmul_contraction_sound
torch.max	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim

torch.mean	lean_theorem	sound	TensorGuard.applyOpExt_sound_mean_reduce, TensorGuard.RankTransfer.rankRun_le
torch.min	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim
torch.mish	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.mm	lean_theorem	sound	TensorGuard.MatmulSound.matmul_contraction_sound, TensorGuard.SmtEncoding.dtype_smt_matches_dtMa
torch.moveaxis	pen_and_paper_rule	sound	structural dimension transform with output shape determined by input shape and static args
torch.movedim	pen_and_paper_rule	sound	structural dimension transform with output shape determined by input shape and static args
torch.multinomial	heuristic	heuristic	no proof rule claimed for data-dependent or approximated output shape
torch.mv	pen_and_paper_rule	sound	linear/tensor contraction rule determined by ranks, broadcast prefixes, and static axes
torch.nan_to_num	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.ne	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.neg	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.norm	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim
torch.outer	pen_and_paper_rule	sound	linear/tensor contraction rule determined by ranks, broadcast prefixes, and static axes
torch.poisson	tested_only_rule	sound	stochastic samplers return statically known tensor shapes
torch.prelu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.prod	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim
torch.relu	lean_theorem	complete	TensorGuard.relu_shape_preserving
torch.repeat_interleave	pen_and_paper_rule	sound	structural dimension transform with output shape determined by input shape and static args
torch.roll	pen_and_paper_rule	sound	structural dimension transform with output shape determined by input shape and static args
torch.rot90	pen_and_paper_rule	sound	structural dimension transform with output shape determined by input shape and static args
torch.round	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.row_stack	pen_and_paper_rule	sound	stack-family operators combine equal-shaped tensors along statically determined axes
torch.rrelu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.rsqrt	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape

torch.scatter	lean_theorem	sound	TensorGuard.V5.applyOp_sound_scatter, TensorGuard.V5.scatter_eq
torch.selu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.sigmoid	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.sign	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.silu	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.sin	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.sinh	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.softplus	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.softshrink	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.softsign	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.sort	tested_only_rule	sound	sort/topk/argsort preserve or statically set selected output axes
torch.sqrt	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.squeeze	lean_theorem	sound	TensorGuard.V5.applyOp_sound_squeeze
torch.stack	lean_theorem	sound	TensorGuard.V5.applyOp_sound_stack, TensorGuard.V5.stackOp_iface
torch.std	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim
torch.sum	lean_theorem	sound	TensorGuard.applyOpExt_sound_sum_reduce, TensorGuard.RankTransfer.rankRun_le, TensorGuard.RankTransfer.rankRun_eq_sub_countDr
torch.swapaxes	pen_and_paper_rule	sound	structural dimension transform with output shape determined by input shape and static args
torch.swapdims	pen_and_paper_rule	sound	structural dimension transform with output shape determined by input shape and static args
torch.tan	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.tanh	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.tanhshrink	pen_and_paper_rule	complete	shape-preserving pointwise/comparison transfer: output shape equals input shape
torch.tensordot	pen_and_paper_rule	sound	linear/tensor contraction rule determined by ranks, broadcast prefixes, and static axes
torch.tile	pen_and_paper_rule	sound	structural dimension transform with output shape determined by input shape and static args
torch.topk	tested_only_rule	sound	sort/topk/argsort preserve or statically set selected output axes
torch.unique	heuristic	heuristic	no proof rule claimed for data-dependent or approximated output shape
torch.unsqueeze	lean_theorem	sound	TensorGuard.V5.applyOp_sound_unsqueeze

torch.var	pen_and_paper_rule	sound	single-axis reduction removes or retains the reduced axis depending on keepdim
torch.vstack	pen_and_paper_rule	sound	stack-family operators combine equal-shaped tensors along statically determined axes

4 Axiom-Audited Lean Statements

`AxiomAudit.lean` is the executable audit surface: each row is sent through `#print axioms`, and the regression suite rejects any dependency on unfinished proofs such as `sorryAx`. The statements below are mined from the Lean files that define those names.

Declaration	Source	Statement
TensorGuard.applyOp_sound_linear	lean/TensorGuard/	<code>(Soundness.applyOp66_sound_linear (i o n : Nat) (s' : Shape) (h : applyOp (.linear i o) (.cons n .nil) = some s') : s' = .cons o .nil /\ n = i</code>
TensorGuard.applyOp_sound_view	lean/TensorGuard/	<code>(Soundness.applyOp77_sound_view (out : List Nat) (s s' : Shape) (h : applyOp (.view out) s = some s') : s.prod = listProd out</code>
TensorGuard.applyOp_sound_broadcast_add	lean/TensorGuard/	<code>(Soundness.applyOp128_sound_broadcast_add (s s' : Shape) (h : applyOp .broadcast_add s = some s') : s' = s</code>
TensorGuard.applyOpExt_sound_matmul	lean/TensorGuard/	<code>(AssumeApplyOpExt_solved_lean249 (s s' : Shape) (h : applyOpExt .matmul s = some s') : s.length >= 3 /\ s' = s</code>
TensorGuard.applyOpExt_sound_transpose	lean/TensorGuard/	<code>(AssumeApplyOpExt_solved_lean191 (dim0 dim1 : Nat) (s s' : Shape) (h : applyOpExt (.transpose dim0 dim1) s = some s') : dim0 < s.length /\ dim1 < s.length</code>
TensorGuard.applyOpExt_sound_permute	lean/TensorGuard/	<code>(AssumeApplyOpExt_solved_lean211 (perm : List Nat) (s s' : Shape) (h : applyOpExt (.permute perm) s = some s') : validPerm s perm = true</code>
TensorGuard.applyOpExt_sound_sum_reduce	lean/TensorGuard/	<code>(AssumeApplyOpExt_solved_lean271 (axis : Nat) (s s' : Shape) (h : applyOpExt (.sum_reduce axis) s = some s') : axis < s.length</code>
TensorGuard.V5.applyOp_sound_cross_entropy	lean/TensorGuard/	<code>(Soundness.V5.Op:296_sound_cross_entropy (input target out : Sh) (h : crossEntropyOp input target = some out) : out = []</code>
TensorGuard.V5.applyOp_sound_argmax	lean/TensorGuard/	<code>(Soundness.V5.Op:270_sound_argmax (input : Sh) (axis : Nat) (out : Sh) (h : argmaxOp input axis = some out) : out.length = input.length - 1</code>
TensorGuard.MatmulSound.matmul	lean/TensorGuard/	<code>(MonoidSMatmulCanc:44 (m k n : Nat) : V5.matmul (rest ++ [m, k]) (rest ++ [k, n]) = some (rest ++ [m, n])</code>
TensorGuard.BroadcastAddSound.broadcast_add_sound_total	lean/TensorGuard/	<code>(BroadcastAddSoundTotal:43 (s : Shape) : exists s', applyOpExt .broadcast_add s = some s'</code>
TensorGuard.RP.reduceTagNul_reductive	lean/TensorGuard/	<code>(ReducedProductTagNul:88 (p : PVal) : (reduceTagNul p).ple p = true</code>
TensorGuard.RP.reduceNulTag_reductive	lean/TensorGuard/	<code>(ReducedProductNulTag:99 (p : PVal) : (reduceNulTag p).ple p = true</code>
TensorGuard.RP.reduce_reductive	lean/TensorGuard/	<code>(ReducedProduct_lean236 (p : PVal) : (reduce p).ple p = true</code>
TensorGuard.RP.pmeet_le_left	lean/TensorGuard/	<code>(ReducedProduct_lean180 (a b : PVal) : (PVal.pmeet a b).ple a = true</code>

TensorGuard.RP.pmeet_le_right	lean/TensorGuard/ReducedProduct.lean#184	(a b : PVal) : (PVal.pmeet a b).ple b = true
TensorGuard.RP.pmeet_mono	lean/TensorGuard/ReducedProduct.lean#274	{a' b' : PVal} (ha : a.ple a' = true) (hb : b.ple b' = true) : (PVal.pmeet a b).ple (PVal.pmeet a' b') = true
TensorGuard.RP.reduce_mono_consistent	lean/TensorGuard/ReducedProduct.lean#296	consistent {a b : PVal} (hca : consistent a = true) (hcb : consistent b = true) (h : a.ple b = true) : (reduce a).ple (reduce b) = true
TensorGuard.RP.gamma_mono	lean/TensorGuard/ReducedProduct.lean#336	{a b : PVal} {c : CVal} (h : a.ple b = true) (hc : mem c a = true) : mem c b = true
TensorGuard.RP.pmeet_gamma	lean/TensorGuard/ReducedProduct.lean#345	{a b : PVal} (c : CVal) : mem c (PVal.pmeet a b) = (mem c a && mem c b)
TensorGuard.RP.reduce_gamma	lean/TensorGuard/ReducedProduct.lean#356	(p : PVal) (c : CVal) : mem c (reduce p) = mem c p
TensorGuard.Cegar.length_le_lsum	lean/TensorGuard/CegarBound.lean#145	sum (l : List Nat) (h : forall g in l, 1 <= g) : l.length <= lsum l
TensorGuard.Cegar.cegar_iter_bound	lean/TensorGuard/CegarBound.lean#168	(gains : List Nat) (h : forall g in gains, 1 <= g) : totalIters gains <= 1 + discovered gains
TensorGuard.Cegar.cegar_terminates	lean/TensorGuard/CegarBound.lean#178	(gains : List Nat) (U : Nat) (h : forall g in gains, 1 <= g) (hU : discovered gains <= U) : gains.length <= U /\ totalIters gains <= U + 1
TensorGuard.Cegar.tight_below_naive	lean/TensorGuard/CegarBound.lean#196	(gains : List Nat) (U : Nat) (hU : discovered gains < U) : 1 + discovered gains < U + 1
TensorGuard.CegarU2.decideNew_safeSound	lean/TensorGuard/CegarInfeasibleNew.lean#58	safeSound (infeasible realBug : Bool) (h : feasibleJustifiesSafety infeasible realBug) : safeSound (decideNew infeasible) realBug
TensorGuard.CegarU2.decideOld_unsound	lean/TensorGuard/CegarInfeasibleOld.lean#71	unsound : exists infeasible realBug, feasibleJustifiesSafety infeasible realBug /\ decideOld infeasible = Verdict.safe /\ realBug = true
TensorGuard.CegarU2.fix_abstains	lean/TensorGuard/CegarInfeasible.lean#82	on_infeasible : decideNew true = Verdict.abstain
TensorGuard.CegarU2.fix_keeps_safe	lean/TensorGuard/CegarInfeasible.lean#84	keeps_safe _when_feasible : decideNew false = Verdict.safe
TensorGuard.CegarU2.old_always_safe	lean/TensorGuard/CegarInfeasible.lean#86	safe (b : Bool) : decideOld b = Verdict.safe
TensorGuard.CegarSerialized.python_record_keys	lean/TensorGuard/CegarPredicatesSerialized.lean#79	pythonRecordKeys = ["schema", "kind", "tensor", "axis", "value", "match_tensor", "match_axis", "divisor", "provenance"]
TensorGuard.CegarSerialized.python_kind_names	lean/TensorGuard/CegarPredicatesSerialized.lean#85	pythonKindNames = ["DIM_EQ", "DIM_GT", "DIM_GE", "DIM_DIVISIBLE", "DIM_MATCH", "NDIM_EQ", "SHAPE_EQ"]
TensorGuard.CegarSerialized.serialized_extends_membership	lean/TensorGuard/CegarPredicatesSerialized.lean#92	old fresh : List SerializedPredicate) : Extends old (old ++ fresh)
TensorGuard.CegarSerialized.serialized_length_mono	lean/TensorGuard/CegarPredicatesSerialized.lean#99	old fresh : List SerializedPredicate) : old.length <= (old ++ fresh).length
TensorGuard.CegarSerialized.serialized_extends_prefix	lean/TensorGuard/CegarPredicatesSerialized.lean#105	previous added : List SerializedPredicate) : Extends previous (previous ++ added)

TensorGuard.CegarSerialized.serialized_decision/TensorGuard.CegarPredicates.Serialized.feasible_decision	lean/FeasibleSound	/CegarPredicates.Serialized.feasible_decision129ns : decideSerialized false = Verdict.abstain
TensorGuard.CegarSerialized.serialized_decision/TensorGuard.CegarPredicates.Serialized.feasible_decision	lean/FeasibleSound	/CegarPredicates.Serialized.feasible_decision132 decideSerialized true = Verdict.safe
TensorGuard.CegarSerialized.decideSerialized/TensorSafeSound	lean/FeasibleSound	/CegarPredicates.Serialized.feasible_sound138 (feasible realBug : Bool) (h : feasibleSerializedJustifiesSafety feasible realBug) : safeSound (decideSerialized feasible) realBug
TensorGuard.CegarSerialized.infeasible/TensorSafeSound	lean/FeasibleSound	/CegarPredicates.Serialized.infeasible_sound147Sound_any_bug (realBug : Bool) : safeSound (decideSerialized false) realBug
TensorGuard.FragmentU1.sound_safeSound	lean/TensorGuard	/FragmentModes.lean55Sound (outOfFragment coreFoundBug realBug : Bool) (h : coreSoundOnFragment outOfFragment coreFoundBug realBug) : safeSound (decide Mode.sound outOfFragment coreFoundBug) realBug
TensorGuard.FragmentU1.balanced_decision/TensorGuard	lean/TensorGuard	/FragmentModes.lean93sound : exists outOfFragment coreFoundBug realBug, coreSoundOnFragment outOfFragment coreFoundBug realBug /\ decide Mode.balanced outOfFragment coreFoundBug = Verdict.safe /\ realBug = true
TensorGuard.FragmentU1.heuristic_decision/TensorGuard	lean/TensorGuard	/FragmentModes.lean190sound : exists outOfFragment coreFoundBug realBug, coreSoundOnFragment outOfFragment coreFoundBug realBug /\ decide Mode.heuristic outOfFragment coreFoundBug = Verdict.safe /\ realBug = true
TensorGuard.FragmentU1.modes_agree/TensorGuard	lean/TensorGuard	/FragmentModes.lean211in_fragment (mode mode' : Mode) (coreFoundBug : Bool) : decide mode false coreFoundBug = decide mode' false coreFoundBug
TensorGuard.FragmentU1.modes_diff/TensorGuard	lean/TensorGuard	/FragmentModes.lean177ff_violation (outOfFragment coreFoundBug : Bool) : (decide Mode.sound outOfFragment coreFoundBug != decide Mode.balanced outOfFragment coreFoundBug) <-> (outOfFragment = true /\ coreFoundBug = false)
TensorGuard.DevDtype.devBug_no_false/TensorGuard	lean/TensorGuard	/DevDtypeBug.66_false_positive (a b : Dev) (h : a = Dev.unknown /\ b = Dev.unknown) : devBug a b = false
TensorGuard.DevDtype.devBug_refutation/TensorGuard	lean/TensorGuard	/DevDtypeBug.78refutation_sound (a b : Dev) (h : devBug a b = true) : forall ca cb, Dev.gamma a ca -> Dev.gamma b cb -> ca != cb
TensorGuard.DevDtype.cuda_then_cpu/TensorGuard	lean/TensorGuard	/DevDtype.lean164cpu_roundtrip (d : Dev) : moveTo Dev.cpu (moveTo Dev.cuda0 d) = Dev.cpu
TensorGuard.DevDtype.pinMemory_preserve/TensorGuard	lean/TensorGuard	/DevDtype.lean191preserves (d : Dev) : pinMemory d = d
TensorGuard.DevDtype.dtMatmulBug_no_false_positive/TensorGuard	lean/TensorGuard	/DevDtype.lean161Bug_no_false_positive (a b : Dt) (h : a = Dt.unknown /\ b = Dt.unknown) : dtMatmulBug a b = false
TensorGuard.DevDtype.dtMatmulBug_refutation/TensorGuard	lean/TensorGuard	/DevDtype.lean165Bug_refutation_sound (a b : Dt) (h : dtMatmulBug a b = true) : forall ca cb, Dt.gamma a ca -> Dt.gamma b cb -> ca != cb
TensorGuard.DevDtype.dtFloatParamBug_refutation/TensorGuard	lean/TensorGuard	/DevDtype.lean194Bug_refutation_sound (input : Dt) (h : dtFloatParamBug input = true) : forall c, Dt.gamma input c -> c.isFloat = false /\ c != Dt.unknown
TensorGuard.DevDtype.dtElementwiseBug_never/TensorGuard	lean/TensorGuard	/DevDtype.lean206ise_never_bug (a b : Dt) : dtElementwiseBug a b = false

TensorGuard.DevDtype.dtPromote	clean/TensorGuard	/DeviceDtypePromote208comm (a b : Dt) : dtPromote a b = dtPromote b a
TensorGuard.DevDtype.dtPromote	ideal/TensorGuard	/DeviceDtypePromote211idem (a : Dt) : dtPromote a a = a
TensorGuard.DevDtype.phaseBug	clean/TensorGuard	/DeviceDtypePhaseBug258count_gt_one (p : Phase) (t : Bool) (n : Nat) (h : n != 1) : phaseBug p t n = false
TensorGuard.DevDtype.phaseBug	evldant/TensorGuard	/DeviceDtypePhaseBug265eval_tracking_safe (n : Nat) : phaseBug Phase.eval true n = false
TensorGuard.DevDtype.phaseBug	lean/TensorGuard	/DeviceDtypePhaseBug272refutation_sound (p : Phase) (t : Bool) (n : Nat) (h : phaseBug p t n = true) : usesBatchStats p t = true /\ n = 1
TensorGuard.DevDtype.gradBrokenBug	lean/TensorGuard	/DeviceDtypeGradBrokenBug308refutation_sound (b : Bool) (h : gradBrokenBug b = true) : b = true
TensorGuard.DevDtype.productBug	lean/TensorGuard	/DeviceDtypeProductBug322false_iff (f : Flags) : productBug f = false <-> (f.dev = false /\ f.dtype = false /\ f.phase = false /\ f.grad = false)
TensorGuard.DevDtype.productBug	lean/TensorGuard	/DeviceDtypeProductBug339refutation_sound (f : Flags) (h : productBug f = true) : f.dev = true /\ f.dtype = true /\ f.phase = true /\ f.grad = true
TensorGuard.SmtEncoding.sat_iff	lean/TensorGuard	/SmtEncodingSat50iff (alpha : Type) (ca cb : alpha) : Sat ca cb <-> ca = cb
TensorGuard.SmtEncoding.unsat_iff	lean/TensorGuard	/SmtEncodingSat57iff {alpha : Type} (ca cb : alpha) : not Sat ca cb <-> ca != cb
TensorGuard.SmtEncoding.unsat_sound	lean/TensorGuard	/SmtEncodingSat61iff {alpha : Type} (ca cb : alpha) (h : not Sat ca cb) : ca != cb
TensorGuard.SmtEncoding.eq_is_sat	lean/TensorGuard	/SmtEncodingSat66iff (alpha : Type) (c : alpha) : Sat c c
TensorGuard.SmtEncoding.device_smt	lean/TensorGuard	/SmtEncodingSat75matches_devBug (ca cb : Dev) (ha : ca != Dev.unknown) (hb : cb != Dev.unknown) : (not Sat ca cb) <-> devBug ca cb = true
TensorGuard.SmtEncoding.phase_smt	lean/TensorGuard	/SmtEncodingSat96unsat_iff_ne (pa pb : Phase) : (not Sat pa pb) <-> pa != pb
TensorGuard.SmtEncoding.grad_smt	lean/TensorGuard	/SmtEncodingSat111unsat_iff_ne (gset greg : Bool) : (not Sat gset greg) <-> gset != greg
TensorGuard.SmtEncoding.dtype_smt	lean/TensorGuard	/SmtEncodingSat125unsat_iff_ne (da db : Dt) : (not Sat da db) <-> da != db
TensorGuard.SmtEncoding.dtype_smt	lean/TensorGuard	/SmtEncodingSat133matches_dtMatmulBug (da db : Dt) (ha : da != Dt.unknown) (hb : db != Dt.unknown) : (not Sat da db) <-> dtMatmulBug da db = true
TensorGuard.SmtEncoding.dtype_sat	lean/TensorGuard	/SmtEncodingSat150sat (d : Dt) : Sat d d
TensorGuard.SmtEncoding.broadcast_smt	lean/TensorGuard	/SmtEncodingSat169smt_sat_iff_bcDim_some (a b : Nat) : BroadcastSat a b <-> exists out, BroadcastChain.bcDim a b = some out
TensorGuard.SmtEncoding.broadcast_smt	lean/TensorGuard	/SmtEncodingSat186smt_unsat_iff_bcDim_none (a b : Nat) : not BroadcastSat a b <-> BroadcastChain.bcDim a b = none
TensorGuard.SmtEncoding.broadcast_smt	lean/TensorGuard	/SmtEncodingSat202smt_unsat_iff_incompatible (a b : Nat) : not BroadcastSat a b <-> (a != 1 /\ b != 1 /\ a != b)
TensorGuard.SmtEncoding.divisibility_smt	lean/TensorGuard	/SmtEncodingSat219smt_sat_iff_reshapeValid (total : Nat) (known : List Nat) : DivisibilitySat total known <-> ReshapeInfer.reshapeValid total known = true

TensorGuard.SmtEncoding.divisibility	<code>lean3 / TensorGuard / SmtEncoding / divisibility</code>	<code>lean3 / TensorGuard / SmtEncoding / divisibility</code>	<code>231</code>	<code>smt_unsat_iff_invalid</code> (total : Nat) (known : List Nat) : not DivisibilitySat total known <-> ReshapeInfer.reshapeValid total known = false
TensorGuard.SmtEncoding.partition	<code>lean3 / TensorGuard / SmtEncoding / partition</code>	<code>lean3 / TensorGuard / SmtEncoding / partition</code>	<code>249</code>	<code>smt_unsat_iff_sum_eq</code> (axisSize : Nat) (sections : List Nat) : PartitionSat axisSize sections <-> ChunkSplit.sum sections = axisSize
TensorGuard.SmtEncoding.partition	<code>lean3 / TensorGuard / SmtEncoding / partition</code>	<code>lean3 / TensorGuard / SmtEncoding / partition</code>	<code>261</code>	<code>smt_matches_splitSectionsValid</code> (s : ChunkSplit.AxisShape) (sections : List Nat) : PartitionSat s.axisSize sections <-> ChunkSplit.splitSectionsValid s sections = true
TensorGuard.SmtEncoding.partition	<code>lean3 / TensorGuard / SmtEncoding / partition</code>	<code>lean3 / TensorGuard / SmtEncoding / partition</code>	<code>268</code>	<code>smt_unsat_iff_mismatch</code> (s : ChunkSplit.AxisShape) (sections : List Nat) : not PartitionSat s.axisSize sections <-> ChunkSplit.splitSectionsValid s sections = false
TensorGuard.SmtEncoding.dtype_promote	<code>lean3 / TensorGuard / SmtEncoding / dtype_promote</code>	<code>lean3 / TensorGuard / SmtEncoding / dtype_promote</code>	<code>288</code>	<code>smt_sat_iff</code> (a b out : Dt) : DtypePromoteSat a b out <-> out = dtPromote a b
TensorGuard.SmtEncoding.dtype_promote	<code>lean3 / TensorGuard / SmtEncoding / dtype_promote</code>	<code>lean3 / TensorGuard / SmtEncoding / dtype_promote</code>	<code>299</code>	<code>smt_unsat_iff_mismatch</code> (a b out : Dt) : not DtypePromoteSat a b out <-> out != dtPromote a b
TensorGuard.SmtEncoding.dtype_promote	<code>lean3 / TensorGuard / SmtEncoding / dtype_promote</code>	<code>lean3 / TensorGuard / SmtEncoding / dtype_promote</code>	<code>311</code>	<code>smt_sat_iff</code> (acc : Dt) (xs : List Dt) (out : Dt) : DtypePromoteChainSat acc xs out <-> out = DtypePromoteChain.promoteRun acc xs
TensorGuard.SmtEncoding.dtype_promote	<code>lean3 / TensorGuard / SmtEncoding / dtype_promote</code>	<code>lean3 / TensorGuard / SmtEncoding / dtype_promote</code>	<code>313</code>	<code>smt_unsat_iff_mismatch</code> (acc : Dt) (xs : List Dt) (out : Dt) : not DtypePromoteChainSat acc xs out <-> out != DtypePromoteChain.promoteRun acc xs
TensorGuard.SparseLayouts.dense_mkAccept	<code>lean3 / TensorGuard / SparseLayouts / dense_mkAccept</code>	<code>lean3 / TensorGuard / SparseLayouts / dense_mkAccept</code>	<code>661</code>	<code>dense_shape_sound</code> (spec : SparseSpec) : denseShapeOf spec = spec.shape
TensorGuard.SparseLayouts.mkAccept	<code>lean3 / TensorGuard / SparseLayouts / mkAccept</code>	<code>lean3 / TensorGuard / SparseLayouts / mkAccept</code>	<code>661</code>	<code>mkAccept</code> (spec : SparseSpec) : acceptedShape spec.shape (mkAccepted ok spec)
TensorGuard.SparseLayouts.coo234	<code>lean3 / TensorGuard / SparseLayouts / coo234</code>	<code>lean3 / TensorGuard / SparseLayouts / coo234</code>	<code>673</code>	<code>cooSpec</code> [2, 3] [3, 4] [2, 3, 4] = some { layout
TensorGuard.SparseLayouts.csr23	<code>lean3 / TensorGuard / SparseLayouts / csr23</code>	<code>lean3 / TensorGuard / SparseLayouts / csr23</code>	<code>686</code>	<code>csrSpec</code> [3] [3] [3] [2, 3] = some { layout
TensorGuard.SparseLayouts.csc23	<code>lean3 / TensorGuard / SparseLayouts / csc23</code>	<code>lean3 / TensorGuard / SparseLayouts / csc23</code>	<code>699</code>	<code>cscSpec</code> [4] [3] [3] [2, 3] = some { layout
TensorGuard.SparseLayouts.bsr43	<code>lean3 / TensorGuard / SparseLayouts / bsr43</code>	<code>lean3 / TensorGuard / SparseLayouts / bsr43</code>	<code>712</code>	<code>bsrSpec</code> [3] [2] [2, 2, 3] [4, 3] = some { layout
TensorGuard.SparseLayouts.bsc23	<code>lean3 / TensorGuard / SparseLayouts / bsc23</code>	<code>lean3 / TensorGuard / SparseLayouts / bsc23</code>	<code>725</code>	<code>bscSpec</code> [2] [1] [1, 2, 3] [2, 3] = some { layout
TensorGuard.SparseLayouts.batched	<code>lean3 / TensorGuard / SparseLayouts / batched</code>	<code>lean3 / TensorGuard / SparseLayouts / batched</code>	<code>738</code>	<code>batched</code> accepts : csrSpec [2, 3] [2, 3] [2, 3, 4] [2, 2, 3, 4] = some { layout
TensorGuard.SparseLayouts.batched	<code>lean3 / TensorGuard / SparseLayouts / batched</code>	<code>lean3 / TensorGuard / SparseLayouts / batched</code>	<code>751</code>	<code>batched</code> accepts : bsrSpec [2, 3] [2, 2] [2, 2, 2, 3, 5] [2, 4, 3, 5] = some { layout
TensorGuard.SparseLayouts.coo234	<code>lean3 / TensorGuard / SparseLayouts / coo234</code>	<code>lean3 / TensorGuard / SparseLayouts / coo234</code>	<code>768</code>	<code>dense_shape</code> : Option.map denseShapeOf (cooSpec [2, 3] [3, 4] [2, 3, 4]) = some [2, 3, 4]
TensorGuard.SparseLayouts.csr23	<code>lean3 / TensorGuard / SparseLayouts / csr23</code>	<code>lean3 / TensorGuard / SparseLayouts / csr23</code>	<code>773</code>	<code>dense_shape</code> : Option.map denseShapeOf (csrSpec [3] [3] [3] [2, 3]) = some [2, 3]
TensorGuard.SparseLayouts.csc23	<code>lean3 / TensorGuard / SparseLayouts / csc23</code>	<code>lean3 / TensorGuard / SparseLayouts / csc23</code>	<code>776</code>	<code>dense_shape</code> : Option.map denseShapeOf (cscSpec [4] [3] [3] [2, 3]) = some [2, 3]

TensorGuard.SparseLayouts.bsr43_toDense	/lean4/TensorGuard/SparseLayouts.lean#286	Dense_shape : Option.map denseShapeOf (bsrSpec [3] [2] [2, 2, 3] [4, 3]) = some [4, 3]
TensorGuard.SparseLayouts.bsc23_toDense	/lean4/TensorGuard/SparseLayouts.lean#284	Dense_shape : Option.map denseShapeOf (bscSpec [2] [1] [1, 2, 3] [2, 3]) = some [2, 3]
TensorGuard.SparseLayouts.batched_csr_toDense	/lean4/TensorGuard/SparseLayouts.lean#288	csr_shape : Option.map denseShapeOf (csrSpec [2, 3] [2, 3] [2, 3, 4] [2, 2, 3, 4]) = some [2, 2, 3, 4]
TensorGuard.SparseLayouts.batched_bsr_toDense	/lean4/TensorGuard/SparseLayouts.lean#292	bsr_shape : Option.map denseShapeOf (bsrSpec [2, 3] [2, 2] [2, 2, 2, 3, 5] [2, 4, 3, 5]) = some [2, 4, 3, 5]
TensorGuard.SparseLayouts.csr_bad_length	/lean4/TensorGuard/SparseLayouts.lean#300	compressed_length_rejected : csrSpec [2] [3] [3] [2, 3] = none
TensorGuard.SparseLayouts.csc_bad_length	/lean4/TensorGuard/SparseLayouts.lean#303	compressed_length_rejected : cscSpec [3] [3] [3] [2, 3] = none
TensorGuard.SparseLayouts.bsr_bad_row_divisibility	/lean4/TensorGuard/SparseLayouts.lean#306	_divisibility_rejected : bsrSpec [3] [2] [2, 2, 3] [5, 3] = none
TensorGuard.SparseLayouts.bsr_bad_column_divisibility	/lean4/TensorGuard/SparseLayouts.lean#309	column_divisibility_rejected : bsrSpec [3] [2] [2, 2, 3] [4, 4] = none
TensorGuard.SparseLayouts.compressed_and_transposed	/lean4/TensorGuard/SparseLayouts.lean#312	dense_tail_mismatch_rejected : csrSpec [3] [3] [3, 5] [2, 3, 4] = none
TensorGuard.RecurrentRule.batch_first_output_preserves_layout	/lean4/TensorGuard/RecurrentRule.lean#106	(cell : CellKind) (input hidden layers proj batch seq : Nat) (bidir : Bool) : outputShape { cell
TensorGuard.RecurrentRule.time_major_output_preserves_layout	/lean4/TensorGuard/RecurrentRule.lean#136	(cell : CellKind) (input hidden layers proj seq batch : Nat) (bidir : Bool) : outputShape { cell
TensorGuard.RecurrentRule.batch_first_state_selects_dim0	/lean4/TensorGuard/RecurrentRule.lean#166	(cell : CellKind) (input hidden layers proj batch seq : Nat) : stateShape { cell
TensorGuard.RecurrentRule.time_major_state_selects_dim1	/lean4/TensorGuard/RecurrentRule.lean#196	(cell : CellKind) (input hidden layers proj seq batch : Nat) : stateShape { cell
TensorGuard.RecurrentRule.bidirectional_output_feature_doubles	/lean4/TensorGuard/RecurrentRule.lean#226	(cell : CellKind) (input hidden layers proj seq batch : Nat) : outputShape { cell
TensorGuard.RecurrentRule.bidirectional_state_depth_doubles	/lean4/TensorGuard/RecurrentRule.lean#255	(cell : CellKind) (input hidden layers proj batch seq : Nat) : stateShape { cell
TensorGuard.RecurrentRule.lstm_cell_state_size_under_projection	/lean4/TensorGuard/RecurrentRule.lean#285	(input hidden layers proj batch seq : Nat) : stateShape { cell
TensorGuard.RecurrentRule.gru_cell_state_size_rejected	/lean4/TensorGuard/RecurrentRule.lean#303	(input hidden layers proj batch seq : Nat) : stateShape { cell
TensorGuard.RecurrentRule.rnn_cell_state_size_rejected	/lean4/TensorGuard/RecurrentRule.lean#321	(input hidden layers proj batch seq : Nat) : stateShape { cell
TensorGuard.RecurrentRule.projected_lstm_output_shape	/lean4/TensorGuard/RecurrentRule.lean#355	projectedBiLSTM [3, 5, 11] = some [3, 5, 10]
TensorGuard.RecurrentRule.projected_lstm_h_state_shape	/lean4/TensorGuard/RecurrentRule.lean#358	stateShape projectedBiLSTM StateKind.hidden [3, 5, 11] = some [4, 3, 5]
TensorGuard.RecurrentRule.projected_lstm_c_state_shape	/lean4/TensorGuard/RecurrentRule.lean#363	stateShape projectedBiLSTM StateKind.cell [3, 5, 11] = some [4, 3, 9]

TensorGuard.SubjectReduction.soundlean4347	lean4347	lean4347	implies_supported_branch {env env' : Env} {cond : BranchCond} {thenBranch elseBranch : Program} (hSome : condExec env cond thenBranch elseBranch = some env') : cond = BranchCond.supported
TensorGuard.SubjectReduction.unsupportedlean353	lean353	lean353	branch_cannot_silently_safe {env env' : Env} {thenBranch elseBranch : Program} : condExec env BranchCond.unsupported thenBranch elseBranch != some env'
TensorGuard.SubjectReduction.supportedlean360	lean360	lean360	additional_subject_reduction {env env' : Env} {thenBranch elseBranch : Program} (hEnv : EnvWF env) (hThenWT : ProgramWT env thenBranch) (hElseWT : ProgramWT env elseBranch) (hExec : condExec env BranchCond.supported thenBranch elseBranch = some env') : EnvWF env'
TensorGuard.SubjectReduction.conditionallean381	lean381	lean381	conditional_program_subject_reduction {env joinedEnv env' : Env} {thenBranch elseBranch tail : Program} (hEnv : EnvWF env) (hThenWT : ProgramWT env thenBranch) (hElseWT : ProgramWT env elseBranch) (hTailWT : ProgramWT joinedEnv tail) (hCond : condExec env BranchCond.supported thenBranch elseBranch = some joinedEnv) (hExec : condThenExec env BranchCond.supported thenBranch elseBranch tail = some env') : EnvWF env'
TensorGuard.SubjectReduction.conditionallean717	lean717	lean717	exec_shape : condExec [[2, 8]] BranchCond.supported branchThenProgram branchElseProgram = some [[2, 8], [2, 16], [2, 16]]
TensorGuard.SubjectReduction.conditionallean722	lean722	lean722	tail_exec_shape : condThenExec [[2, 8]] BranchCond.supported branchThenProgram branchElseProgram branchTailProgram = some [[2, 8], [2, 16], [2, 16], [2, 4]]
TensorGuard.SubjectReduction.divergentlean732	lean732	lean732	join_rejected : condExec [[2, 8]] BranchCond.supported branchThenProgram branchElseDivergentProgram = none
TensorGuard.SubjectReduction.unsupportedlean737	lean737	lean737	branch_abstains_example : condExec [[2, 8]] BranchCond.unsupported branchThenProgram branchElseProgram = none
TensorGuard.SubjectReduction.exec_programlean427	lean427	lean427	list_subject_reduction {env env' : Env} {bodies : List Program} (hEnv : EnvWF env) (hWT : ProgramListWT env bodies) (hExec : execProgramList env bodies = some env') : EnvWF env'
TensorGuard.SubjectReduction.boundedlean453	lean453	lean453	bounded_unroll_limit_abstains {env env' : Env} {kind : LoopKind} {bodies : List Program} {limit : Nat} (hExec : boundedUnrollExec env kind bodies limit = some env') : kind != LoopKind.unsupported /\ bodies.length <= limit /\ execProgramList env bodies = some env'
TensorGuard.SubjectReduction.moduleListlean479	lean479	lean479	end_unroll_limit_abstains {env : Env} {bodies : List Program} {limit : Nat} (hTooMany : limit < bodies.length) : boundedUnrollExec env LoopKind.moduleList bodies limit = none

TensorGuard.SubjectReduction.static_range/TensorGuard/SubjectReduction/lean485	lean485	lean485	beyond_unroll_limit abstains {env : Env} {bodies : List Program} {limit : Nat} (hTooMany : limit < bodies.length) : boundedUnrollExec env LoopKind.staticRange bodies limit = none
TensorGuard.SubjectReduction.unsupported/TensorGuard/SubjectReduction/lean491	lean491	lean491	loop_cannot_silently_safe {env env' : Env} {bodies : List Program} {limit : Nat} : boundedUnrollExec env LoopKind.unsupported bodies limit != some env'
TensorGuard.SubjectReduction.bounded/TensorGuard/SubjectReduction/lean497	lean497	lean497	subject_reduction {env env' : Env} {kind : LoopKind} {bodies : List Program} {limit : Nat} (hEnv : EnvWF env) (hWT : ProgramListWT env bodies) (hExec : boundedUnrollExec env kind bodies limit = some env') : EnvWF env'
TensorGuard.SubjectReduction.moduleList/TensorGuard/SubjectReduction/lean758	lean758	lean758	exec_shape : boundedUnrollExec [[2, 8]] LoopKind.moduleList moduleListUnrollBodies 3 = some [[2, 8], [2, 16], [2, 16], [2, 16], [2, 16], [2, 4]]
TensorGuard.SubjectReduction.static_range/TensorGuard/SubjectReduction/lean776	lean776	lean776	roll_exec_shape : boundedUnrollExec [[2, 8]] LoopKind.staticRange staticRangeUnrollBodies 3 = some [[2, 8], [2, 8], [2, 8], [2, 4]]
TensorGuard.SubjectReduction.moduleList/TensorGuard/SubjectReduction/lean782	lean782	lean782	limit_rejected : boundedUnrollExec [[2, 8]] LoopKind.moduleList moduleListUnrollBodies 2 = none
TensorGuard.SubjectReduction.static_range/TensorGuard/SubjectReduction/lean787	lean787	lean787	beyond_limit_rejected : boundedUnrollExec [[2, 8]] LoopKind.staticRange staticRangeUnrollBodies 2 = none
TensorGuard.SubjectReduction.unsupported/TensorGuard/SubjectReduction/lean792	lean792	lean792	loop_abstains_example : boundedUnrollExec [[2, 8]] LoopKind.unsupported moduleListUnrollBodies 3 = none
TensorGuard.CrossDomain.transfer_skiff/TensorGuard/CrossDomain/lean53	lean53	lean53	iff_shape_eq (pre post : List Nat) : transferHolds pre post <=> pre = post
TensorGuard.CrossDomain.transfer_unsat/TensorGuard/CrossDomain/lean57	lean57	lean57	iff_shape_ne (pre post : List Nat) : not transferHolds pre post <=> pre != post
TensorGuard.CrossDomain.transfer_device/TensorGuard/CrossDomain/lean64	lean64	lean64	free (pre post : List Nat) (h : pre = post) : forall _dpre _dpost : Dev, transferHolds pre post
TensorGuard.CrossDomain.nontransfer_skiff/TensorGuard/CrossDomain/lean78	lean78	lean78	iff_dev_eq (dpre dpost : Dev) : nonTransferHolds dpre dpost <=> dpre = dpost
TensorGuard.CrossDomain.nontransfer_unsat/TensorGuard/CrossDomain/lean82	lean82	lean82	iff_dev_ne (dpre dpost : Dev) : not nonTransferHolds dpre dpost <=> dpre != dpost
TensorGuard.CrossDomain.nontransfer_shape/TensorGuard/CrossDomain/lean88	lean88	lean88	shape_free (dpre dpost : Dev) (h : dpre = dpost) : forall _pre _post : List Nat, nonTransferHolds dpre dpost
TensorGuard.CrossDomain.branch_select/TensorGuard/CrossDomain/lean93	lean93	lean93	is_preserved (isTransfer : Bool) (pre post : List Nat) (dpre dpost : Dev) : (if isTransfer then transferHolds pre post else nonTransferHolds dpre dpost) = (if isTransfer then pre = post else dpre = dpost)
TensorGuard.DevDtype.dtPromote_assoc/TensorGuard/DevDtype/lean219	lean219	lean219	assoc (a b c : Dt) : dtPromote (dtPromote a b) c = dtPromote a (dtPromote b c)

TensorGuard.DevDtype.dtPromote_unknown/TensorGuard/DeviceDtypePromote226	unknown/TensorGuard/DeviceDtypePromote226	unknown_absorbs_left (a : Dt) : dtPromote Dt.unknown a = Dt.unknown
TensorGuard.DevDtype.dtPromote_unknown/TensorGuard/DeviceDtypePromote230	unknown/TensorGuard/DeviceDtypePromote230	unknown_absorbs_right (a : Dt) : dtPromote a Dt.unknown = Dt.unknown
TensorGuard.GradFlow.keep_id_lean/TensorGuard/GradFlowKeep61	lean/TensorGuard/GradFlowKeep61	(b : Bool) : gradStep b GradOp.keep = b
TensorGuard.GradFlow.detach_false_lean/TensorGuard/GradFlowDetach62	lean/TensorGuard/GradFlowDetach62	false (b : Bool) : gradStep b GradOp.detach = false
TensorGuard.GradFlow.reattach_true_lean/TensorGuard/GradFlowReattach64	lean/TensorGuard/GradFlowReattach64	true (b : Bool) : gradStep b GradOp.reattach = true
TensorGuard.GradFlow.reset_absorbing_lean/TensorGuard/GradFlowReset68	lean/TensorGuard/GradFlowReset68	absorbing (op : GradOp) (h : isReset op = true) (b b' : Bool) : gradStep b op = gradStep b' op
TensorGuard.GradFlow.gradRun_append_lean/TensorGuard/GradFlowGradRun77	lean/TensorGuard/GradFlowGradRun77	append (b0 : Bool) (xs ys : List GradOp) : gradRun b0 (xs ++ ys) = gradRun (gradRun b0 xs) ys
TensorGuard.GradFlow.run_after_reset_lean/TensorGuard/GradFlowRunAfter87	lean/TensorGuard/GradFlowRunAfter87	reset (b0 b' : Bool) (xs : List GradOp) (op : GradOp) (h : isReset op = true) : gradRun b0 (xs ++ [op]) = gradRun b' (xs ++ [op])
TensorGuard.GradFlow.run_noReattach_lean/TensorGuard/GradFlowRunNoReattach90	lean/TensorGuard/GradFlowRunNoReattach90	Reattach_true_iff (b0 : Bool) (xs : List GradOp) (h : forall op in xs, op != GradOp.reattach) : gradRun b0 xs = (b0 && xs.all (fun op => !isReset op))
TensorGuard.DevicePlacement.keep_id_lean/TensorGuard/DevicePlacementKeep82	lean/TensorGuard/DevicePlacementKeep82	(d : Dev) : devStep d DevOp.keep = d
TensorGuard.DevicePlacement.move_absorbing_lean/TensorGuard/DevicePlacementMove88	lean/TensorGuard/DevicePlacementMove88	absorbing (op : DevOp) (h : isMove op = true) (d d' : Dev) : devStep d op = devStep d' op
TensorGuard.DevicePlacement.devRun_append_lean/TensorGuard/DevicePlacementDevRun92	lean/TensorGuard/DevicePlacementDevRun92	append (d0 : Dev) (xs ys : List DevOp) : devRun d0 (xs ++ ys) = devRun (devRun d0 xs) ys
TensorGuard.DevicePlacement.run_after_move_lean/TensorGuard/DevicePlacementRunAfter10	lean/TensorGuard/DevicePlacementRunAfter10	move (d0 d' : Dev) (xs : List DevOp) (op : DevOp) (h : isMove op = true) : devRun d0 (xs ++ [op]) = devRun d' (xs ++ [op])
TensorGuard.DevicePlacement.run_endless_lean/TensorGuard/DevicePlacementRunEndless119	lean/TensorGuard/DevicePlacementRunEndless119	target (d0 : Dev) (xs : List DevOp) (op : DevOp) (h : isMove op = true) : devRun d0 (xs ++ [op]) = moveTarget op
TensorGuard.DevicePlacement.run_noMove_lean/TensorGuard/DevicePlacementRunNoMove127	lean/TensorGuard/DevicePlacementRunNoMove127	(d0 : Dev) (xs : List DevOp) (h : forall op in xs, isMove op = false) : devRun d0 xs = d0
TensorGuard.DevicePlacement.binValid_lean/TensorGuard/DevicePlacementBinValid147	lean/TensorGuard/DevicePlacementBinValid147	(d : Dev) : binValid d d = true
TensorGuard.DevicePlacement.binValid_iff_lean/TensorGuard/DevicePlacementBinValid151	lean/TensorGuard/DevicePlacementBinValid151	(a b : Dev) : binValid a b = true <=> a = b
TensorGuard.DevicePlacement.cpu_accel_lean/TensorGuard/DevicePlacementCpuAccel155	lean/TensorGuard/DevicePlacementCpuAccel155	binValid Dev.cpu : binValid Dev.accel = false
TensorGuard.DevicePlacement.chain_binValid_iff_lean/TensorGuard/DevicePlacementChainBinValid160	lean/TensorGuard/DevicePlacementChainBinValid160	iff (d0 d1 : Dev) (xs ys : List DevOp) : binValid (devRun d0 xs) (devRun d1 ys) = true <=> devRun d0 xs = devRun d1 ys
TensorGuard.PhaseFlow.keep_id_lean/TensorGuard/PhaseFlowKeep64	lean/TensorGuard/PhaseFlowKeep64	(b : Bool) : phaseStep b PhaseOp.keep = b
TensorGuard.PhaseFlow.setTrain_true_lean/TensorGuard/PhaseFlowSetTrain65	lean/TensorGuard/PhaseFlowSetTrain65	true (b : Bool) : phaseStep b PhaseOp.setTrain = true
TensorGuard.PhaseFlow.setEval_false_lean/TensorGuard/PhaseFlowSetEval66	lean/TensorGuard/PhaseFlowSetEval66	false (b : Bool) : phaseStep b PhaseOp.setEval = false

TensorGuard.PhaseFlow.setter_absorb	lean/TensorGuard	/PhaseFlow.lean:70	absorbing (op : PhaseOp) (h : isSetter op = true) (b b' : Bool) : phaseStep b op = phaseStep b' op
TensorGuard.PhaseFlow.phaseRun_append	lean/TensorGuard	/PhaseFlow.lean:84	append (b0 : Bool) (xs ys : List PhaseOp) : phaseRun b0 (xs ++ ys) = phaseRun (phaseRun b0 xs) ys
TensorGuard.PhaseFlow.run_after_setter	lean/TensorGuard	/PhaseFlow.lean:92	after_setter (b0 b0' : Bool) (xs : List PhaseOp) (op : PhaseOp) (h : isSetter op = true) : phaseRun b0 (xs ++ [op]) = phaseRun b0' (xs ++ [op])
TensorGuard.PhaseFlow.run_ends_at_value	lean/TensorGuard	/PhaseFlow.lean:101	ends_at_value (b0 : Bool) (xs : List PhaseOp) (op : PhaseOp) (h : isSetter op = true) : phaseRun b0 (xs ++ [op]) = setterValue op
TensorGuard.PhaseFlow.run_noSetter_id	lean/TensorGuard	/PhaseFlow.lean:109	noSetter_id (b0 : Bool) (xs : List PhaseOp) (h : forall op in xs, isSetter op = false) : phaseRun b0 xs = b0
TensorGuard.RankTransfer.keep_id	lean/TensorGuard	/RankTransfer.lean:56	keep_id (r : Nat) : rankStep r RedOp.reduceKeep = r
TensorGuard.RankTransfer.drop_pre	lean/TensorGuard	/RankTransfer.lean:67	drop_pre (r : Nat) : rankStep r RedOp.reduceDrop = r - 1
TensorGuard.RankTransfer.step_le	lean/TensorGuard	/RankTransfer.lean:60	step_le (r : Nat) (op : RedOp) : rankStep r op <= r
TensorGuard.RankTransfer.rankRun_append	lean/TensorGuard	/RankTransfer.lean:70	append (r0 : Nat) (xs ys : List RedOp) : rankRun r0 (xs ++ ys) = rankRun (rankRun r0 xs) ys
TensorGuard.RankTransfer.rankRun_sub_countDrop	lean/TensorGuard	/RankTransfer.lean:78	sub_countDrop (r0 : Nat) (xs : List RedOp) : rankRun r0 xs = r0 - countDrop xs
TensorGuard.RankTransfer.rankRun_le	lean/TensorGuard	/RankTransfer.lean:93	le (r0 : Nat) (xs : List RedOp) : rankRun r0 xs <= r0
TensorGuard.RankTransfer.rankRun_keep	lean/TensorGuard	/RankTransfer.lean:98	Keep (r0 : Nat) (xs : List RedOp) (h : forall op in xs, op = RedOp.reduceKeep) : rankRun r0 xs = r0
TensorGuard.RankTransfer.rankRun_countDrop	lean/TensorGuard	/RankTransfer.lean:116	countDrop (r0 : Nat) (xs : List RedOp) (h : countDrop xs <= r0) : rankRun r0 xs + countDrop xs = r0
TensorGuard.DtypePromoteChain.promoteRun_append	lean/TensorGuard	/DtypePromoteChain.lean:79	append (acc : Dt) (xs ys : List Dt) : promoteRun acc (xs ++ ys) = promoteRun (promoteRun acc xs) ys
TensorGuard.DtypePromoteChain.promoteRun_acc	lean/TensorGuard	/DtypePromoteChain.lean:86	acc (acc : Dt) (xs : List Dt) : dtLe acc (promoteRun acc xs) = true
TensorGuard.DtypePromoteChain.promoteRun_lem	lean/TensorGuard	/DtypePromoteChain.lean:97	lem (acc : Dt) (xs : List Dt) (x : Dt) (hx : x in xs) : dtLe x (promoteRun acc xs) = true
TensorGuard.DtypePromoteChain.promoteRun_swap	lean/TensorGuard	/DtypePromoteChain.lean:113	swap (acc x y : Dt) (rest : List Dt) : promoteRun acc (x :: y :: rest) = promoteRun acc (y :: x :: rest)
TensorGuard.DtypePromoteChain.promoteRun_unknown	lean/TensorGuard	/DtypePromoteChain.lean:122	unknown (xs : List Dt) : promoteRun Dt.unknown xs = Dt.unknown
TensorGuard.BroadcastChain.bcDim	lean/TensorGuard	/BroadcastChain.lean:59	bcDim (a b : Nat) : bcDim a b = bcDim b a
TensorGuard.BroadcastChain.bcDim_left	lean/TensorGuard	/BroadcastChain.lean:70	left (a : Nat) : bcDim 1 a = some a
TensorGuard.BroadcastChain.bcDim_right	lean/TensorGuard	/BroadcastChain.lean:74	right (a : Nat) : bcDim a 1 = some a
TensorGuard.BroadcastChain.bcDim_self	lean/TensorGuard	/BroadcastChain.lean:83	self (a : Nat) : bcDim a a = some a

TensorGuard.BroadcastChain.bcDim_leam/TensorGuard/BroadcastChain_leam	88	max (a b : Nat) (ha1 : a >= 1) (hb1 : b >= 1) (h : a = 1 $\vee$ b = 1 $\vee$ a = b) : bcDim a b = some (max a b)
TensorGuard.BroadcastChain.bcDim_leam/TensorGuard/BroadcastChain_leam	98	suff (a b : Nat) : bcDim a b = none <-> (a != 1 $\wedge$ b != 1 $\wedge$ a != b)
TensorGuard.BroadcastChain.bcRun_leam/TensorGuard/BroadcastChain_leam	109	(acc : Option Nat) (xs ys : List Nat) : bcRun acc (xs ++ ys) = bcRun (bcRun acc xs) ys
TensorGuard.BroadcastChain.bcRun_leam/TensorGuard/BroadcastChain_leam	166	(xs : List Nat) : bcRun none xs = none
TensorGuard.BroadcastChain.bcRun_leam/TensorGuard/BroadcastChain_leam	222	(acc : Option Nat) (xs : List Nat) (h : forall x in xs, x = 1) : bcRun acc xs = acc
TensorGuard.ContigFlow.ctgRun_append/TensorGuard/ContigFlowRun	73	append (b0 : Bool) (xs ys : List LayoutOp) : ctgRun b0 (xs ++ ys) = ctgRun (ctgRun b0 xs) ys
TensorGuard.ContigFlow.run_cons_contig/TensorGuard/ContigFlow_leam	80	contig (b0 : Bool) (rest : List LayoutOp) : ctgRun b0 (LayoutOp.contiguous :: rest) = ctgRun true rest
TensorGuard.ContigFlow.run_after_contig/TensorGuard/ContigFlow_leam	84	contig (b0 : Bool) (pre post : List LayoutOp) : ctgRun b0 (pre ++ LayoutOp.contiguous :: post) = ctgRun true post
TensorGuard.ContigFlow.run_transpose/TensorGuard/ContigFlow_leam	91	pose_involution (b0 : Bool) (rest : List LayoutOp) : ctgRun b0 (LayoutOp.transpose :: LayoutOp.transpose :: rest) = ctgRun b0 rest
TensorGuard.ContigFlow.run_allKeep/TensorGuard/ContigFlow_leam	107	keep (b0 : Bool) (xs : List LayoutOp) (h : forall op in xs, op = LayoutOp.keep) : ctgRun b0 xs = b0
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	205	high2low_is_contiguous (shape : List Nat) (off : Nat) : isContiguous { shape
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	232	NCHWTail_viewable (n c h w off : Nat) : canViewNCHWTail { shape
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	244	NCHWTail_view_layout (n c h w off : Nat) : viewNCHWTail { shape
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	255	transpose01_preserves_storageOffset (l : Layout) : (transpose01 l).storageOffset = l.storageOffset
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	259	permuteNCHWtoNHWC_preserves_storageOffset (l : Layout) : (permuteNCHWtoNHWC l).storageOffset = l.storageOffset
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	268	rowMajor4_strides : rowMajor4.strides = [60, 20, 5, 1]
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	271	rowMajor4_view_tail : viewNCHWTail rowMajor4 = some { shape
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	285	channelsLast4_last_canonical_channels_last : isCanonicalChannelsLast4 channelsLast4 = true
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	286	channelsLast4_row_major_contiguous : isContiguous channelsLast4 = false
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	288	channelsLast4_tail_not_viewable : canViewNCHWTail channelsLast4 = false
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	289	channelsLast4_view_tail_rejected : viewNCHWTail channelsLast4 = none
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	287	narrowChannel_rowMajor_offset : narrowChannel 1 2 rowMajor4 = some { shape
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow_leam	291	narrowChannel_channelsLast_offset : narrowChannel 1 2 channelsLast4 = some { shape

TensorGuard.ContigFlow.LayoutAlgebra	lean/ContigFlow/Algebra	ContigFlowAlgebra298C abstains canonicalChannelsLast4_degenerateC_abstains : canonicalChannelsLast4Strides [2, 1, 4, 5] = none
TensorGuard.Flatten.prod_append	lean/TensorGuard/Flatten	Flatten.prod19append (xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.Flatten.prod_flatten	lean/TensorGuard/Flatten	Flatten.prod61flatten (pre span suf : List Nat) : prod (flattenShape pre span suf) = prod (pre ++ span ++ suf)
TensorGuard.Flatten.length_flatten	lean/TensorGuard/Flatten	Flatten.length74_flatten (pre span suf : List Nat) : (flattenShape pre span suf).length = pre.length + 1 + suf.length
TensorGuard.Flatten.flatten_full	lean/TensorGuard/Flatten	Flatten.flatten86_full (span : List Nat) : flattenShape [] span [] = [prod span]
TensorGuard.Flatten.flatten_singleton	lean/TensorGuard/Flatten	Flatten.flatten92_singleton (pre : List Nat) (d : Nat) (suf : List Nat) : flattenShape pre [d] suf = pre ++ (d :: suf)
TensorGuard.Flatten.flatten_dim_value	lean/TensorGuard/Flatten	Flatten.flatten98_dim_value (pre span suf : List Nat) : (flattenShape pre span suf).drop pre.length = prod span :: suf
TensorGuard.CatRule.prod_append	lean/TensorGuard/CatRule	CatRule.prod50append (xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.CatRule.catValid_iff	lean/TensorGuard/CatRule	CatRule.catValid79_iff (a b : CatShape) : catValid a b = true <-> (a.pre = b.pre /\ a.post = b.post)
TensorGuard.CatRule.catValid_pre_mismatch	lean/TensorGuard/CatRule	CatRule.catValid85_pre_mismatch (a b : CatShape) (h : a.pre != b.pre) : catValid a b = false
TensorGuard.CatRule.catAxis_value	lean/TensorGuard/CatRule	CatRule.catAxis95_value (a b : CatShape) : (catShape a b).axisSize = a.axisSize + b.axisSize
TensorGuard.CatRule.prod_cat	lean/TensorGuard/CatRule	CatRule.prod105_cat (a b : CatShape) (h : catValid a b = true) : prod (toList (catShape a b)) = prod (toList a) + prod (toList b)
TensorGuard.CatRule.cat_axis_comm	lean/TensorGuard/CatRule	CatRule.cat_axis125_comm (a b : CatShape) : (catShape a b).axisSize = (catShape b a).axisSize
TensorGuard.CatRule.cat_assoc	lean/TensorGuard/CatRule	CatRule.cat_axis136_assoc (a b c : CatShape) : (catShape (catShape a b) c).axisSize = (catShape a (catShape b c)).axisSize
TensorGuard.CatRule.cat_zero_right	lean/TensorGuard/CatRule	CatRule.cat_axis136_zero_right (a : CatShape) : (catShape a { pre
TensorGuard.EmbeddingRule.prod_append	lean/EmbeddingRule	EmbeddingRule.prod47(xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.EmbeddingRule.embeddingValid	lean/EmbeddingRule	EmbeddingRule.embeddingValid62(input : List Nat) (embDim : Nat) : (embShape input embDim).length = input.length + 1
TensorGuard.EmbeddingRule.embeddingValid	lean/EmbeddingRule	EmbeddingRule.embeddingValid68(input : List Nat) (embDim : Nat) : (embShape input embDim).drop input.length = [embDim]
TensorGuard.EmbeddingRule.embeddingValid	lean/EmbeddingRule	EmbeddingRule.embeddingValid73(input : List Nat) (embDim : Nat) : (embShape input embDim).take input.length = input
TensorGuard.EmbeddingRule.embeddingValid	lean/EmbeddingRule	EmbeddingRule.embeddingValid79(input : List Nat) (embDim : Nat) : prod (embShape input embDim) = prod input * embDim
TensorGuard.EmbeddingRule.idxValid_iff	lean/EmbeddingRule	EmbeddingRule.idxValid95(idx num : Nat) : idxValid idx num = true <-> idx < num
TensorGuard.EmbeddingRule.allValid_iff	lean/EmbeddingRule	EmbeddingRule.allValid100(idxs : List Nat) (num : Nat) : allValid idxs num = true <-> forall i in idxs, i < num

TensorGuard.EmbeddingRule.outOfRange	lean/TensorGuard	/TensorGuard.EmbeddingRule.outOfRange100	flagged (idx num : Nat) (h : num <= idx) : idxValid idx num = false
TensorGuard.EmbeddingRule.allValid	lean/TensorGuard	/TensorGuard.EmbeddingRule.allValid105	(idxs : List Nat) (num num' : Nat) (hle : num <= num') (h : allValid idxs num = true) : allValid idxs num' = true
TensorGuard.ReshapeInfer.prod_append	lean/TensorGuard	/TensorGuard.ReshapeInfer.prod_append143	(xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.ReshapeInfer.reshapeValid	lean/TensorGuard	/TensorGuard.ReshapeInfer.reshapeValid161	total : Nat) (known : List Nat) : reshapeValid total known = true <-> (0 < prod known /\ prod known ? total)
TensorGuard.ReshapeInfer.reshapeValid	lean/TensorGuard	/TensorGuard.ReshapeInfer.reshapeValid170	total : Nat) (known : List Nat) (h : reshapeValid total known = true) : prod known ? total
TensorGuard.ReshapeInfer.nondivisible	lean/TensorGuard	/TensorGuard.ReshapeInfer.nondivisible176	total : Nat) (known : List Nat) (h : not prod known ? total) : reshapeValid total known = false
TensorGuard.ReshapeInfer.inferDim	lean/TensorGuard	/TensorGuard.ReshapeInfer.inferDim189	total : Nat) (known : List Nat) (_hpos : 0 < prod known) (hdvd : prod known ? total) : prod known * inferDim total known = total
TensorGuard.ReshapeInfer.prod_reshapeValid	lean/TensorGuard	/TensorGuard.ReshapeInfer.prod_reshapeValid197	total : Nat) (known : List Nat) (h : reshapeValid total known = true) : prod (reshapeShape total known) = total
TensorGuard.ReshapeInfer.reshapeInfer	lean/TensorGuard	/TensorGuard.ReshapeInfer.reshapeInfer198	total : Nat) (known : List Nat) : (reshapeShape total known).getLast? = some (inferDim total known)
TensorGuard.LinearRule.lin_rank	lean/TensorGuard	/TensorGuard.LinearRule.lin_rank15	(pre : List Nat) (inF outF : Nat) : (linShape pre outF).length = (pre ++ [inF]).length
TensorGuard.LinearRule.lin_last	lean/TensorGuard	/TensorGuard.LinearRule.lin_last19	(pre : List Nat) (outF : Nat) : (linShape pre outF).drop pre.length = [outF]
TensorGuard.LinearRule.lin_prefix	lean/TensorGuard	/TensorGuard.LinearRule.lin_prefix53	(pre : List Nat) (outF : Nat) : (linShape pre outF).take pre.length = pre
TensorGuard.LinearRule.lin_numel	lean/TensorGuard	/TensorGuard.LinearRule.lin_numel57	(pre : List Nat) (outF : Nat) : prod (linShape pre outF) = prod pre * outF
TensorGuard.LinearRule.linValid	lean/TensorGuard	/TensorGuard.LinearRule.linValid62	(lastDim inF : Nat) : linValid lastDim inF = true <-> lastDim = inF
TensorGuard.LinearRule.mismatch	lean/TensorGuard	/TensorGuard.LinearRule.mismatch67	(lastDim inF : Nat) (h : lastDim != inF) : linValid lastDim inF = false
TensorGuard.Conv2d.convOut_identity	lean/TensorGuard	/TensorGuard.Conv2d.convOut_identity	(input : Nat) (h : 1 <= input) : convOut input 0 1 1 = input
TensorGuard.Conv2d.convOut_stride_one	lean/TensorGuard	/TensorGuard.Conv2d.convOut_stride_one	(input pad e : Nat) : convOut input pad e 1 = input + 2 * pad - e + 1
TensorGuard.Conv2d.convOut_mono	lean/TensorGuard	/TensorGuard.Conv2d.convOut_mono	(i? i? pad e stride : Nat) (h : i? <= i?) : convOut i? pad e stride <= convOut i? pad e stride
TensorGuard.Conv2d.convOut_le	lean/TensorGuard	/TensorGuard.Conv2d.convOut_le	(input pad e stride : Nat) (he : 1 <= e) (hs : 1 <= stride) (hv : e <= input + 2 * pad) : convOut input pad e stride <= input + 2 * pad
TensorGuard.Conv2d.convOut_pos	lean/TensorGuard	/TensorGuard.Conv2d.convOut_pos	(input pad e stride : Nat) : 1 <= convOut input pad e stride
TensorGuard.Conv2d.conv2d_rank	lean/TensorGuard	/TensorGuard.Conv2d.conv2d_rank	(n cout hOut wOut : Nat) : (conv2dShape n cout hOut wOut).length = 4
TensorGuard.Conv2d.conv2d_channels	lean/TensorGuard	/TensorGuard.Conv2d.conv2d_channels	(n cout hOut wOut : Nat) : (conv2dShape n cout hOut wOut).get? 1 = some cout

TensorGuard.Pool2d.poolOut_identity	lean/TensorGuard	/Theorem.poolOut_identity (input : Nat) (h : 1 <= input) : poolOut input 0 1 1 = input
TensorGuard.Pool2d.poolOut_stride_one	lean/TensorGuard	/Theorem.poolOut_stride_one (input pad kernel : Nat) : poolOut input pad kernel 1 = input + 2 * pad - kernel + 1
TensorGuard.Pool2d.poolOut_mono	lean/TensorGuard	/Theorem.poolOut_mono (i? i? pad kernel stride : Nat) (h : i? <= i?) : poolOut i? pad kernel stride <= poolOut i? pad kernel stride
TensorGuard.Pool2d.poolOut_le	lean/TensorGuard	/Theorem.poolOut_le (input pad kernel stride : Nat) (hk : 1 <= kernel) (hv : kernel <= input + 2 * pad) : poolOut input pad kernel stride <= input + 2 * pad
TensorGuard.Pool2d.poolOut_pos	lean/TensorGuard	/Theorem.poolOut_pos (input pad kernel stride : Nat) : 1 <= poolOut input pad kernel stride
TensorGuard.Pool2d.pool2d_channels_preserved	lean/TensorGuard	/Theorem.pool2d_channels_preserved (n c hOut wOut : Nat) : (pool2dShape n c hOut wOut).get? 1 = some c
TensorGuard.ConvTranspose.ctOut_identity	lean/TensorGuard	/Theorem.ConvTranspose_identity (input : Nat) (h : 1 <= input) : ctOut input 1 0 0 0 = input
TensorGuard.ConvTranspose.ctOut_stride	lean/TensorGuard	/Theorem.ConvTranspose_stride (input stride dk op : Nat) : ctOut input stride 0 dk op = (input - 1) * stride + dk + op + 1
TensorGuard.ConvTranspose.ctOut_mono	lean/TensorGuard	/Theorem.ConvTranspose_mono (i? i? stride pad dk op : Nat) (h : i? <= i?) : ctOut i? stride pad dk op <= ctOut i? stride pad dk op
TensorGuard.ConvTranspose.ctOut_ge	lean/TensorGuard	/Theorem.ConvTranspose_ge (input stride dk op : Nat) (hs : 1 <= stride) (h : 1 <= input) : input <= ctOut input stride 0 dk op
TensorGuard.ConvTranspose.ct_channels	lean/TensorGuard	/Theorem.ConvTranspose_channels (n cout lOut : Nat) : (ctShape n cout lOut).get? 1 = some cout
TensorGuard.LayerNormRule.In_preshape	lean/TensorGuard	/Theorem.LayerNormRule_in_preshape (pre normShape : List Nat) : InShape pre normShape = pre ++ normShape
TensorGuard.LayerNormRule.In_numel	lean/TensorGuard	/Theorem.LayerNormRule_in_numel (pre normShape : List Nat) : prod (InShape pre normShape) = prod pre * prod normShape
TensorGuard.LayerNormRule.In_length	lean/TensorGuard	/Theorem.LayerNormRule_in_length (pre normShape : List Nat) : (InShape pre normShape).length - normShape.length = pre.length
TensorGuard.LayerNormRule.In_suffixed	lean/TensorGuard	/Theorem.LayerNormRule_in_suffixed (pre normShape : List Nat) : (InShape pre normShape).drop pre.length = normShape
TensorGuard.LayerNormRule.In_mismatched	lean/TensorGuard	/Theorem.LayerNormRule_in_mismatched (pre : List Nat) (a b : Nat) (h : a != b) : (InShape pre [a]).drop pre.length != [b]
TensorGuard.PixelShuffle.ps_numel	lean/TensorGuard	/Theorem.PixelShuffle_numel (n c r h w : Nat) : inNumel n c r h w = outNumel n c r h w
TensorGuard.PixelShuffle.ps_divisible	lean/TensorGuard	/Theorem.PixelShuffle_divisible (c r : Nat) : (r * r) ? (c * (r * r))
TensorGuard.PixelShuffle.ps_cout	lean/TensorGuard	/Theorem.PixelShuffle_cout (c r : Nat) (hr : 0 < r) : (c * (r * r)) / (r * r) = c
TensorGuard.PixelShuffle.psValid_iff	lean/TensorGuard	/Theorem.PixelShuffle_valid_iff (cin r : Nat) (hr : 0 < r) : psValid cin r = true <-> (r * r) ? cin
TensorGuard.PixelShuffle.ps_construct_valid	lean/TensorGuard	/Theorem.PixelShuffle_construct_valid (c r : Nat) : psValid (c * (r * r)) r = true
TensorGuard.AdaptivePool.ap_spatial	lean/TensorGuard	/Theorem.AdaptivePool_spatial (n c oh ow : Nat) : (apShape n c oh ow).get? 2 = some oh

TensorGuard.AdaptivePool.ap_spatial_lean	/TensorGuard	/AdaptivePoolSpatial39w (n c oh ow : Nat) : (apShape n c oh ow).get? 3 = some ow
TensorGuard.AdaptivePool.ap_channel_lean	/TensorGuard	/AdaptivePoolChannel34 (n c oh ow : Nat) : (apShape n c oh ow).get? 1 = some c
TensorGuard.AdaptivePool.ap_idempotent	/TensorGuard	/AdaptivePoolIdempotent43 (n c h w : Nat) : apShape n c h w = [n, c, h, w]
TensorGuard.Unflatten.unflatten_numel	/TensorGuard	/UnflattenNumel45n (pre sizes suf : List Nat) (orig : Nat) (h : prod sizes = orig) : prod (unflattenShape pre sizes suf) = prod (pre ++ [orig] ++ suf)
TensorGuard.Unflatten.unflatten_rank	/TensorGuard	/UnflattenRank53n (pre sizes suf : List Nat) : (unflattenShape pre sizes suf).length = pre.length + sizes.length + suf.length
TensorGuard.Unflatten.unflatten_then_flatten	/TensorGuard	/UnflattenThenFlatten61n (pre sizes suf : List Nat) : pre ++ [prod sizes] ++ suf = pre ++ [prod (unflattenShape pre sizes suf >.drop pre.length >.take sizes.length)] ++ suf
TensorGuard.Unflatten.unflattenValid_iff	/TensorGuard	/UnflattenValid70n (sizes : List Nat) (orig : Nat) : unflattenValid sizes orig = true <-> prod sizes = orig
TensorGuard.Unflatten.size_mismatch_flagged	/TensorGuard	/UnflattenSizeMismatchFlagged74 (sizes : List Nat) (orig : Nat) (h : prod sizes != orig) : unflattenValid sizes orig = false
TensorGuard.BatchNormRule.bn_prekernel	/TensorGuard	/BatchNormRulePrekernel34n (feat : Nat) (rest : List Nat) : bnShape n feat rest = n :: feat :: rest
TensorGuard.BatchNormRule.bn_numel	/TensorGuard	/BatchNormRuleNumel37n (feat : Nat) (rest : List Nat) : prod (bnShape n feat rest) = prod (n :: feat :: rest)
TensorGuard.BatchNormRule.featValid_iff	/TensorGuard	/BatchNormRuleFeatValid48n (feat numFeatures : Nat) : featValid feat numFeatures = true <-> feat = numFeatures
TensorGuard.BatchNormRule.feat_mismatch_flagged	/TensorGuard	/BatchNormRuleFeatMismatchFlagged45n (feat numFeatures : Nat) (h : feat != numFeatures) : featValid feat numFeatures = false
TensorGuard.BatchNormRule.bn_channel_index	/TensorGuard	/BatchNormRuleChannelIndex52n (n feat : Nat) (rest : List Nat) : (bnShape n feat rest).get? 1 = some feat
TensorGuard.Conv1d.convOut_identity	/TensorGuard	/Conv1dConvOutIdentity37n (input : Nat) (h : 1 <= input) : convOut input 0 1 1 = input
TensorGuard.Conv1d.convOut_stride_one	/TensorGuard	/Conv1dConvOutStrideOne40n (input pad e : Nat) : convOut input pad e 1 = input + 2 * pad - e + 1
TensorGuard.Conv1d.convOut_mono	/TensorGuard	/Conv1dConvOutMono49n (i? i? pad e stride : Nat) (h : i? <= i?) : convOut i? pad e stride <= convOut i? pad e stride
TensorGuard.Conv1d.convOut_le	/TensorGuard	/Conv1dConvOutLe52n (input pad e stride : Nat) (he : 1 <= e) (hv : e <= input + 2 * pad) : convOut input pad e stride <= input + 2 * pad
TensorGuard.Conv1d.convOut_pos	/TensorGuard	/Conv1dConvOutPos60n (input pad e stride : Nat) : 1 <= convOut input pad e stride
TensorGuard.Conv1d.conv1d_channels	/TensorGuard	/Conv1dConv1dChannels67n (n cout lOut : Nat) : (conv1dShape n cout lOut).get? 1 = some cout
TensorGuard.Einops.prod_append	/TensorGuard	/EinopsProd35 append (xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.Einops.decompValid_iff	/TensorGuard	/EinopsDecompValid54n (axis knownProduct : Nat) : decompValid axis knownProduct = true <-> (0 < knownProduct /\ knownProduct ? axis)
TensorGuard.Einops.decompValid_imp_dvd	/TensorGuard	/EinopsDecompValidImpDvd60n (axis knownProduct : Nat) (h : decompValid axis knownProduct = true) : knownProduct ? axis

TensorGuard.Einops.nondivisible_decomposition_flagged	lean/TensorGuard/	TensorGuard.Einops.nondivisible_decomposition_flagged (axis knownProduct : Nat) (h : not knownProduct ? axis) : decompValid axis knownProduct = false
TensorGuard.Einops.inferSubaxis_spec	lean/TensorGuard/	TensorGuard.Einops.inferSubaxis_spec (axis knownProduct : Nat) (_hpos : 0 < knownProduct) (hdvd : knownProduct ? axis) : knownProduct * inferSubaxis axis knownProduct = axis
TensorGuard.Einops.decomposedGroup_product	lean/TensorGuard/	TensorGuard.Einops.decomposedGroup_product (axis knownProduct : Nat) (h : decompValid axis knownProduct = true) : prod (decomposedGroup axis knownProduct) = axis
TensorGuard.Einops.inferSubaxis_position	lean/TensorGuard/	TensorGuard.Einops.inferSubaxis_position (axis knownProduct : Nat) : (decomposedGroup axis knownProduct).getLast? = some (inferSubaxis axis knownProduct)
TensorGuard.Einops.axisBijection_iff_counts	lean/TensorGuard/	TensorGuard.Einops.axisBijection_iff_counts (lhs rhs : List Nat) : axisBijection lhs rhs <=> forall a, countAxis a lhs = countAxis a rhs
TensorGuard.Einops.axisBijection_refl	lean/TensorGuard/	TensorGuard.Einops.axisBijection_refl (xs : List Nat) : axisBijection xs xs
TensorGuard.Einops.axisBijection_sym	lean/TensorGuard/	TensorGuard.Einops.axisBijection_sym {lhs rhs : List Nat} (h : axisBijection lhs rhs) : axisBijection rhs lhs
TensorGuard.Einops.axisBijection_trans	lean/TensorGuard/	TensorGuard.Einops.axisBijection_trans {xs ys zs : List Nat} (hxy : axisBijection xs ys) (hyz : axisBijection ys zs) : axisBijection xs zs
TensorGuard.Einops.adjacent_swap_axis_bijection	lean/TensorGuard/	TensorGuard.Einops.adjacent_swap_axis_bijection (a b : Nat) (rest : List Nat) : axisBijection (a :: b :: rest) (b :: a :: rest)
TensorGuard.Einops.dropped_axis_not_bijection	lean/TensorGuard/	TensorGuard.Einops.dropped_axis_not_bijection {a b : Nat} (h : a != b) : not axisBijection [a, b] [a]
TensorGuard.Einops.added_axis_not_bijection	lean/TensorGuard/	TensorGuard.Einops.added_axis_not_bijection {a b : Nat} (h : a != b) : not axisBijection [a] [a, b]
TensorGuard.Einops.duplicated_axis_not_bijection	lean/TensorGuard/	TensorGuard.Einops.duplicated_axis_not_bijection (a : Nat) : not axisBijection [a] [a, a]
TensorGuard.SDPA.bcDim_none_iff	lean/TensorGuard/	TensorGuard.SDPA.bcDim_none_iff (a b : Nat) : bcDim a b = none <=> (a != 1 /\ b != 1 /\ a != b)
TensorGuard.SDPA.bcShape_same	lean/TensorGuard/	TensorGuard.SDPA.bcShape_same (xs : List Nat) : bcShape xs xs = some xs
TensorGuard.SDPA.bcShape_suffix_same	lean/TensorGuard/	TensorGuard.SDPA.bcShape_suffix_same (pre suffix : List Nat) : bcShape (pre ++ suffix) suffix = some (pre ++ suffix)
TensorGuard.SDPA.standard_output_shape	lean/TensorGuard/	TensorGuard.SDPA.standard_output_shape (qLead kLead vLead lead : List Nat) (lq eq ev : Nat) (hbc : bc3 qLead kLead vLead = some lead) : standardOutput? qLead kLead vLead lq eq eq ev = some (lead ++ [lq, ev])
TensorGuard.SDPA.standard_output_rank	lean/TensorGuard/	TensorGuard.SDPA.standard_output_rank (lead : List Nat) (lq ev : Nat) : (lead ++ [lq, ev]).length = lead.length + 2
TensorGuard.SDPA.standard_equal_leads	lean/TensorGuard/	TensorGuard.SDPA.standard_equal_leads (lead : List Nat) (lq eq ev : Nat) : standardOutput? lead lead lead lq eq eq ev = some (lead ++ [lq, ev])
TensorGuard.SDPA.mask_exact_valid	lean/TensorGuard/	TensorGuard.SDPA.mask_exact_valid (lead : List Nat) (lq lk : Nat) : maskValid lead lq lk (scoreShape lead lq lk) = true
TensorGuard.SDPA.mask_trailing_valid	lean/TensorGuard/	TensorGuard.SDPA.mask_trailing_valid (lead : List Nat) (lq lk : Nat) : maskValid lead lq lk [lq, lk] = true

TensorGuard.SDPA.gqaHeadsValid_iff	lean/TensorGuard/SDPA/lean	SDPA.lean14gqaHeadsValid_iff (hq hk hv : Nat) : gqaHeadsValid hq hk hv = true <-> (0 < hk /\ hk ? hq /\ 0 < hv /\ hv ? hq)
TensorGuard.SDPA.gqa_key_repetition_count	lean/TensorGuard/SDPA/lean18gqa_key_repetition_count	(h : gqaHeadsValid hq hk hv = true) : hk * (hq / hk) = hq
TensorGuard.SDPA.gqa_value_repetition_count	lean/TensorGuard/SDPA/lean18gqa_value_repetition_count	(h : gqaHeadsValid hq hk hv = true) : hv * (hq / hv) = hq
TensorGuard.SDPA.gqa_nondivisible_key_flagged	lean/TensorGuard/SDPA/lean19gqa_nondivisible_key_flagged	(h : not hk ? hq) : gqaHeadsValid hq hk hv = false
TensorGuard.SDPA.gqa_nondivisible_value_flagged	lean/TensorGuard/SDPA/lean19gqa_nondivisible_value_flagged	(h : not hv ? hq) : gqaHeadsValid hq hk hv = false
TensorGuard.SDPA.gqa_output_shape	lean/TensorGuard/SDPA/lean20gqa_output_shape	(qBatch kBatch vBatch batch : List Nat) (hq hk hv lq eq ev : Nat) (hheads : gqaHeadsValid hq hk hv = true) (hbc : bc3 qBatch kBatch vBatch = some batch) : gqaOutput? qBatch kBatch vBatch hq hk hv lq eq eq ev = some (batch ++ [hq, lq, ev])
TensorGuard.SDPA.gqa_output_rank	lean/TensorGuard/SDPA/lean21gqa_output_rank	(batch : List Nat) (hq lq ev : Nat) : (batch ++ [hq, lq, ev]).length = batch.length + 3
TensorGuard.SDPA.gqa_output_uses_query_heads	lean/TensorGuard/SDPA/lean21gqa_output_uses_query_heads	(batch : List Nat) (hq lq ev : Nat) : (batch ++ [hq, lq, ev]).get? batch.length = some hq
TensorGuard.SDPA.gqa_prefix_broadcast_required	lean/TensorGuard/SDPA/lean220gqa_prefix_broadcast_required	(qBatch kBatch vBatch : List Nat) (hq hk hv lq eq ev : Nat) (hheads : gqaHeadsValid hq hk hv = true) (hbc : bc3 qBatch kBatch vBatch = none) : gqaOutput? qBatch kBatch vBatch hq hk hv lq eq eq ev = none
TensorGuard.ChunkSplit.sum_append	lean/TensorGuard/ChunkSplit/lean45sum_append	(xs ys : List Nat) : sum (xs ++ ys) = sum xs + sum ys
TensorGuard.ChunkSplit.prod_append	lean/TensorGuard/ChunkSplit/lean45prod_append	(xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.ChunkSplit.splitValid_iff	lean/TensorGuard/ChunkSplit/lean82splitValid_iff	(s : AxisShape) (sections : List Nat) : splitSectionsValid s sections = true <-> sum sections = s.axisSize
TensorGuard.ChunkSplit.split_list_mismatch_flagged	lean/TensorGuard/ChunkSplit/lean87split_list_mismatch_flagged	(s : AxisShape) (sections : List Nat) (h : sum sections != s.axisSize) : splitSectionsValid s sections = false
TensorGuard.ChunkSplit.axisConcat_reconstruct	lean/TensorGuard/ChunkSplit/lean91axisConcat_reconstruct	(s : AxisShape) (sections : List Nat) (h : sum sections = s.axisSize) : (concatShape s sections).axisSize = s.axisSize
TensorGuard.ChunkSplit.splitConcat_shape	lean/TensorGuard/ChunkSplit/lean91splitConcat_shape	(s : AxisShape) (sections : List Nat) (h : splitSectionsValid s sections = true) : toList (concatShape s sections) = toList s
TensorGuard.ChunkSplit.splitConcat_numel	lean/TensorGuard/ChunkSplit/lean99splitConcat_numel	(s : AxisShape) (sections : List Nat) (h : splitSectionsValid s sections = true) : prod (toList (concatShape s sections)) = prod (toList s)
TensorGuard.ChunkSplit.split_int_even_example	lean/TensorGuard/ChunkSplit/lean154split_int_even_example	splitIntSizes 13 6 = some [6, 6, 1]
TensorGuard.ChunkSplit.split_int_tail_example	lean/TensorGuard/ChunkSplit/lean158split_int_tail_example	splitIntSizes 10 3 = some [3, 3, 3, 1]

TensorGuard.ChunkSplit.split_int_zero_axis	TensorGuard.ChunkSplit.split_int_zero_axis	example : splitIntSizes 0 0 = some [0]
TensorGuard.ChunkSplit.split_list	TensorGuard.ChunkSplit	plan : 166 with_empty_section_valid : splitSectionsValid { pre
TensorGuard.ChunkSplit.split_list_mismatch	TensorGuard.ChunkSplit	plan : 170 mismatch_example : splitSectionsValid { pre
TensorGuard.ChunkSplit.chunk_uneven	TensorGuard.ChunkSplit	plan : 174 even_example : chunkSizes 10 3 = some [4, 4, 2]
TensorGuard.ChunkSplit.chunk_many_sections	TensorGuard.ChunkSplit	plan : 178 many_sections_example : chunkSizes 13 6 = some [3, 3, 3, 3, 1]
TensorGuard.ChunkSplit.chunk_fewer_than_requested	TensorGuard.ChunkSplit	plan : 182 fewer_than_requested_example : chunkSizes 5 8 = some [1, 1, 1, 1, 1]
TensorGuard.ChunkSplit.chunk_fewer_than_requested_len	TensorGuard.ChunkSplit	plan : 186 fewer_than_requested_len : (match chunkSizes 5 8 with some xs => xs.length none => 0) = 5
TensorGuard.ChunkSplit.chunk_zero_axis_returns_requested_emptyies	TensorGuard.ChunkSplit	plan : 190 zero_axis_returns_requested_emptyies : chunkSizes 0 3 = some [0, 0, 0]
TensorGuard.ChunkSplit.split_concat_and_reconstruct	TensorGuard.ChunkSplit	plan : 194 split_concat_and_reconstruct_example : toList (concatShape { pre
TensorGuard.ChunkSplit.chunk_concat_and_reconstruct	TensorGuard.ChunkSplit	plan : 199 chunk_concat_and_reconstruct_example : toList (concatShape { pre
TensorGuard.NamedTensor.containsNamed	TensorGuard.NamedTensor	plan : 200 containsNamed_head (n : Nat) (rest : List AxisName) : containsNamed n (named n :: rest) = true
TensorGuard.NamedTensor.uniqueNamed	TensorGuard.NamedTensor	plan : 204 uniqueNamed_head (n : Nat) (rest : List AxisName) : uniqueNamed (named n :: named n :: rest) = false
TensorGuard.NamedTensor.uniqueNamed_allows_repeated_anon	TensorGuard.NamedTensor	plan : 209 uniqueNamed_allows_repeated_anon (rest : List AxisName) : uniqueNamed (anon :: anon :: rest) = uniqueNamed rest
TensorGuard.NamedTensor.refine_existing_name_preserved	TensorGuard.NamedTensor	plan : 216 refineExisting_name_preserved (n : Nat) : refineAxis? (named n) (named n) = some (named n)
TensorGuard.NamedTensor.refine_rejected	TensorGuard.NamedTensor	plan : 220 refine_rejected (old new : Nat) (h : old != new) : refineAxis? (named old) (named new) = none
TensorGuard.NamedTensor.refine_rejected_anon	TensorGuard.NamedTensor	plan : 225 refine_rejected_anon (n : Nat) : refineAxis? (named n) anon = none
TensorGuard.NamedTensor.refine_duplicate_requested_rejected	TensorGuard.NamedTensor	plan : 229 refine_duplicate_requested_rejected (shape : List Nat) (current requested : List AxisName) (hdup : uniqueNamed requested = false) : refineNames? shape current requested = none
TensorGuard.NamedTensor.refine_duplicate_current_rejected	TensorGuard.NamedTensor	plan : 238 refine_duplicate_current_rejected (shape : List Nat) (current requested : List AxisName) (hdup : uniqueNamed current = false) : refineNames? shape current requested = none
TensorGuard.NamedTensor.refine_shape_preserved	TensorGuard.NamedTensor	plan : 246 refine_shape_preserved (shape : List Nat) (current requested : List AxisName) (out : NamedShape) (h : refineNames? shape current requested = some out) : out.shape = shape
TensorGuard.NamedTensor.refine_fill_anon	TensorGuard.NamedTensor	plan : 264 refine_fill_anon_example : refineNames? [2, 3] [anon, anon] [named 0, named 1] = some { shape
TensorGuard.NamedTensor.refine_preserve_existing	TensorGuard.NamedTensor	plan : 269 refine_preserve_existing_example : refineNames? [2, 3] [named 0, named 1] [named 0, named 1] = some { shape

TensorGuard.NamedTensor.refine_duplicate_names	NamedTensor.refine_duplicate_names	NamedTensor.refine_duplicate_names lean1174 state_names_rejected : refineNames? [2, 3] [anon, anon] [named 0, named 0] = none
TensorGuard.NamedTensor.existing_name	NamedTensor.existing_name	NamedTensor.existing_name lean1231 dim_preserved (shape : List Nat) (current : List AxisName) (n d : Nat) (h : lookupDimByName n shape current = some d) : targetDim shape current (named n) = d
TensorGuard.NamedTensor.fresh_name	NamedTensor.fresh_name	NamedTensor.fresh_name lean1237 inserts_singleton (shape : List Nat) (current : List AxisName) (n : Nat) (h : lookupDimByName n shape current = none) : targetDim shape current (named n) = 1
TensorGuard.NamedTensor.anon_target	NamedTensor.anon_target	NamedTensor.anon_target lean1243 inserts_singleton (shape : List Nat) (current : List AxisName) : targetDim shape current anon = 1
TensorGuard.NamedTensor.align_names	NamedTensor.align_names	NamedTensor.align_names lean1248 dim_preserved (shape : List Nat) (current target : List AxisName) (out : NamedShape) (h : alignTo? shape current target = some out) : out.names = target
TensorGuard.NamedTensor.align_duplicate_target	NamedTensor.align_duplicate_target	NamedTensor.align_duplicate_target lean1260 state_target_rejected (shape : List Nat) (current target : List AxisName) (hdup : uniqueNamed target = false) : alignTo? shape current target = none
TensorGuard.NamedTensor.align_duplicate_current	NamedTensor.align_duplicate_current	NamedTensor.align_duplicate_current lean1268 state_current_rejected (shape : List Nat) (current target : List AxisName) (hdup : uniqueNamed current = false) : alignTo? shape current target = none
TensorGuard.NamedTensor.align_underinput	NamedTensor.align_underinput	NamedTensor.align_underinput lean1275 underinput_rejected (d : Nat) (restShape : List Nat) (currentTail target : List AxisName) : alignTo? (d :: restShape) (anon :: currentTail) target = none
TensorGuard.NamedTensor.align_perexample	NamedTensor.align_perexample	NamedTensor.align_perexample lean1281 align_perexample : alignTo? [2, 3] [named 0, named 1] [named 1, named 0] = some { shape
TensorGuard.NamedTensor.align_singleton	NamedTensor.align_singleton	NamedTensor.align_singleton lean1288 align_singleton_insert_example : alignTo? [2, 3] [named 0, named 1] [named 0, named 2, named 1] = some { shape
TensorGuard.NamedTensor.align_andanon_target	NamedTensor.align_andanon_target	NamedTensor.align_andanon_target lean1293 align_andanon_target_insert_example : alignTo? [2, 3] [named 0, named 1] [named 0, anon, named 1] = some { shape
TensorGuard.NamedTensor.align_mismatch	NamedTensor.align_mismatch	NamedTensor.align_mismatch lean1298 align_mismatch_name_rejected : alignTo? [2, 3] [named 0, named 1] [named 0] = none
TensorGuard.NamedTensor.align_duplicate_names	NamedTensor.align_duplicate_names	NamedTensor.align_duplicate_names lean1302 state_names_rejected : alignTo? [2, 3] [named 0, named 1] [named 0, named 0, named 1] = none
TensorGuard.GridSample.gridSample2DValid	GridSample.gridSample2DValid	GridSample.gridSample2DValid lean1315 gridSample2DValid_iff (n h w gn coord : Nat) : gridSample2DValid n h w gn coord = true <-> (((0 < h /\ 0 < w) /\ gn = n) /\ coord = 2)
TensorGuard.GridSample.gridSample3DValid	GridSample.gridSample3DValid	GridSample.gridSample3DValid lean1315 gridSample3DValid_iff (n d h w gn coord : Nat) : gridSample3DValid n d h w gn coord = true <-> (((((0 < d /\ 0 < h) /\ 0 < w) /\ gn = n) /\ coord = 3)
TensorGuard.GridSample.gridSample2DValidLink	GridSample.gridSample2DValidLink	GridSample.gridSample2DValidLink lean1315 gridSample2D_valid_link (n c h w gn oh ow coord : Nat) (hvalid : gridSample2DValid n h w gn coord = true) : gridSample? [n, c, h, w] [gn, oh, ow, coord] = some [n, c, oh, ow]

TensorGuard.GridSample.gridSample	3Dn/ValidTensorGuard	(GridSampleSample6)3D_valid_link (n c d h w gn od oh ow coord : Nat) (hvalid : gridSample3DValid n d h w gn coord = true) : gridSample? [n, c, d, h, w] [gn, od, oh, ow, coord] = some [n, c, od, oh, ow]
TensorGuard.GridSample.gridSample	2Dn/ValidTensorGuard	(GridSampleSample7)2D_invalid_link (n c h w gn oh ow coord : Nat) (hvalid : gridSample2DValid n h w gn coord = false) : gridSample? [n, c, h, w] [gn, oh, ow, coord] = none
TensorGuard.GridSample.gridSample	3Dn/ValidTensorGuard	(GridSampleSample8)3D_invalid_link (n c d h w gn od oh ow coord : Nat) (hvalid : gridSample3DValid n d h w gn coord = false) : gridSample? [n, c, d, h, w] [gn, od, oh, ow, coord] = none
TensorGuard.GridSample.gridSample	2Dn/ValidTensorShape	(GridSampleSample9)2D_output_shape (n c h w oh ow : Nat) (hh : 0 < h) (hw : 0 < w) : gridSample? [n, c, h, w] [n, oh, ow, 2] = some [n, c, oh, ow]
TensorGuard.GridSample.gridSample	3Dn/ValidTensorShape	(GridSampleSample9B)3D_output_shape (n c d h w od oh ow : Nat) (hd : 0 < d) (hh : 0 < h) (hw : 0 < w) : gridSample? [n, c, d, h, w] [n, od, oh, ow, 3] = some [n, c, od, oh, ow]
TensorGuard.GridSample.gridSample	2Dn/ValidTensorRank	(GridSampleSample9A)2D_output_rank (n c oh ow : Nat) : [n, c, oh, ow].length = 4
TensorGuard.GridSample.gridSample	3Dn/ValidTensorRank	(GridSampleSample10)3D_output_rank (n c od oh ow : Nat) : [n, c, od, oh, ow].length = 5
TensorGuard.GridSample.gridSample	InvalidTensorGuard	(GridSampleSample11)7_wrong_input_rank_rejected (n c h gn gh coord : Nat) : gridSample? [n, c, h] [gn, gh, coord] = none
TensorGuard.GridSample.gridSample	InvalidTensorGuard	(GridSampleSample11A)grid_rank_mismatch_rejected (n c h w gn oh ow coord extra : Nat) : gridSample? [n, c, h, w] [gn, oh, ow, coord, extra] = none
TensorGuard.GridSample.gridSample	2Dn/ValidTensorCoord	(GridSampleSample11B)2D_coord_dim_flagged (n c h w oh ow : Nat) : gridSample? [n, c, h, w] [n, oh, ow, 3] = none
TensorGuard.GridSample.gridSample	3Dn/ValidTensorCoord	(GridSampleSample11C)3D_coord_dim_flagged (n c d h w od oh ow : Nat) : gridSample? [n, c, d, h, w] [n, od, oh, ow, 2] = none
TensorGuard.GridSample.gridSample	2Dn/ValidTensorHeight	(GridSampleSample12)2D_zero_height_flagged (n c w oh ow : Nat) : gridSample? [n, c, 0, w] [n, oh, ow, 2] = none
TensorGuard.GridSample.gridSample	2Dn/ValidTensorWidth	(GridSampleSample12A)2D_zero_width_flagged (n c h oh ow : Nat) : gridSample? [n, c, h, 0] [n, oh, ow, 2] = none
TensorGuard.GridSample.gridSample	3Dn/ValidTensorDepth	(GridSampleSample13)3D_zero_depth_flagged (n c h w od oh ow : Nat) : gridSample? [n, c, 0, h, w] [n, od, oh, ow, 3] = none
TensorGuard.GridSample.gridSample	3Dn/ValidTensorHeight	(GridSampleSample13A)3D_zero_height_flagged (n c d w od oh ow : Nat) : gridSample? [n, c, d, 0, w] [n, od, oh, ow, 3] = none
TensorGuard.GridSample.gridSample	3Dn/ValidTensorWidth	(GridSampleSample13B)3D_zero_width_flagged (n c d h od oh ow : Nat) : gridSample? [n, c, d, h, 0] [n, od, oh, ow, 3] = none
TensorGuard.GridSample.gridSample	2Dn/BatchTensorCoord	(GridSampleSample14)2D_batch_mismatch_flagged (n gn c h w oh ow : Nat) (hne : gn != n) : gridSample? [n, c, h, w] [gn, oh, ow, 2] = none

TensorGuard.GridSample.gridSample	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSample accepts_empty_output_grid (n c h w ow : Nat) (hh : 0 < h) (hw : 0 < w) : gridSample? [n, c, h, w] [n, 0, ow, 2] = some [n, c, 0, ow]
TensorGuard.GridSample.affineGrid2DValid	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleValid_iff (tn rows cols n c h w : Nat) : affineGrid2DValid tn rows cols n c h w = true <-> ((((((0 < n /\ 0 < c) /\ 0 < h) /\ 0 < w) /\ 0 < tn) /\ tn = n) /\ rows = 2) /\ cols = 3)
TensorGuard.GridSample.affineGrid3DValid	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleValid_iff (tn rows cols n c d h w : Nat) : affineGrid3DValid tn rows cols n c d h w = true <-> ((((((0 < n /\ 0 < c) /\ 0 < d) /\ 0 < h) /\ 0 < w) /\ 0 < tn) /\ tn = n) /\ rows = 3) /\ cols = 4)
TensorGuard.GridSample.affineGrid2DValidTensorGuard	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleValid_link (tn rows cols n c h w : Nat) (hvalid : affineGrid2DValid tn rows cols n c h w = true) : affineGrid? [tn, rows, cols] [n, c, h, w] = some [n, h, w, 2]
TensorGuard.GridSample.affineGrid3DValidTensorGuard	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleValid_link (tn rows cols n c d h w : Nat) (hvalid : affineGrid3DValid tn rows cols n c d h w = true) : affineGrid? [tn, rows, cols] [n, c, d, h, w] = some [n, d, h, w, 3]
TensorGuard.GridSample.affineGrid2DInvalidTensorGuard	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleInvalid_link (tn rows cols n c h w : Nat) (hvalid : affineGrid2DValid tn rows cols n c h w = false) : affineGrid? [tn, rows, cols] [n, c, h, w] = none
TensorGuard.GridSample.affineGrid3DInvalidTensorGuard	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleInvalid_link (tn rows cols n c d h w : Nat) (hvalid : affineGrid3DValid tn rows cols n c d h w = false) : affineGrid? [tn, rows, cols] [n, c, d, h, w] = none
TensorGuard.GridSample.affineGrid2DOutputTensorGuard	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleOutput_shape (n c h w : Nat) (hn : 0 < n) (hc : 0 < c) (hh : 0 < h) (hw : 0 < w) : affineGrid? [n, 2, 3] [n, c, h, w] = some [n, h, w, 2]
TensorGuard.GridSample.affineGrid3DOutputTensorGuard	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleOutput_shape (n c d h w : Nat) (hn : 0 < n) (hc : 0 < c) (hd : 0 < d) (hh : 0 < h) (hw : 0 < w) : affineGrid? [n, 3, 4] [n, c, d, h, w] = some [n, d, h, w, 3]
TensorGuard.GridSample.affineGrid2DOutputRankTensorGuard	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleOutput_rank (n h w : Nat) : [n, h, w, 2].length = 4
TensorGuard.GridSample.affineGrid3DOutputRankTensorGuard	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleOutput_rank (n d h w : Nat) : [n, d, h, w, 3].length = 5
TensorGuard.GridSample.affineGrid_sizeTensorReject	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSample_size_rank_rejected (tn rows cols n c h : Nat) : affineGrid? [tn, rows, cols] [n, c, h] = none
TensorGuard.GridSample.affineGrid_thetaTensorReject	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSample_theta_rank_rejected (n c h w : Nat) : affineGrid? [n, 2] [n, c, h, w] = none
TensorGuard.GridSample.affineGrid2DValidTensorGuardFlag	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleValid_theta_rows_flagged (n c h w : Nat) : affineGrid? [n, 3, 3] [n, c, h, w] = none
TensorGuard.GridSample.affineGrid2DValidTensorGuardFlag	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleValid_theta_cols_flagged (n c h w : Nat) : affineGrid? [n, 2, 4] [n, c, h, w] = none
TensorGuard.GridSample.affineGrid3DValidTensorGuardFlag	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleValid_theta_rows_flagged (n c d h w : Nat) : affineGrid? [n, 2, 4] [n, c, d, h, w] = none
TensorGuard.GridSample.affineGrid3DValidTensorGuardFlag	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleValid_theta_cols_flagged (n c d h w : Nat) : affineGrid? [n, 3, 3] [n, c, d, h, w] = none
TensorGuard.GridSample.affineGrid2DValidTensorGuardBatch	learn/TensorGuard/	GridSampleGridSample	GridSampleGridSampleValid_size_batch_positive_required (c h w : Nat) : affineGrid? [0, 2, 3] [0, c, h, w] = none

TensorGuard.GridSample.affineGrid2D_size/TensorGuard/ChannelSampleAffineGrid2D	size_channel_positive_required (n h w : Nat) : affineGrid? [n, 2, 3] [n, 0, h, w] = none
TensorGuard.GridSample.affineGrid2D_size/TensorGuard/ChannelSampleAffineGrid2D	size_height_positive_required (n c w : Nat) : affineGrid? [n, 2, 3] [n, c, 0, w] = none
TensorGuard.GridSample.affineGrid2D_size/TensorGuard/ChannelSampleAffineGrid2D	size_width_positive_required (n c h : Nat) : affineGrid? [n, 2, 3] [n, c, h, 0] = none
TensorGuard.GridSample.affineGrid3D_size/TensorGuard/ChannelSampleAffineGrid3D	size_depth_positive_required (n c h w : Nat) : affineGrid? [n, 3, 4] [n, c, 0, h, w] = none
TensorGuard.GridSample.affineGrid_theta/TensorGuard/ChannelSampleAffineGrid	theta_batch_positive_required (c h w : Nat) : affineGrid? [0, 2, 3] [1, c, h, w] = none
TensorGuard.GridSample.affineGrid_batch/TensorGuard/ChannelSampleAffineGrid	batch_mismatch_flagged (tn n c h w : Nat) (hne : tn != n) : affineGrid? [tn, 2, 3] [n, c, h, w] = none
TensorGuard.Distributions.bcDim_some/TensorGuard/DistributionsDilemma	some (n : Nat) : bcDim? n n = some n
TensorGuard.Distributions.bcDim_one_left/TensorGuard/DistributionsDilemma	one_left (n : Nat) : bcDim? 1 n = some n
TensorGuard.Distributions.bcDim_one_right/TensorGuard/DistributionsDilemma	one_right (n : Nat) : bcDim? n 1 = some n
TensorGuard.Distributions.bcDim_incompatible/TensorGuard/DistributionsDilemma	incompatible_example : bcDim? 2 3 = none
TensorGuard.Distributions.broadcast_shape/TensorGuard/DistributionsDilemma	example : broadcastShapes? [2, 1, 3] [1, 4, 3] = some [2, 4, 3]
TensorGuard.Distributions.broadcast_shape/TensorGuard/DistributionsDilemma	incompatible_example : broadcastShapes? [2] [3] = none
TensorGuard.Distributions.normal_broadcast/TensorGuard/DistributionsDilemma	broadcast_output : normal? [2, 1, 3] [1, 4, 3] = some { batch
TensorGuard.Distributions.normal_broadcast/TensorGuard/DistributionsDilemma	broadcast_rejected : normal? [2] [3] = none
TensorGuard.Distributions.categorical_batch/TensorGuard/DistributionsDilemma	batch_drops_category_dim : categorical? [2, 3, 4] = some { batch
TensorGuard.Distributions.categorical_rank/TensorGuard/DistributionsDilemma	empty_rank_rejected : categorical? [] = none
TensorGuard.Distributions.categorical_rank/TensorGuard/DistributionsDilemma	zero_categories_rejected : categorical? [2, 0] = none
TensorGuard.Distributions.mvn_batch/TensorGuard/DistributionsDilemma	event_output : multivariateNormal? [4, 3, 5] [1, 3, 5, 5] = some { batch
TensorGuard.Distributions.mvn_matrix/TensorGuard/DistributionsDilemma	square_rejected : multivariateNormal? [3] [3, 4] = none
TensorGuard.Distributions.mvn_event/TensorGuard/DistributionsDilemma	mismatch_rejected : multivariateNormal? [3] [4, 4] = none
TensorGuard.Distributions.mvn_batch/TensorGuard/DistributionsDilemma	broadcast_rejected : multivariateNormal? [2, 3] [4, 3, 3] = none
TensorGuard.Distributions.independent/TensorGuard/DistributionsDilemma	moves_batch_to_event : independent? { batch
TensorGuard.Distributions.independent/TensorGuard/DistributionsDilemma	preserves_when_zero : independent? { batch
TensorGuard.Distributions.independent/TensorGuard/DistributionsDilemma	too_many_rejected : independent? { batch
TensorGuard.Distributions.normal_logProb/TensorGuard/DistributionsDilemma	Prob_broadcasts_value : logProb? { batch

TensorGuard.Distributions.categorical_logProb	lean/TensorGuard/Distributions.lean	203 logProb_drops_no_event : logProb? { batch
TensorGuard.Distributions.mvn_logProb	lean/TensorGuard/Distributions.lean	208 logProb_drops_event_dim : logProb? { batch
TensorGuard.Distributions.logProb_bad	lean/TensorGuard/Distributions.lean	213 bad_value_broadcast_rejected : logProb? { batch
TensorGuard.Distributions.reshapeShape	lean/TensorGuard/Distributions.lean	327 reshape_output : reshapeShape? [4, 2, 3] [2, 3] [6] = some [4, 6]
TensorGuard.Distributions.reshapeShape_wrong_suffix	lean/TensorGuard/Distributions.lean	331 reshape_wrong_suffix_rejected : reshapeShape? [4, 2, 3] [3, 3] [9] = none
TensorGuard.Distributions.reshapeShape_mismatch	lean/TensorGuard/Distributions.lean	335 reshape_numel_mismatch_rejected : reshapeShape? [4, 2, 3] [2, 3] [5] = none
TensorGuard.Distributions.transformed_and_identity_preserves_shape	lean/TensorGuard/Distributions.lean	339 transformed_identity_preserves_shape : transformed? { batch
TensorGuard.Distributions.transformed_and_reshape_event_shape	lean/TensorGuard/Distributions.lean	344 reshape_event_shape : transformed? { batch
TensorGuard.Distributions.transformed_and_reshape_reinterprets_batch	lean/TensorGuard/Distributions.lean	349 reshape_reinterprets_batch : transformed? { batch
TensorGuard.Distributions.transformed_and_composed_reshape_identity	lean/TensorGuard/Distributions.lean	354 composed_reshape_identity : transformed? { batch
TensorGuard.Distributions.transformed_and_wrong_domain_rejected	lean/TensorGuard/Distributions.lean	359 wrong_domain_rejected : transformed? { batch

5 All Lean Theorem and Lemma Statements

This census is intentionally broad: it lists every theorem and lemma statement found in committed Lean source. The namespace-qualified names are reconstructed from Lean namespace blocks rather than from file paths, so declarations such as `TensorGuard.V5.applyOp_sound_clamp` resolve even though they live in `lean/TensorGuard/SoundnessV5.lean`.

5.1 Declaration Count by Module

Module	Theorems	Lemmas
TensorGuard.AdaptivePool	6	0
TensorGuard.AssumeGuarantee	4	0
TensorGuard.AssumeGuaranteeExtended	25	0
TensorGuard.BatchNormRule	5	0
TensorGuard.BroadcastAddSound	4	0
TensorGuard.BroadcastChain	9	0
TensorGuard.CatRule	8	0
TensorGuard.CegarBound	4	0
TensorGuard.CegarInfeasible	5	0
TensorGuard.CegarPredicateSerialization	9	0
TensorGuard.ChunkSplit	19	0
TensorGuard.ContigFlow	22	0
TensorGuard.Conv1d	9	0
TensorGuard.Conv2d	9	0
TensorGuard.ConvTranspose	7	0
TensorGuard.CrossDomain	9	0
TensorGuard.DeviceDtype	27	0
TensorGuard.DevicePlacement	13	0
TensorGuard.Distributions	30	0

TensorGuard.DtypePromoteChain	12	0
TensorGuard.Einops	15	0
TensorGuard.EmbeddingRule	9	0
TensorGuard.Extended	9	0
TensorGuard.Flatten	6	0
TensorGuard.FragmentModes	5	0
TensorGuard.GradFlow	8	0
TensorGuard.GridSample	44	0
TensorGuard.LayerNormRule	6	0
TensorGuard.LinearRule	7	0
TensorGuard.MatmulSound	4	0
TensorGuard.NamedTensor	24	0
TensorGuard.Parity	3	0
TensorGuard.PhaseFlow	9	0
TensorGuard.PixelShuffle	5	0
TensorGuard.Pool2d	9	0
TensorGuard.RankTransfer	8	0
TensorGuard.RecurrentRule	18	0
TensorGuard.ReducedProduct	23	0
TensorGuard.ReshapeInfer	7	0
TensorGuard.SDPA	23	0
TensorGuard.SmtEncoding	24	0
TensorGuard.Soundness	7	0
TensorGuard.SoundnessV5	36	0
TensorGuard.SparseLayouts	21	0
TensorGuard.SubjectReduction	38	0
TensorGuard.Unflatten	6	0
TensorGuard.V5OperatorRules	32	0
TheoryCombination	61	0

5.2 Declaration Ledger

Qualified declaration	Source	Statement
TensorGuard.AdaptivePool.ap_rank	lean/TensorGuard/AdaptivePool.lean:32	$\text{AdaptivePool.rank}(n \text{ c oh ow} : \text{Nat}) : (\text{apShape } n \text{ c oh ow}).\text{length} = 4$
TensorGuard.AdaptivePool.ap_batch	lean/TensorGuard/AdaptivePool.lean:33	$\text{AdaptivePool.batch}(n \text{ c oh ow} : \text{Nat}) : (\text{apShape } n \text{ c oh ow}).\text{head?} = \text{some } n$
TensorGuard.AdaptivePool.ap_channel	lean/TensorGuard/AdaptivePool.lean:34	$\text{AdaptivePool.channel}(n \text{ c oh ow} : \text{Nat}) : (\text{apShape } n \text{ c oh ow}).\text{get? } 1 = \text{some } c$
TensorGuard.AdaptivePool.ap_spatial_h	lean/TensorGuard/AdaptivePool.lean:38	$\text{AdaptivePool.spatial}_h(n \text{ c oh ow} : \text{Nat}) : (\text{apShape } n \text{ c oh ow}).\text{get? } 2 = \text{some } oh$
TensorGuard.AdaptivePool.ap_spatial_w	lean/TensorGuard/AdaptivePool.lean:39	$\text{AdaptivePool.spatial}_w(n \text{ c oh ow} : \text{Nat}) : (\text{apShape } n \text{ c oh ow}).\text{get? } 3 = \text{some } ow$
TensorGuard.AdaptivePool.ap_idempotent	lean/TensorGuard/AdaptivePool.lean:40	$\text{AdaptivePool.idempotent}(n \text{ c h w} : \text{Nat}) : \text{apShape } n \text{ c h w} = [n, c, h, w]$

TensorGuard.applyChain_append	lean/TensorGuard	thAssumeApplyChainAppend : forall (c1 c2 : List Op) (s : Shape), applyChain (c1 ++ c2) s = (match applyChain c1 s with none => none some s' => applyChain c2 s') [], c2, s => by simp [applyChain] op :: r, c2, s => by simp [applyChain] cases h : applyOp op s with none => simp [h] some s' => simp [h] exact applyChain_append r c2 s' theorem ag_composition (c1 c2 : List Op) (k1 k2 : Contract) (h1 : satisfies c1 k1) (h2 : satisfies c2 k2) (link : forall s, k1.guarantee s -> k2.assume s) : satisfies (c1 ++ c2) <k1.assume, k2.guarantee>
TensorGuard.ag_composition	lean/TensorGuard	thAssumeGuaranteesComposition : forall (c1 c2 : List Op) (k1 k2 : Contract) (h1 : satisfies c1 k1) (h2 : satisfies c2 k2) (link : forall s, k1.guarantee s -> k2.assume s) : satisfies (c1 ++ c2) <k1.assume, k2.guarantee>
TensorGuard.satisfies_trivial	lean/TensorGuard	thAssumeSatisfiesTrivial : forall (c : List Op) (h : forall s, exists s', applyChain c s = some s') : satisfies c <fun _ => True, fun _ => True>
TensorGuard.satisfies_nil	lean/TensorGuard	thAssumeSatisfiesNil : forall s, satisfies [] <fun s => True, fun s => True>
TensorGuard.applyOpExt_sound_transpose	lean/TensorGuard	thAssumeApplyOpExtSoundTranspose : forall (dim0 dim1 : Nat) (s s' : Shape) (h : applyOpExt (.transpose dim0 dim1) s = some s') : dim0 < s.length /\ dim1 < s.length
TensorGuard.applyOpExt_sound_transpose_perm	lean/TensorGuard	thAssumeApplyOpExtSoundTransposePerm : forall (dim0 dim1 : Nat) (s s' : Shape) (h : applyOpExt (.transpose dim0 dim1) s = some s') : s' = s.swap dim0 dim1
TensorGuard.applyOpExt_sound_permute	lean/TensorGuard	thAssumeApplyOpExtSoundPermute : forall (perm : List Nat) (s s' : Shape) (h : applyOpExt (.permute perm) s = some s') : validPerm s perm = true
TensorGuard.applyOpExt_sound_relu	lean/TensorGuard	thAssumeApplyOpExtSoundRelu : forall (s s' : Shape) (h : applyOpExt .relu s = some s') : s' = s
TensorGuard.applyOpExt_sound_broadcast_add	lean/TensorGuard	thAssumeApplyOpExtSoundBroadcastAdd : forall (s s' : Shape) (h : applyOpExt .broadcast_add s = some s') : s' = s
TensorGuard.applyOpExt_sound_matmul	lean/TensorGuard	thAssumeApplyOpExtSoundMatmul : forall (s s' : Shape) (h : applyOpExt .matmul s = some s') : s.length >= 3 /\ s' = s
TensorGuard.applyOpExt_sound_sum	lean/TensorGuard	thAssumeApplyOpExtSoundSum : forall (axis : Nat) (s s' : Shape) (h : applyOpExt (.sum_reduce axis) s = some s') : axis < s.length
TensorGuard.applyOpExt_sound_mean_reduce	lean/TensorGuard	thAssumeApplyOpExtSoundMeanReduce : forall (axis : Nat) (s s' : Shape) (h : applyOpExt (.mean_reduce axis) s = some s') : axis < s.length
TensorGuard.applyOpExt_sound_cat	lean/TensorGuard	thAssumeApplyOpExtSoundCat : forall (axis : Nat) (s s' : Shape) (h : applyOpExt (.cat axis n) s = some s') : axis < s.length /\ n > 0
TensorGuard.applyOpExt_sound_expand	lean/TensorGuard	thAssumeApplyOpExtSoundExpand : forall (target : List Nat) (s s' : Shape) (h : applyOpExt (.expand target) s = some s') : s.length <= target.length
TensorGuard.applyOpExt_sound_gather	lean/TensorGuard	thAssumeApplyOpExtSoundGather : forall (axis : Nat) (s s' : Shape) (h : applyOpExt (.gather axis) s = some s') : axis < s.length
TensorGuard.applyOpExt_sound_embedding	lean/TensorGuard	thAssumeApplyOpExtSoundEmbedding : forall (vocab embed : Nat) (s s' : Shape) (h : applyOpExt (.embedding vocab embed) s = some s') : s.length = 1 /\ s.length = 2

TensorGuard.applyChainExt_append	lean/TensorGuard	thAssumeApplyChainExtAppend.lean:355 forall (c1 c2 : List OpExt) (s : Shape), applyChainExt (c1 ++ c2) s = (match applyChainExt c1 s with none => none some s' => applyChainExt c2 s') [], c2, s => by simp [applyChainExt] op :: r, c2, s => by simp [applyChainExt] cases h : applyOpExt op s with none => simp [h] some s' => simp [h] exact applyChainExt_append r c2 s' theorem ag_composition_ext (c1 c2 : List OpExt) (k1 k2 : ContractExt) (h1 : satisfiesExt c1 k1) (h2 : satisfiesExt c2 k2) (link : forall s, k1.guarantee s -> k2.assume s) : satisfiesExt (c1 ++ c2) <k1.assume, k2.guarantee>
TensorGuard.ag_composition_ext	lean/TensorGuard	thAssumeGuaranteeExtExtended.lean:376 List OpExt) (k1 k2 : ContractExt) (h1 : satisfiesExt c1 k1) (h2 : satisfiesExt c2 k2) (link : forall s, k1.guarantee s -> k2.assume s) : satisfiesExt (c1 ++ c2) <k1.assume, k2.guarantee>
TensorGuard.satisfiesExt_singleton	lean/TensorGuard	thAssumeGuaranteeExtSingleton.lean:411 (op : OpExt) (htot : forall s, exists s', applyOpExt op s = some s') : satisfiesExt [op] contract_trivial_ext
TensorGuard.relu_satisfies_trivial	lean/TensorGuard	thAssumeGuaranteeExtTrivial.lean:421 satisfiesExt [.relu] contract_trivial_ext
TensorGuard.broadcast_add_satisfies_trivial	lean/TensorGuard	thAssumeGuaranteeExtTrivial.lean:428 satisfiesExt [.broadcast_add] contract_trivial_ext
TensorGuard.relu_relu_compose	lean/TensorGuard	thAssumeGuaranteeExtComposed.lean:436 satisfiesExt ([.relu] ++ [.relu]) contract_trivial_ext
TensorGuard.applyOpExt_sound_conv2d	lean/TensorGuard	thAssumeApplyOpExtSound.lean:448 (out_ch : Nat) (s s' : Shape) (h : applyOpExt (.conv2d out_ch) s = some s') : s.length = 4
TensorGuard.applyOpExt_sound_reshape	lean/TensorGuard	thAssumeApplyOpExtSound.lean:470 (out : List Nat) (s s' : Shape) (h : applyOpExt (.reshape out) s = some s') : s.prod = listProd out
TensorGuard.applyOpExt_sound_unbind	lean/TensorGuard	thAssumeApplyOpExtSound.lean:480 (dim n : Nat) (s s' : Shape) (h : applyOpExt (.unbind dim n) s = some s') : dim < s.length /\ s.get dim = n /\ s' = s.remove dim
TensorGuard.applyOpExt_sound_view	lean/TensorGuard	thAssumeApplyOpExtSound.lean:492 (out : List Nat) (s s' : Shape) (h : applyOpExt (.view out) s = some s') : s.prod = listProd out
TensorGuard.applyOpExt_sound_view_ndict	lean/TensorGuard	thAssumeApplyOpExtSound.lean:503 verdict (out : List Nat) (s s' : Shape) (h : applyOpExt (.view out) s = some s') : s' = Shape.ofList out
TensorGuard.applyOpExt_sound_einsum	lean/TensorGuard	thAssumeApplyOpExtSound.lean:515 (out_rank : Nat) (s s' : Shape) (h : applyOpExt (.einsum out_rank) s = some s') : s.length >= out_rank
TensorGuard.operator_count	lean/TensorGuard	thAssumeGuaranteeExtOperatorCount.lean:535 operators.length = 17
TensorGuard.BatchNormRule.bn_prekernel	lean/TensorGuard	thBatchNormRule.lean:34 (n feat : Nat) (rest : List Nat) : bnShape n feat rest = n :: feat :: rest
TensorGuard.BatchNormRule.bn_numfeatures	lean/TensorGuard	thBatchNormRule.lean:36 (n feat : Nat) (rest : List Nat) : prod (bnShape n feat rest) = prod (n :: feat :: rest)
TensorGuard.BatchNormRule.featValid	lean/TensorGuard	thBatchNormRule.lean:43 (at numFeatures : Nat) : featValid feat numFeatures = true <-> feat = numFeatures
TensorGuard.BatchNormRule.feat_unflagged	lean/TensorGuard	thBatchNormRule.lean:47 flagged (feat numFeatures : Nat) (h : feat != numFeatures) : featValid feat numFeatures = false
TensorGuard.BatchNormRule.bn_channel	lean/TensorGuard	thBatchNormRule.lean:52 flex (n feat : Nat) (rest : List Nat) : (bnShape n feat rest).get? 1 = some feat

TensorGuard.Cegar.tight_below_naive	lean/TensorGuard	hCegarBright.lean90	naive (gains : List Nat) (U : Nat) (hU : discovered gains < U) : 1 + discovered gains < U + 1
TensorGuard.CegarU2.decideNew_safeSound	lean/TensorGuard	hCegarInfeasibleNew.lean58	safeSound (infeasible realBug : Bool) (h : feasibleJustifiesSafety infeasible realBug) : safeSound (decideNew infeasible) realBug
TensorGuard.CegarU2.decideOld_unsound	lean/TensorGuard	hCegarInfeasibleOld.lean71	unsound : exists infeasible realBug, feasibleJustifiesSafety infeasible realBug /\ decideOld infeasible = Verdict.safe /\ realBug = true
TensorGuard.CegarU2.fix_abstains	lean/TensorGuard	hCegarInfeasibleNew.lean82	fix_abstains : decideNew true = Verdict.abstain
TensorGuard.CegarU2.fix_keeps_safeSound	lean/TensorGuard	hCegarInfeasibleNew.lean84	fix_keeps_safe : decideNew false = Verdict.safe
TensorGuard.CegarU2.old_always_safe	lean/TensorGuard	hCegarInfeasibleNew.lean86	old_always_safe (b : Bool) : decideOld b = Verdict.safe
TensorGuard.CegarSerialized.python_recordTensorSerialized	lean/TensorGuard	hCegarPythonRecordSerialized.lean70	pythonRecordKeys = ["schema", "kind", "tensor", "axis", "value", "match_tensor", "match_axis", "divisor", "provenance"]
TensorGuard.CegarSerialized.python_kindTensorSerialized	lean/TensorGuard	hCegarPythonKindSerialized.lean85	pythonKindNames = ["DIM_EQ", "DIM_GT", "DIM_GE", "DIM_DIVISIBLE", "DIM_MATCH", "NDIM_EQ", "SHAPE_EQ"]
TensorGuard.CegarSerialized.serialized_applyTensorSerialized	lean/TensorGuard	hCegarSerializedApplyTensorSerialized.lean92	applyTensorSerialized preserves_membership (old fresh : List SerializedPredicate) : Extends old (old ++ fresh)
TensorGuard.CegarSerialized.serialized_applyTensorSerialized	lean/TensorGuard	hCegarSerializedApplyTensorSerialized.lean99	applyTensorSerialized mono (old fresh : List SerializedPredicate) : old.length <= (old ++ fresh).length
TensorGuard.CegarSerialized.serialized_applyTensorSerialized	lean/TensorGuard	hCegarSerializedApplyTensorSerialized.lean105	applyTensorSerialized prefix (previous added : List SerializedPredicate) : Extends previous (previous ++ added)
TensorGuard.CegarSerialized.serialized_applyTensorSerialized	lean/TensorGuard	hCegarSerializedApplyTensorSerialized.lean129	applyTensorSerialized abs : decideSerialized false = Verdict.abstain
TensorGuard.CegarSerialized.serialized_applyTensorSerialized	lean/TensorGuard	hCegarSerializedApplyTensorSerialized.lean132	applyTensorSerialized safe : decideSerialized true = Verdict.safe
TensorGuard.CegarSerialized.decideSerializedTensorSerialized	lean/TensorGuard	hCegarDecideSerializedTensorSerialized.lean138	decideSerialized (feasible realBug : Bool) (h : feasibleSerializedJustifiesSafety feasible realBug) : safeSound (decideSerialized feasible) realBug
TensorGuard.CegarSerialized.infeasibleTensorSerialized	lean/TensorGuard	hCegarInfeasibleSerializedTensorSerialized.lean147	infeasibleTensorSerialized safeSound_any_bug (realBug : Bool) : safeSound (decideSerialized false) realBug
TensorGuard.ChunkSplit.sum_append	lean/TensorGuard	hChunkSplit.lean45	sum_append (xs ys : List Nat) : sum (xs ++ ys) = sum xs + sum ys
TensorGuard.ChunkSplit.prod_append	lean/TensorGuard	hChunkSplit.lean50	prod_append (xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.ChunkSplit.splitValid_iff	lean/TensorGuard	hChunkSplitValid.lean82	splitValid_iff (s : AxisShape) (sections : List Nat) : splitSectionsValid s sections = true <-> sum sections = s.axisSize
TensorGuard.ChunkSplit.split_list	lean/TensorGuard	hChunkSplitList.lean87	split_list (s : AxisShape) (sections : List Nat) (h : sum sections != s.axisSize) : splitSectionsValid s sections = false
TensorGuard.ChunkSplit.axisConcat_reconstruct	lean/TensorGuard	hChunkSplitConcat.lean94	axisConcat_reconstruct (s : AxisShape) (sections : List Nat) (h : sum sections = s.axisSize) : (concatShape s sections).axisSize = s.axisSize

TensorGuard.ChunkSplit.splitConcat_shape	lean/TensorGuard/ChunkSplit/Concat101	shape (s : AxisShape) (sections : List Nat) (h : splitSectionsValid s sections = true) : toList (concatShape s sections) = toList s
TensorGuard.ChunkSplit.splitConcat_numel	lean/TensorGuard/ChunkSplit/Concat109	numel (s : AxisShape) (sections : List Nat) (h : splitSectionsValid s sections = true) : prod (toList (concatShape s sections)) = prod (toList s)
TensorGuard.ChunkSplit.split_int	lean/TensorGuard/ChunkSplit/Int154	even_example : splitIntSizes 13 6 = some [6, 6, 1]
TensorGuard.ChunkSplit.split_int	lean/TensorGuard/ChunkSplit/Int158	nil_example : splitIntSizes 10 3 = some [3, 3, 3, 1]
TensorGuard.ChunkSplit.split_int	lean/TensorGuard/ChunkSplit/Int162	zero_axis_example : splitIntSizes 0 0 = some [0]
TensorGuard.ChunkSplit.split_list	lean/TensorGuard/ChunkSplit/List166	with_empty_section_valid : splitSectionsValid { pre
TensorGuard.ChunkSplit.split_list	lean/TensorGuard/ChunkSplit/List170	mismatch_example : splitSectionsValid { pre
TensorGuard.ChunkSplit.chunk_uneven	lean/TensorGuard/ChunkSplit/Uneven174	ten_example : chunkSizes 10 3 = some [4, 4, 2]
TensorGuard.ChunkSplit.chunk_many_sections	lean/TensorGuard/ChunkSplit/ManySections178	sections_example : chunkSizes 13 6 = some [3, 3, 3, 3, 1]
TensorGuard.ChunkSplit.chunk_fewer_than_requested	lean/TensorGuard/ChunkSplit/FewerThanRequested182	chunkSizes 5 8 = some [1, 1, 1, 1, 1]
TensorGuard.ChunkSplit.chunk_fewer_than_requested	lean/TensorGuard/ChunkSplit/FewerThanRequested186	len : (match chunkSizes 5 8 with some xs => xs.length none => 0) = 5
TensorGuard.ChunkSplit.chunk_zero_axis	lean/TensorGuard/ChunkSplit/ZeroAxis190	zero_axis_returns_requested_emptyies : chunkSizes 0 3 = some [0, 0, 0]
TensorGuard.ChunkSplit.split_concat_reconstruct	lean/TensorGuard/ChunkSplit/ConcatReconstruct194	concat_reconstruct_example : toList (concatShape { pre
TensorGuard.ChunkSplit.chunk_concat_reconstruct	lean/TensorGuard/ChunkSplit/ConcatReconstruct198	concat_reconstruct_example : toList (concatShape { pre
TensorGuard.ContigFlow.keep_id	lean/TensorGuard/ContigFlow/Keep64	(b : Bool) : ctgStep b LayoutOp.keep = b
TensorGuard.ContigFlow.transpose	lean/TensorGuard/ContigFlow/Transpose65	not (b : Bool) : ctgStep b LayoutOp.transpose = !b
TensorGuard.ContigFlow.contiguous	lean/TensorGuard/ContigFlow/Contiguous66	true (b : Bool) : ctgStep b LayoutOp.contiguous = true
TensorGuard.ContigFlow.ctgRun_append	lean/TensorGuard/ContigFlow/RunAppend73	(b0 : Bool) (xs ys : List LayoutOp) : ctgRun b0 (xs ++ ys) = ctgRun (ctgRun b0 xs) ys
TensorGuard.ContigFlow.run_consistent	lean/TensorGuard/ContigFlow/RunConsistent80	contig (b0 : Bool) (rest : List LayoutOp) : ctgRun b0 (LayoutOp.contiguous :: rest) = ctgRun true rest
TensorGuard.ContigFlow.run_after	lean/TensorGuard/ContigFlow/RunAfter84	contig (b0 : Bool) (pre post : List LayoutOp) : ctgRun b0 (pre ++ LayoutOp.contiguous :: post) = ctgRun true post
TensorGuard.ContigFlow.run_transpose_involution	lean/TensorGuard/ContigFlow/RunTransposeInvolution89	transpose_involution (b0 : Bool) (rest : List LayoutOp) : ctgRun b0 (LayoutOp.transpose :: LayoutOp.transpose :: rest) = ctgRun b0 rest
TensorGuard.ContigFlow.run_allKeep	lean/TensorGuard/ContigFlow/RunAllKeep97	(b0 : Bool) (xs : List LayoutOp) (h : forall op in xs, op = LayoutOp.keep) : ctgRun b0 xs = b0
TensorGuard.ContigFlow.LayoutAlgebra.isContiguous	lean/TensorGuard/ContigFlow/LayoutAlgebra927	isContiguous (shape : List Nat) (off : Nat) : isContiguous { shape

TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor231HWTail_viewable (n c h w off : Nat) : canViewNCHWTail { shape
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor241HWTail_view_layout (n c h w off : Nat) : viewNCHWTail { shape
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor251preserves_storageOffset (l : Layout) : (transpose01 l).storageOffset = l.storageOffset
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor261permuteNCHWtoNHWC_preserves_storageOffset (l : Layout) : (permuteNCHWtoNHWC l).storageOffset = l.storageOffset
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor268strides : rowMajor4.strides = [60, 20, 5, 1]
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor271view_tail : viewNCHWTail rowMajor4 = some { shape
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor275is_is_anonical_channels_last : isCanonicalChannelsLast4 channelsLast4 = true
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor278isContiguous_row_major_contiguous : isContiguous channelsLast4 = false
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor281channelsLast4_tail_not_viewable : canViewNCHWTail channelsLast4 = false
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor285channelsLast4_view_tail_rejected : viewNCHWTail channelsLast4 = none
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor287channel_rowMajor_offset : narrowChannel 1 2 rowMajor4 = some { shape
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor289channel_channelsLast_offset : narrowChannel 1 2 channelsLast4 = some { shape
TensorGuard.ContigFlow.LayoutAlgebra/TensorGuard/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	lean/ContigFlow/Algebra/TensorGuard/ContigFlow/Algebra	ContigFlowMajor298C_abstains_canonicalChannelsLast4_degenerateC_abstains : canonicalChannelsLast4Strides [2, 1, 4, 5] = none
TensorGuard.Conv1d.div_mono	lean/TensorGuard/Conv1d/Algebra/TensorGuard/Conv1d/Algebra	Conv1dMajor25no {a b k : Nat} (h : a <= b) : a / k <= b / k
TensorGuard.Conv1d.convOut_identity	lean/TensorGuard/Conv1d/Algebra/TensorGuard/Conv1d/Algebra	Conv1dMajor37out_identity (input : Nat) (h : 1 <= input) : convOut input 0 1 1 = input
TensorGuard.Conv1d.convOut_stride_one	lean/TensorGuard/Conv1d/Algebra/TensorGuard/Conv1d/Algebra	Conv1dMajor41out_stride_one (input pad e : Nat) : convOut input pad e 1 = input + 2 * pad - e + 1
TensorGuard.Conv1d.convOut_mono	lean/TensorGuard/Conv1d/Algebra/TensorGuard/Conv1d/Algebra	Conv1dMajor45out_mono (i? i? pad e stride : Nat) (h : i? <= i?) : convOut i? pad e stride <= convOut i? pad e stride
TensorGuard.Conv1d.convOut_le	lean/TensorGuard/Conv1d/Algebra/TensorGuard/Conv1d/Algebra	Conv1dMajor52out_le (input pad e stride : Nat) (he : 1 <= e) (hv : e <= input + 2 * pad) : convOut input pad e stride <= input + 2 * pad
TensorGuard.Conv1d.convOut_pos	lean/TensorGuard/Conv1d/Algebra/TensorGuard/Conv1d/Algebra	Conv1dMajor60out_pos (input pad e stride : Nat) : 1 <= convOut input pad e stride
TensorGuard.Conv1d.conv1d_rank	lean/TensorGuard/Conv1d/Algebra/TensorGuard/Conv1d/Algebra	Conv1dMajor63rank (n cout lOut : Nat) : (conv1dShape n cout lOut).length = 3
TensorGuard.Conv1d.conv1d_batch	lean/TensorGuard/Conv1d/Algebra/TensorGuard/Conv1d/Algebra	Conv1dMajor66batch (n cout lOut : Nat) : (conv1dShape n cout lOut).head? = some n
TensorGuard.Conv1d.conv1d_channels	lean/TensorGuard/Conv1d/Algebra/TensorGuard/Conv1d/Algebra	Conv1dMajor67channels (n cout lOut : Nat) : (conv1dShape n cout lOut).get? 1 = some cout
TensorGuard.Conv2d.div_mono	lean/TensorGuard/Conv2d/Algebra/TensorGuard/Conv2d/Algebra	Conv2dMajor45no {a b k : Nat} (h : a <= b) : a / k <= b / k
TensorGuard.Conv2d.convOut_identity	lean/TensorGuard/Conv2d/Algebra/TensorGuard/Conv2d/Algebra	Conv2dMajor61out_identity (input : Nat) (h : 1 <= input) : convOut input 0 1 1 = input
TensorGuard.Conv2d.convOut_stride_one	lean/TensorGuard/Conv2d/Algebra/TensorGuard/Conv2d/Algebra	Conv2dMajor68out_stride_one (input pad e : Nat) : convOut input pad e 1 = input + 2 * pad - e + 1

TensorGuard.Conv2d.convOut_mono	lean/TensorGuard	dt/Conv2dConvOut_mono (i? i? pad e stride : Nat) (h : i? <= i?) : convOut i? pad e stride <= convOut i? pad e stride	
TensorGuard.Conv2d.convOut_le	lean/TensorGuard	dt/Conv2dConvOut_le (input pad e stride : Nat) (he : 1 <= e) (hs : 1 <= stride) (hv : e <= input + 2 * pad) : convOut input pad e stride <= input + 2 * pad	
TensorGuard.Conv2d.convOut_pos	lean/TensorGuard	dt/Conv2dConvOut_pos (input pad e stride : Nat) : 1 <= convOut input pad e stride	
TensorGuard.Conv2d.conv2d_rank	lean/TensorGuard	dt/Conv2dConv2d_rank (n cout hOut wOut : Nat) : (conv2dShape n cout hOut wOut).length = 4	
TensorGuard.Conv2d.conv2d_batch	lean/TensorGuard	dt/Conv2dConv2d_batch (n cout hOut wOut : Nat) : (conv2dShape n cout hOut wOut).head? = some n	
TensorGuard.Conv2d.conv2d_channels	lean/TensorGuard	dt/Conv2dConv2d_channels (n cout hOut wOut : Nat) : (conv2dShape n cout hOut wOut).get? 1 = some cout	
TensorGuard.ConvTranspose.ctOut_ilean/TensorGuard	dt/ConvTransOpctOut_ilean18	(input : Nat) (h : 1 <= input) : ctOut input 1 0 0 0 = input	
TensorGuard.ConvTranspose.ctOut_lean/TensorGuard	dt/ConvTransOpctOut_lean42	(input stride dk op : Nat) : ctOut input stride 0 dk op = (input - 1) * stride + dk + op + 1	
TensorGuard.ConvTranspose.ctOut_lean/TensorGuard	dt/ConvTransOpctOut_lean46	i? stride pad dk op : Nat) (h : i? <= i?) : ctOut i? stride pad dk op <= ctOut i? stride pad dk op	
TensorGuard.ConvTranspose.ctOut_lean/TensorGuard	dt/ConvTransOpctOut_lean57	(input stride dk op : Nat) (hs : 1 <= stride) (h : 1 <= input) : input <= ctOut input stride 0 dk op	
TensorGuard.ConvTranspose.ct_rank	lean/TensorGuard	dt/ConvTransOpct_rank_lean66	(n cout lOut : Nat) : (ctShape n cout lOut).length = 3
TensorGuard.ConvTranspose.ct_batch	lean/TensorGuard	dt/ConvTransOpct_batch_lean67	(n cout lOut : Nat) : (ctShape n cout lOut).head? = some n
TensorGuard.ConvTranspose.ct_channels	lean/TensorGuard	dt/ConvTransOpct_channels_lean68	(n cout lOut : Nat) : (ctShape n cout lOut).get? 1 = some cout
TensorGuard.CrossDomain.transfer	skaniff/TensorGuard	dt/CrossDomaintransfer_lean53	iff_shape_eq (pre post : List Nat) : transferHolds pre post <-> pre = post
TensorGuard.CrossDomain.transfer	unsat/TensorGuard	dt/CrossDomaintransfer_lean54	iff_shape_ne (pre post : List Nat) : not transferHolds pre post <-> pre != post
TensorGuard.CrossDomain.transfer	device/TensorGuard	dt/CrossDomaintransfer_lean64	ce_free (pre post : List Nat) (h : pre = post) : forall _dpre _dpost : Dev, transferHolds pre post
TensorGuard.CrossDomain.transfer	lean/TensorGuard	dt/CrossDomaintransfer_lean70	false_positive (s : List Nat) : transferHolds s s
TensorGuard.CrossDomain.nontransfer	lean/TensorGuard	dt/CrossDomainnontransfer_lean78	sat_iff_dev_eq (dpre dpost : Dev) : nonTransferHolds dpre dpost <-> dpre = dpost
TensorGuard.CrossDomain.nontransfer	lean/TensorGuard	dt/CrossDomainnontransfer_lean82	unsat_iff_dev_ne (dpre dpost : Dev) : not nonTransferHolds dpre dpost <-> dpre != dpost
TensorGuard.CrossDomain.nontransfer	lean/TensorGuard	dt/CrossDomainnontransfer_lean88	shape_free (dpre dpost : Dev) (h : dpre = dpost) : forall _pre _post : List Nat, nonTransferHolds dpre dpost
TensorGuard.CrossDomain.nontransfer	lean/TensorGuard	dt/CrossDomainnontransfer_lean93	no_false_positive (d : Dev) : nonTransferHolds d d
TensorGuard.CrossDomain.branch_selected/TensorGuard	dt/CrossDomainbranch_selected_lean105	(isTransfer : Bool) (pre post : List Nat) (dpre dpost : Dev) : (if isTransfer then transferHolds pre post else nonTransferHolds dpre dpost) = (if isTransfer then pre = post else dpre = dpost)	

TensorGuard.DevDtype.devBug_no_false/TensorGuard	devBug66	positive (a b : Dev) (h : a = Dev.unknown $\vee$ b = Dev.unknown) : devBug a b = false
TensorGuard.DevDtype.devBug_refutation/TensorGuard	devBug73	refutation_sound (a b : Dev) (h : devBug a b = true) : forall ca cb, Dev.gamma a ca -> Dev.gamma b cb -> ca != cb
TensorGuard.DevDtype.moveTo_unknown/TensorGuard	moveToFan99	unknown_inherits (d : Dev) : moveTo Dev.unknown d = d
TensorGuard.DevDtype.pinMemory_preserves/TensorGuard	pinMemory10	preserves (d : Dev) : pinMemory d = d
TensorGuard.DevDtype.cuda_then_clean/TensorGuard	cuda_then104	cpu_roundtrip (d : Dev) : moveTo Dev.cpu (moveTo Dev.cuda0 d) = Dev.cpu
TensorGuard.DevDtype.pin_then_clean/TensorGuard	pin_then109	cpu_safe : devBug (pinMemory Dev.cpu) Dev.cpu = false
TensorGuard.DevDtype.dtMatmulBug_no_false/TensorGuard	dtMatmulBug16	no_false_positive (a b : Dt) (h : a = Dt.unknown $\vee$ b = Dt.unknown) : dtMatmulBug a b = false
TensorGuard.DevDtype.dtMatmulBug_refutation/TensorGuard	dtMatmulBug65	refutation_sound (a b : Dt) (h : dtMatmulBug a b = true) : forall ca cb, Dt.gamma a ca -> Dt.gamma b cb -> ca != cb
TensorGuard.DevDtype.dtFloatParamBug/TensorGuard	dtFloatParamBug80	no_false_positive (input : Dt) (h : input = Dt.unknown) : dtFloatParamBug input = false
TensorGuard.DevDtype.dtFloatParamBug_float_safe/TensorGuard	dtFloatParamBug85	float_safe (input : Dt) (h : input.isFloat = true) : dtFloatParamBug input = false
TensorGuard.DevDtype.dtFloatParamBug_refutation/TensorGuard	dtFloatParamBug94	refutation_sound (input : Dt) (h : dtFloatParamBug input = true) : forall c, Dt.gamma input c -> c.isFloat = false $\wedge$ c != Dt.unknown
TensorGuard.DevDtype.dtElementwiseBug/TensorGuard	dtElementwiseBug206	never_bug (a b : Dt) : dtElementwiseBug a b = false
TensorGuard.DevDtype.dtPromote_clean/TensorGuard	dtPromote208	comm (a b : Dt) : dtPromote a b = dtPromote b a
TensorGuard.DevDtype.dtPromote_idem/TensorGuard	dtPromote211	idem (a : Dt) : dtPromote a a = a
TensorGuard.DevDtype.dtPromote_assoc/TensorGuard	dtPromote219	assoc (a b c : Dt) : dtPromote (dtPromote a b) c = dtPromote a (dtPromote b c)
TensorGuard.DevDtype.dtPromote_unknown/TensorGuard	dtPromote226	unknown_absorbs_left (a : Dt) : dtPromote Dt.unknown a = Dt.unknown
TensorGuard.DevDtype.dtPromote_unknown/TensorGuard	dtPromote230	unknown_absorbs_right (a : Dt) : dtPromote a Dt.unknown = Dt.unknown
TensorGuard.DevDtype.phaseBug_clean/TensorGuard	phaseBug258	unt_gt_one (p : Phase) (t : Bool) (n : Nat) (h : n != 1) : phaseBug p t n = false
TensorGuard.DevDtype.phaseBug_eval/TensorGuard	phaseBug265	eval_tracking_safe (n : Nat) : phaseBug Phase.eval true n = false
TensorGuard.DevDtype.phaseBug_refutation/TensorGuard	phaseBug272	refutation_sound (p : Phase) (t : Bool) (n : Nat) (h : phaseBug p t n = true) : usesBatchStats p t = true $\wedge$ n = 1
TensorGuard.DevDtype.detachOut_clean/TensorGuard	detachOut290	no_grad : detachOut = false
TensorGuard.DevDtype.gradBrokenBug_no_false/TensorGuard	gradBrokenBug294	no_false_positive (h : gradBrokenBug false = false) : gradBrokenBug false = false
TensorGuard.DevDtype.gradBrokenBug_false/TensorGuard	gradBrokenBug297	bug_false : gradBrokenBug false = false
TensorGuard.DevDtype.gradBrokenBug_refutation/TensorGuard	gradBrokenBug301	refutation_sound (b : Bool) (h : gradBrokenBug b = true) : b = true

TensorGuard.Distributions.categorical	lean4/TensorGuard/Distributions/categorical	lean414batch_drops_category_dim : categorical? [2, 3, 4] = some { batch
TensorGuard.Distributions.categorical	lean4/TensorGuard/Distributions/categorical	lean148empty_rank_rejected : categorical? [] = none
TensorGuard.Distributions.categorical	lean4/TensorGuard/Distributions/categorical	lean152zero_categories_rejected : categorical? [2, 0] = none
TensorGuard.Distributions.mvn_batch	lean4/TensorGuard/Distributions/mvn_batch	lean156event_output : multivariateNormal? [4, 3, 5] [1, 3, 5, 5] = some { batch
TensorGuard.Distributions.mvn_matrix	lean4/TensorGuard/Distributions/mvn_matrix	lean161square_rejected : multivariateNormal? [3] [3, 4] = none
TensorGuard.Distributions.mvn_event	lean4/TensorGuard/Distributions/mvn_event	lean165mismatch_rejected : multivariateNormal? [3] [4, 4] = none
TensorGuard.Distributions.mvn_batch	lean4/TensorGuard/Distributions/mvn_batch	lean169broadcast_rejected : multivariateNormal? [2, 3] [4, 3, 3] = none
TensorGuard.Distributions.independent	lean4/TensorGuard/Distributions/independent	lean173moves_batch_to_event : independent? { batch
TensorGuard.Distributions.independent	lean4/TensorGuard/Distributions/independent	lean178preserves_when_zero : independent? { batch
TensorGuard.Distributions.independent	lean4/TensorGuard/Distributions/independent	lean183too_many_rejected : independent? { batch
TensorGuard.Distributions.normal_logProb	lean4/TensorGuard/Distributions/normal_logProb	lean188logProb_broadcasts_value : logProb? { batch
TensorGuard.Distributions.categorical_logProb	lean4/TensorGuard/Distributions/categorical_logProb	lean203logProb_drops_no_event : logProb? { batch
TensorGuard.Distributions.mvn_logProb	lean4/TensorGuard/Distributions/mvn_logProb	lean208logProb_drops_event_dim : logProb? { batch
TensorGuard.Distributions.logProb_broadcast	lean4/TensorGuard/Distributions/logProb_broadcast	lean231logProb_value_broadcast_rejected : logProb? { batch
TensorGuard.Distributions.reshapeShape	lean4/TensorGuard/Distributions/reshapeShape	lean327reshape_output : reshapeShape? [4, 2, 3] [2, 3] [6] = some [4, 6]
TensorGuard.Distributions.reshapeShape	lean4/TensorGuard/Distributions/reshapeShape	lean331wrong_suffix_rejected : reshapeShape? [4, 2, 3] [3, 3] [9] = none
TensorGuard.Distributions.reshapeShape	lean4/TensorGuard/Distributions/reshapeShape	lean335numel_mismatch_rejected : reshapeShape? [4, 2, 3] [2, 3] [5] = none
TensorGuard.Distributions.transformed	lean4/TensorGuard/Distributions/transformed	lean380identity_preserves_shape : transformed? { batch
TensorGuard.Distributions.transformed	lean4/TensorGuard/Distributions/transformed	lean384reshape_event_shape : transformed? { batch
TensorGuard.Distributions.transformed	lean4/TensorGuard/Distributions/transformed	lean389reshape_reinterprets_batch : transformed? { batch
TensorGuard.Distributions.transformed	lean4/TensorGuard/Distributions/transformed	lean394composed_reshape_identity : transformed? { batch
TensorGuard.Distributions.transformed	lean4/TensorGuard/Distributions/transformed	lean399wrong_domain_rejected : transformed? { batch
TensorGuard.DtypePromoteChain.dtypes	lean4/TensorGuard/DtypePromoteChain.dtypes	lean51dtLe a a = true
TensorGuard.DtypePromoteChain.dtypes	lean4/TensorGuard/DtypePromoteChain.dtypes	lean52Dt (hab : dtLe a b = true) (hbc : dtLe b c = true) : dtLe a c = true
TensorGuard.DtypePromoteChain.dtypes	lean4/TensorGuard/DtypePromoteChain.dtypes	lean54Dt (hba : dtLe b a = true) : a = b
TensorGuard.DtypePromoteChain.light	lean4/TensorGuard/DtypePromoteChain.light	lean58b : Dt) : dtLe a (dtPromote a b) = true
TensorGuard.DtypePromoteChain.light	lean4/TensorGuard/DtypePromoteChain.light	lean61a b : Dt) : dtLe b (dtPromote a b) = true

TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean66	TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean66	$\text{promote} (\text{dtPromote acc } x) y = \text{dtPromote} (\text{dtPromote acc } y) x$
TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean71	TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean71	$\text{Dt.unknown } x = \text{Dt.unknown}$
TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean79	TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean79	$\text{promoteRun acc } (xs ++ ys) = \text{promoteRun} (\text{promoteRun acc } xs) ys$
TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean86	TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean86	$\text{dtLe acc } (\text{promoteRun acc } xs) = \text{true}$
TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean97	TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean97	$\text{dtLe } x (\text{promoteRun acc } xs) = \text{true}$
TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean113	TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean113	$\text{promoteRun acc } (x :: y :: \text{rest}) = \text{promoteRun acc } (y :: x :: \text{rest})$
TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean121	TensorGuard.DtypePromoteChain.promoteTensorGuard.DtypePromoteChain.lean121	$\text{promoteRun Dt.unknown } xs = \text{Dt.unknown}$
TensorGuard.Einops.prod_append	lean/TensorGuard/Einops.lean35	$\text{prod } (xs ++ ys) = \text{prod } xs * \text{prod } ys$
TensorGuard.Einops.decompValid_iff	lean/TensorGuard/Einops.lean54	$\text{decompValid axis knownProduct} = \text{true} \leftrightarrow (0 < \text{knownProduct} \wedge \text{knownProduct} ? \text{axis})$
TensorGuard.Einops.decompValid_imp_dvd	lean/TensorGuard/Einops.lean60	$(h : \text{decompValid axis knownProduct} = \text{true}) : \text{knownProduct} ? \text{axis}$
TensorGuard.Einops.nondivisible_decomposition_flagged	lean/TensorGuard/Einops.lean66	$(\text{axis knownProduct} : \text{Nat}) (h : \text{not knownProduct} ? \text{axis}) : \text{decompValid axis knownProduct} = \text{false}$
TensorGuard.Einops.inferSubaxis_spec	lean/TensorGuard/Einops.lean58	$(_hpos : 0 < \text{knownProduct}) (\text{hdvd} : \text{knownProduct} ? \text{axis}) : \text{knownProduct} * \text{inferSubaxis axis knownProduct} = \text{axis}$
TensorGuard.Einops.decomposedGroup_product	lean/TensorGuard/Einops.lean81	$(h : \text{decompValid axis knownProduct} = \text{true}) : \text{prod} (\text{decomposedGroup axis knownProduct}) = \text{axis}$
TensorGuard.Einops.inferSubaxis_position	lean/TensorGuard/Einops.lean89	$(\text{axis knownProduct} : \text{Nat}) : (\text{decomposedGroup axis knownProduct}).\text{getLast?} = \text{some} (\text{inferSubaxis axis knownProduct})$
TensorGuard.Einops.axisBijection_iff_counts	lean/TensorGuard/Einops.lean112	$\text{axisBijection lhs rhs} \leftrightarrow \text{forall } a, \text{countAxis } a \text{ lhs} = \text{countAxis } a \text{ rhs}$
TensorGuard.Einops.axisBijection_refl	lean/TensorGuard/Einops.lean116	$\text{axisBijection } xs \text{ } xs$
TensorGuard.Einops.axisBijection_sym	lean/TensorGuard/Einops.lean120	$\{\text{lhs rhs} : \text{List Nat}\} (h : \text{axisBijection lhs rhs}) : \text{axisBijection rhs lhs}$
TensorGuard.Einops.axisBijection_trans	lean/TensorGuard/Einops.lean125	$\{\text{xs ys zs} : \text{List Nat}\} (\text{hxy} : \text{axisBijection } xs \text{ } ys) (\text{hyz} : \text{axisBijection } ys \text{ } zs) : \text{axisBijection } xs \text{ } zs$
TensorGuard.Einops.adjacent_swap	lean/TensorGuard/Einops.lean132	$\text{adjacent_swap_axis_bijection } (a \text{ } b : \text{Nat}) (\text{rest} : \text{List Nat}) : \text{axisBijection } (a :: b :: \text{rest}) (b :: a :: \text{rest})$
TensorGuard.Einops.dropped_axis_not_bijection	lean/TensorGuard/Einops.lean138	$\{a \text{ } b : \text{Nat}\} (h : a \neq b) : \text{not axisBijection } [a, b] [a]$

TensorGuard.Einops.added_axis_not_bijection	lean/TensorGuard/Extensible.lean:145	axis_not_bijection {a b : Nat} (h : a != b) : not axisBijection [a] [a, b]
TensorGuard.Einops.duplicated_axis_not_bijection	lean/TensorGuard/Extensible.lean:152	axisBijection (a : Nat) : not axisBijection [a] [a, a]
TensorGuard.EmbeddingRule.prod_append	lean/TensorGuard/EmbeddingRule.lean:45	(xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.EmbeddingRule.embeddingRule	lean/TensorGuard/EmbeddingRule.lean:62	(input : List Nat) (embDim : Nat) : (embShape input embDim).length = input.length + 1
TensorGuard.EmbeddingRule.embeddingRule	lean/TensorGuard/EmbeddingRule.lean:68	(input : List Nat) (embDim : Nat) : (embShape input embDim).drop input.length = [embDim]
TensorGuard.EmbeddingRule.embeddingRule	lean/TensorGuard/EmbeddingRule.lean:73	(input : List Nat) (embDim : Nat) : (embShape input embDim).take input.length = input
TensorGuard.EmbeddingRule.embeddingRule	lean/TensorGuard/EmbeddingRule.lean:79	(input : List Nat) (embDim : Nat) : prod (embShape input embDim) = prod input * embDim
TensorGuard.EmbeddingRule.idxValid	lean/TensorGuard/EmbeddingRule.lean:93	(idx num : Nat) : idxValid idx num = true <-> idx < num
TensorGuard.EmbeddingRule.allValid	lean/TensorGuard/EmbeddingRule.lean:100	(idxs : List Nat) (num : Nat) : allValid idxs num = true <-> forall i in idxs, i < num
TensorGuard.EmbeddingRule.outOfRange	lean/TensorGuard/EmbeddingRule.lean:109	(idx num : Nat) (h : num <= idx) : idxValid idx num = false
TensorGuard.EmbeddingRule.allValid	lean/TensorGuard/EmbeddingRule.lean:115	(idxs : List Nat) (num num' : Nat) (hle : num <= num') (h : allValid idxs num = true) : allValid idxs num' = true
TensorGuard.matmul2_sound	lean/TensorGuard/Extensible.lean:135	sound (m k1 k2 n : Nat) (s' : Shape) (h : matmul2 (.cons m (.cons k1 .nil)) (.cons k2 (.cons n .nil))) = some s' : k1 = k2 /\ s' = .cons m (.cons n .nil)
TensorGuard.bmm_sound	lean/TensorGuard/Extensible.lean:51	sound (b1 b2 m k1 k2 n : Nat) (s' : Shape) (h : bmm (.cons b1 (.cons m (.cons k1 .nil))) (.cons b2 (.cons k2 (.cons n .nil)))) = some s' : b1 = b2 /\ k1 = k2 /\ s' = .cons b1 (.cons m (.cons n .nil))
TensorGuard.transpose2_involutive	lean/TensorGuard/Extensible.lean:672	involutive (m n : Nat) : (transpose2 (.cons m (.cons n .nil))).bind transpose2 = some (.cons m (.cons n .nil))
TensorGuard.permList_compose_inrange	lean/TensorGuard/Extensible.lean:1104	compose_inrange (p q : List Nat) (dims : List Nat) (_ hp : Inrange p dims) (hpq : Inrange q (permList p dims)) : permList q (permList p dims) = permList (permCompose q p) dims
TensorGuard.conv1dOut_monotone	lean/TensorGuard/Extensible.lean:1050	conv1d_monotone (h? h? pad dilation k stride o? o? : Nat) (hle : h? <= h?) (h1 : conv1dOut h? pad dilation k stride = some o?) (h2 : conv1dOut h? pad dilation k stride = some o?) : o? <= o?
TensorGuard.relu_shape_preserving	lean/TensorGuard/Extensible.lean:183	relu_shape_preserving (s : Shape) : relu s = s
TensorGuard.bcastDim_comm	lean/TensorGuard/Extensible.lean:205	bcastDim a b = bcastDim b a
TensorGuard.bcast_comm	lean/TensorGuard/Extensible.lean:220	bcastComm : forall s? s? : Shape, bcast s? s? = bcast s? s? .nil, .nil => rfl .nil, .cons _ _ => rfl .cons _ _, .nil => rfl .cons a r?, .cons b r? => by unfold bcast rw [bcastDim_comm a b, bcast_comm r? r?] theorem bcast_rank1_sound (a b d : Nat) (h : bcast (.cons a .nil) (.cons b .nil) = some (.cons d .nil)) : a = b /\ a = 1 /\ b = 1

TensorGuard.bcast_rank1_sound	lean/TensorGuard/Extensible.lean:231	Extensible.lean:231 (a b d : Nat) (h : bcast (.cons a .nil) (.cons b .nil) = some (.cons d .nil)) : a = b $\vee$ a = 1 $\vee$ b = 1
TensorGuard.Flatten.prod_append	lean/TensorGuard/Flatten.lean:49	Flatten.lean:49 append (xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.Flatten.prod_flatten	lean/TensorGuard/Flatten.lean:61	Flatten.lean:61 flatten (pre span suf : List Nat) : prod (flattenShape pre span suf) = prod (pre ++ span ++ suf)
TensorGuard.Flatten.length_flatten	lean/TensorGuard/Flatten.lean:74	Flatten.lean:74 flatten (pre span suf : List Nat) : (flattenShape pre span suf).length = pre.length + 1 + suf.length
TensorGuard.Flatten.flatten_full	lean/TensorGuard/Flatten.lean:86	Flatten.lean:86 full (span : List Nat) : flattenShape [] span [] = [prod span]
TensorGuard.Flatten.flatten_singleton	lean/TensorGuard/Flatten.lean:92	Flatten.lean:92 singleton (pre : List Nat) (d : Nat) (suf : List Nat) : flattenShape pre [d] suf = pre ++ (d :: suf)
TensorGuard.Flatten.flatten_dim_value	lean/TensorGuard/Flatten.lean:98	Flatten.lean:98 dim_value (pre span suf : List Nat) : (flattenShape pre span suf).drop pre.length = prod span :: suf
TensorGuard.FragmentU1.sound_safeSound	lean/TensorGuard/FragmentU1.lean:75	FragmentU1.lean:75 sound (outOfFragment coreFoundBug realBug : Bool) (h : coreSoundOnFragment outOfFragment coreFoundBug realBug) : safeSound (decide Mode.sound outOfFragment coreFoundBug) realBug
TensorGuard.FragmentU1.balanced_unsound	lean/TensorGuard/FragmentU1.lean:90	FragmentU1.lean:90 balanced : exists outOfFragment coreFoundBug realBug, coreSoundOnFragment outOfFragment coreFoundBug realBug $\wedge$ decide Mode.balanced outOfFragment coreFoundBug = Verdict.safe $\wedge$ realBug = true
TensorGuard.FragmentU1.heuristic_unsound	lean/TensorGuard/FragmentU1.lean:100	FragmentU1.lean:100 heuristic : exists outOfFragment coreFoundBug realBug, coreSoundOnFragment outOfFragment coreFoundBug realBug $\wedge$ decide Mode.heuristic outOfFragment coreFoundBug = Verdict.safe $\wedge$ realBug = true
TensorGuard.FragmentU1.modes_agree	lean/TensorGuard/FragmentU1.lean:111	FragmentU1.lean:111 agree (mode mode' : Mode) (coreFoundBug : Bool) : decide mode false coreFoundBug = decide mode' false coreFoundBug
TensorGuard.FragmentU1.modes_diff	lean/TensorGuard/FragmentU1.lean:117	FragmentU1.lean:117 diff (outOfFragment coreFoundBug : Bool) : (decide Mode.sound outOfFragment coreFoundBug != decide Mode.balanced outOfFragment coreFoundBug) <-> (outOfFragment = true $\wedge$ coreFoundBug = false)
TensorGuard.GradFlow.keep_id	lean/TensorGuard/GradFlow.lean:61	GradFlow.lean:61 keep (b : Bool) : gradStep b GradOp.keep = b
TensorGuard.GradFlow.detach_false	lean/TensorGuard/GradFlow.lean:62	GradFlow.lean:62 detach (b : Bool) : gradStep b GradOp.detach = false
TensorGuard.GradFlow.noGrad_false	lean/TensorGuard/GradFlow.lean:63	GradFlow.lean:63 noGrad (b : Bool) : gradStep b GradOp.noGrad = false
TensorGuard.GradFlow.reattach_true	lean/TensorGuard/GradFlow.lean:64	GradFlow.lean:64 reattach (b : Bool) : gradStep b GradOp.reattach = true
TensorGuard.GradFlow.reset_absorb	lean/TensorGuard/GradFlow.lean:68	GradFlow.lean:68 absorb (op : GradOp) (h : isReset op = true) (b b' : Bool) : gradStep b op = gradStep b' op
TensorGuard.GradFlow.gradRun_append	lean/TensorGuard/GradFlow.lean:77	GradFlow.lean:77 append (b0 : Bool) (xs ys : List GradOp) : gradRun b0 (xs ++ ys) = gradRun (gradRun b0 xs) ys

TensorGuard.GradFlow.run_after_reset	lean/TensorGuard/GradFlow.lean:35	GradFlow.reset (b0 b0' : Bool) (xs : List GradOp) (op : GradOp) (h : isReset op = true) : gradRun b0 (xs ++ [op]) = gradRun b0' (xs ++ [op])
TensorGuard.GradFlow.run_noReattach	lean/TensorGuard/GradFlow.lean:46	GradFlow.noReattach_true_iff (b0 : Bool) (xs : List GradOp) (h : forall op in xs, op != GradOp.reattach) : gradRun b0 xs = (b0 && xs.all (fun op => !isReset op))
TensorGuard.GridSample.gridSample2DValid_iff	lean/TensorGuard/GridSample.lean:52	GridSample.gridSample2DValid_iff (n h w gn coord : Nat) : gridSample2DValid n h w gn coord = true <-> (((0 < h /\ 0 < w) /\ gn = n) /\ coord = 2)
TensorGuard.GridSample.gridSample3DValid_iff	lean/TensorGuard/GridSample.lean:53	GridSample.gridSample3DValid_iff (n d h w gn coord : Nat) : gridSample3DValid n d h w gn coord = true <-> (((0 < d /\ 0 < h) /\ 0 < w) /\ gn = n) /\ coord = 3)
TensorGuard.GridSample.gridSample2D_valid_link	lean/TensorGuard/GridSample.lean:62	GridSample.gridSample2D_valid_link (n c h w gn oh ow coord : Nat) (hvalid : gridSample2DValid n h w gn coord = true) : gridSample? [n, c, h, w] [gn, oh, ow, coord] = some [n, c, oh, ow]
TensorGuard.GridSample.gridSample3D_valid_link	lean/TensorGuard/GridSample.lean:63	GridSample.gridSample3D_valid_link (n c d h w gn od oh ow coord : Nat) (hvalid : gridSample3DValid n d h w gn coord = true) : gridSample? [n, c, d, h, w] [gn, od, oh, ow, coord] = some [n, c, od, oh, ow]
TensorGuard.GridSample.gridSample2D_invalid_link	lean/TensorGuard/GridSample.lean:72	GridSample.gridSample2D_invalid_link (n c h w gn oh ow coord : Nat) (hvalid : gridSample2DValid n h w gn coord = false) : gridSample? [n, c, h, w] [gn, oh, ow, coord] = none
TensorGuard.GridSample.gridSample3D_invalid_link	lean/TensorGuard/GridSample.lean:82	GridSample.gridSample3D_invalid_link (n c d h w gn od oh ow coord : Nat) (hvalid : gridSample3DValid n d h w gn coord = false) : gridSample? [n, c, d, h, w] [gn, od, oh, ow, coord] = none
TensorGuard.GridSample.gridSample2D_output_shape	lean/TensorGuard/GridSample.lean:82	GridSample.gridSample2D_output_shape (n c h w oh ow : Nat) (hh : 0 < h) (hw : 0 < w) : gridSample? [n, c, h, w] [n, oh, ow, 2] = some [n, c, oh, ow]
TensorGuard.GridSample.gridSample3D_output_shape	lean/TensorGuard/GridSample.lean:93	GridSample.gridSample3D_output_shape (n c d h w od oh ow : Nat) (hd : 0 < d) (hh : 0 < h) (hw : 0 < w) : gridSample? [n, c, d, h, w] [n, od, oh, ow, 3] = some [n, c, od, oh, ow]
TensorGuard.GridSample.gridSample2D_output_rank	lean/TensorGuard/GridSample.lean:92	GridSample.gridSample2D_output_rank (n c oh ow : Nat) : [n, c, oh, ow].length = 4
TensorGuard.GridSample.gridSample3D_output_rank	lean/TensorGuard/GridSample.lean:103	GridSample.gridSample3D_output_rank (n c od oh ow : Nat) : [n, c, od, oh, ow].length = 5
TensorGuard.GridSample.gridSample_wrong_input_rank_rejected	lean/TensorGuard/GridSample.lean:107	GridSample.gridSample_wrong_input_rank_rejected (n c h gn gh coord : Nat) : gridSample? [n, c, h] [gn, gh, coord] = none
TensorGuard.GridSample.gridSample_grid_rank_mismatch_rejected	lean/TensorGuard/GridSample.lean:111	GridSample.gridSample_grid_rank_mismatch_rejected (n c h w gn oh ow coord extra : Nat) : gridSample? [n, c, h, w] [gn, oh, ow, coord, extra] = none
TensorGuard.GridSample.gridSample2D_coord_dim_flagged	lean/TensorGuard/GridSample.lean:121	GridSample.gridSample2D_coord_dim_flagged (n c h w oh ow : Nat) : gridSample? [n, c, h, w] [n, oh, ow, 3] = none
TensorGuard.GridSample.gridSample3D_coord_dim_flagged	lean/TensorGuard/GridSample.lean:120	GridSample.gridSample3D_coord_dim_flagged (n c d h w od oh ow : Nat) : gridSample? [n, c, d, h, w] [n, od, oh, ow, 2] = none
TensorGuard.GridSample.gridSample2D_zero_height_flagged	lean/TensorGuard/GridSample.lean:124	GridSample.gridSample2D_zero_height_flagged (n c w oh ow : Nat) : gridSample? [n, c, 0, w] [n, oh, ow, 2] = none

TensorGuard.GridSample.gridSample	2D/any/TensorGuard	44GridSampleSample28_zero_width_flagged (n c h oh ow : Nat) : gridSample? [n, c, h, 0] [n, oh, ow, 2] = none
TensorGuard.GridSample.gridSample	3D/any/TensorGuard	44GridSampleSample30D_zero_depth_flagged (n c h w od oh ow : Nat) : gridSample? [n, c, 0, h, w] [n, od, oh, ow, 3] = none
TensorGuard.GridSample.gridSample	3D/any/TensorGuard	44GridSampleSample30D_zero_height_flagged (n c d w od oh ow : Nat) : gridSample? [n, c, d, 0, w] [n, od, oh, ow, 3] = none
TensorGuard.GridSample.gridSample	3D/any/TensorGuard	44GridSampleSample40D_zero_width_flagged (n c d h od oh ow : Nat) : gridSample? [n, c, d, h, 0] [n, od, oh, ow, 3] = none
TensorGuard.GridSample.gridSample	2D/any/TensorGuard	44GridSampleSample42D_batch_mismatch_flagged (n gn c h w oh ow : Nat) (hne : gn != n) : gridSample? [n, c, h, w] [gn, oh, ow, 2] = none
TensorGuard.GridSample.gridSample	4D/any/TensorGuard	44GridSampleSample49accepts_empty_output_grid (n c h w ow : Nat) (hh : 0 < h) (hw : 0 < w) : gridSample? [n, c, h, w] [n, 0, ow, 2] = some [n, c, 0, ow]
TensorGuard.GridSample.affineGrid	2D/Valid/TensorGuard	44GridSampleSample20Valid_iff (tn rows cols n c h w : Nat) : affineGrid2DValid tn rows cols n c h w = true <-> ((((((0 < n /\ 0 < c) /\ 0 < h) /\ 0 < w) /\ 0 < tn) /\ tn = n) /\ rows = 2) /\ cols = 3)
TensorGuard.GridSample.affineGrid	3D/Valid/TensorGuard	44GridSampleSample30Valid_iff (tn rows cols n c d h w : Nat) : affineGrid3DValid tn rows cols n c d h w = true <-> (((((((0 < n /\ 0 < c) /\ 0 < d) /\ 0 < h) /\ 0 < w) /\ 0 < tn) /\ tn = n) /\ rows = 3) /\ cols = 4)
TensorGuard.GridSample.affineGrid	2D/any/Invalid/TensorGuard	44GridSampleSample205_invalid_link (tn rows cols n c h w : Nat) (hvalid : affineGrid2DValid tn rows cols n c h w = true) : affineGrid? [tn, rows, cols] [n, c, h, w] = some [n, h, w, 2]
TensorGuard.GridSample.affineGrid	3D/any/Invalid/TensorGuard	44GridSampleSample301_invalid_link (tn rows cols n c d h w : Nat) (hvalid : affineGrid3DValid tn rows cols n c d h w = true) : affineGrid? [tn, rows, cols] [n, c, d, h, w] = some [n, d, h, w, 3]
TensorGuard.GridSample.affineGrid	2D/any/Invalid/TensorGuard	44GridSampleSample207_invalid_link (tn rows cols n c h w : Nat) (hvalid : affineGrid2DValid tn rows cols n c h w = false) : affineGrid? [tn, rows, cols] [n, c, h, w] = none
TensorGuard.GridSample.affineGrid	3D/any/Invalid/TensorGuard	44GridSampleSample303_invalid_link (tn rows cols n c d h w : Nat) (hvalid : affineGrid3DValid tn rows cols n c d h w = false) : affineGrid? [tn, rows, cols] [n, c, d, h, w] = none
TensorGuard.GridSample.affineGrid	2D/any/Invalid/TensorGuard	44GridSampleSample209_output_shape (n c h w : Nat) (hn : 0 < n) (hc : 0 < c) (hh : 0 < h) (hw : 0 < w) : affineGrid? [n, 2, 3] [n, c, h, w] = some [n, h, w, 2]
TensorGuard.GridSample.affineGrid	3D/any/Invalid/TensorGuard	44GridSampleSample304_output_shape (n c d h w : Nat) (hn : 0 < n) (hc : 0 < c) (hd : 0 < d) (hh : 0 < h) (hw : 0 < w) : affineGrid? [n, 3, 4] [n, c, d, h, w] = some [n, d, h, w, 3]
TensorGuard.GridSample.affineGrid	2D/any/Invalid/TensorGuard	44GridSampleSample230_output_rank (n h w : Nat) : [n, h, w, 2].length = 4
TensorGuard.GridSample.affineGrid	3D/any/Invalid/TensorGuard	44GridSampleSample304_output_rank (n d h w : Nat) : [n, d, h, w, 3].length = 5
TensorGuard.GridSample.affineGrid	3D/any/Invalid/TensorGuard	44GridSampleSample238_size_rank_rejected (tn rows cols n c h : Nat) : affineGrid? [tn, rows, cols] [n, c, h] = none

TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	242	theta_rank_rejected (n c h w : Nat) : affineGrid? [n, 2] [n, c, h, w] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	246	theta_rows_flagged (n c h w : Nat) : affineGrid? [n, 3, 3] [n, c, h, w] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	250	theta_cols_flagged (n c h w : Nat) : affineGrid? [n, 2, 4] [n, c, h, w] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	254	theta_rows_flagged (n c d h w : Nat) : affineGrid? [n, 2, 4] [n, c, d, h, w] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	258	theta_cols_flagged (n c d h w : Nat) : affineGrid? [n, 3, 3] [n, c, d, h, w] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	262	size_batch_positive_required (c h w : Nat) : affineGrid? [0, 2, 3] [0, c, h, w] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	266	size_channel_positive_required (n h w : Nat) : affineGrid? [n, 2, 3] [n, 0, h, w] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	270	size_height_positive_required (n c w : Nat) : affineGrid? [n, 2, 3] [n, c, 0, w] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	274	size_width_positive_required (n c h : Nat) : affineGrid? [n, 2, 3] [n, c, h, 0] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	278	size_depth_positive_required (n c h w : Nat) : affineGrid? [n, 3, 4] [n, c, 0, h, w] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	282	theta_batch_positive_required (c h w : Nat) : affineGrid? [0, 2, 3] [1, c, h, w] = none
TensorGuard.GridSample.affineGrid	lean4/TensorGuard/KernelSampleGrid	286	batch_mismatch_flagged (tn n c h w : Nat) (hne : tn != n) : affineGrid? [tn, 2, 3] [n, c, h, w] = none
TensorGuard.LayerNormRule.prod_append	lean4/TensorGuard/LayerNormRule	34	(xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.LayerNormRule.ln_pre_normShape	lean4/TensorGuard/LayerNormRule	40	(pre normShape : List Nat) : lnShape pre normShape = pre ++ normShape
TensorGuard.LayerNormRule.ln_normShape	lean4/TensorGuard/LayerNormRule	45	(pre normShape : List Nat) : prod (lnShape pre normShape) = prod pre * prod normShape
TensorGuard.LayerNormRule.ln_length	lean4/TensorGuard/LayerNormRule	50	(pre normShape : List Nat) : (lnShape pre normShape).length - normShape.length = pre.length
TensorGuard.LayerNormRule.ln_suffix_drop	lean4/TensorGuard/LayerNormRule	55	(pre normShape : List Nat) : (lnShape pre normShape).drop pre.length = normShape
TensorGuard.LayerNormRule.ln_mismatch_flagged	lean4/TensorGuard/LayerNormRule	61	(pre : List Nat) (a b : Nat) (h : a != b) : (lnShape pre [a]).drop pre.length != [b]
TensorGuard.LinearRule.prod_append	lean4/TensorGuard/LinearRule	34	(xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.LinearRule.lin_rank	lean4/TensorGuard/LinearRule	45	(pre : List Nat) (inF outF : Nat) : (linShape pre outF).length = (pre ++ [inF]).length
TensorGuard.LinearRule.lin_last	lean4/TensorGuard/LinearRule	49	(pre : List Nat) (outF : Nat) : (linShape pre outF).drop pre.length = [outF]
TensorGuard.LinearRule.lin_prefix	lean4/TensorGuard/LinearRule	53	(pre : List Nat) (outF : Nat) : (linShape pre outF).take pre.length = pre
TensorGuard.LinearRule.lin_numel	lean4/TensorGuard/LinearRule	57	(pre : List Nat) (outF : Nat) : prod (linShape pre outF) = prod pre * outF

TensorGuard.LinearRule.linValid_iff	lean/TensorGuard/LinearRuleValid.lean:66	linValid (lastDim inF : Nat) : linValid lastDim inF = true <-> lastDim = inF
TensorGuard.LinearRule.mismatch	flagged/TensorGuard/LinearRuleMismatch.lean:67	flagged (lastDim inF : Nat) (h : lastDim != inF) : linValid lastDim inF = false
TensorGuard.MatmulSound.applyOpExt	lean/TensorGuard/MatmulSoundApplyOpExt.lean:30	sound_matmul (s s' : Shape) (h : applyOpExt .matmul s = some s') : s.length >= 3 /\ s' = s
TensorGuard.MatmulSound.matmul	lean/TensorGuard/MatmulSoundMatmul.lean:44	fraction_sound (rest : V5.Sh) (m k n : Nat) : V5.matmul (rest ++ [m, k]) (rest ++ [k, n]) = some (rest ++ [m, n])
TensorGuard.MatmulSound.applyOpExt	lean/TensorGuard/MatmulSoundApplyOpExt.lean:52	matmul_rank_lb (s s' : Shape) (h : applyOpExt .matmul s = some s') : 3 <= s.length
TensorGuard.MatmulSound.applyOpExt	lean/TensorGuard/MatmulSoundApplyOpExt.lean:62	matmul_verdict_eq (s s' : Shape) (h : applyOpExt .matmul s = some s') : s' = s
TensorGuard.NamedTensor.containsNamed	lean/TensorGuard/NamedTensorContainsNamed.lean:59	containsNamed_head (n : Nat) (rest : List AxisName) : containsNamed n (named n :: rest) = true
TensorGuard.NamedTensor.uniqueNamed	lean/TensorGuard/NamedTensorUniqueNamed.lean:64	uniqueNamed_head (n : Nat) (rest : List AxisName) : uniqueNamed (named n :: named n :: rest) = false
TensorGuard.NamedTensor.uniqueNamed	lean/TensorGuard/NamedTensorUniqueNamed.lean:69	allows_repeated_anon (rest : List AxisName) : uniqueNamed (anon :: anon :: rest) = uniqueNamed rest
TensorGuard.NamedTensor.refine_existing	lean/TensorGuard/NamedTensorRefineExisting.lean:116	existing_name_preserved (n : Nat) : refineAxis? (named n) (named n) = some (named n)
TensorGuard.NamedTensor.refine_rejected	lean/TensorGuard/NamedTensorRefineRejected.lean:120	rejected (old new : Nat) (h : old != new) : refineAxis? (named old) (named new) = none
TensorGuard.NamedTensor.refine_deleted	lean/TensorGuard/NamedTensorRefineDeleted.lean:125	deleted_rejected (n : Nat) : refineAxis? (named n) anon = none
TensorGuard.NamedTensor.refine_duplicate_requested	lean/TensorGuard/NamedTensorRefineDuplicateRequested.lean:129	duplicate_requested_rejected (shape : List Nat) (current requested : List AxisName) (hdup : uniqueNamed requested = false) : refineNames? shape current requested = none
TensorGuard.NamedTensor.refine_duplicate_current	lean/TensorGuard/NamedTensorRefineDuplicateCurrent.lean:138	duplicate_current_rejected (shape : List Nat) (current requested : List AxisName) (hdup : uniqueNamed current = false) : refineNames? shape current requested = none
TensorGuard.NamedTensor.refine_shape	lean/TensorGuard/NamedTensorRefineShape.lean:146	shape_preserved (shape : List Nat) (current requested : List AxisName) (out : NamedShape) (h : refineNames? shape current requested = some out) : out.shape = shape
TensorGuard.NamedTensor.refine_fill	lean/TensorGuard/NamedTensorRefineFill.lean:164	fill_example : refineNames? [2, 3] [anon, anon] [named 0, named 1] = some { shape
TensorGuard.NamedTensor.refine_preserve_existing	lean/TensorGuard/NamedTensorRefinePreserveExisting.lean:169	existing_example : refineNames? [2, 3] [named 0, named 1] [named 0, named 1] = some { shape
TensorGuard.NamedTensor.refine_duplicate_names	lean/TensorGuard/NamedTensorRefineDuplicateNames.lean:174	duplicate_names_rejected : refineNames? [2, 3] [anon, anon] [named 0, named 0] = none
TensorGuard.NamedTensor.existing	lean/TensorGuard/NamedTensorExisting.lean:234	dim_preserved (shape : List Nat) (current : List AxisName) (n d : Nat) (h : lookupDimByName n shape current = some d) : targetDim shape current (named n) = d

TensorGuard.NamedTensor.fresh_name	lean/NamedTensorGuard/NamedTensor.lean:237	inserts_singleton (shape : List Nat) (current : List AxisName) (n : Nat) (h : lookupDimByName n shape current = none) : targetDim shape current (named n) = 1
TensorGuard.NamedTensor.anon_target	lean/NamedTensorGuard/NamedTensor.lean:243	inserts_singleton (shape : List Nat) (current : List AxisName) : targetDim shape current anon = 1
TensorGuard.NamedTensor.align_name	lean/NamedTensorGuard/NamedTensor.lean:248	preserved (shape : List Nat) (current target : List AxisName) (out : NamedShape) (h : alignTo? shape current target = some out) : out.names = target
TensorGuard.NamedTensor.align_duplicate_target	lean/NamedTensorGuard/NamedTensor.lean:260	target_rejected (shape : List Nat) (current target : List AxisName) (hdup : uniqueNamed target = false) : alignTo? shape current target = none
TensorGuard.NamedTensor.align_duplicate_current	lean/NamedTensorGuard/NamedTensor.lean:268	current_rejected (shape : List Nat) (current target : List AxisName) (hdup : uniqueNamed current = false) : alignTo? shape current target = none
TensorGuard.NamedTensor.align_underinput	lean/NamedTensorGuard/NamedTensor.lean:275	underinput_rejected (d : Nat) (restShape : List Nat) (currentTail target : List AxisName) : alignTo? (d :: restShape) (anon :: currentTail) target = none
TensorGuard.NamedTensor.align_permute	lean/NamedTensorGuard/NamedTensor.lean:283	example : alignTo? [2, 3] [named 0, named 1] [named 1, named 0] = some { shape
TensorGuard.NamedTensor.align_singleton	lean/NamedTensorGuard/NamedTensor.lean:288	example : alignTo? [2, 3] [named 0, named 1] [named 0, named 2, named 1] = some { shape
TensorGuard.NamedTensor.align_andanon	lean/NamedTensorGuard/NamedTensor.lean:293	example : alignTo? [2, 3] [named 0, named 1] [named 0, anon, named 1] = some { shape
TensorGuard.NamedTensor.align_mismatch	lean/NamedTensorGuard/NamedTensor.lean:298	example : alignTo? [2, 3] [named 0, named 1] [named 0] = none
TensorGuard.NamedTensor.align_duplicate_names	lean/NamedTensorGuard/NamedTensor.lean:302	example : alignTo? [2, 3] [named 0, named 1] [named 0, named 0, named 1] = none
TensorGuard.squeeze_removes_ones	lean/NamedTensorGuard/NamedTensor.lean:307	squeeze_removes_ones (shape : List Nat) (d : Nat) (h : d in squeeze shape) : d != 1
TensorGuard.layerNorm_identity	lean/NamedTensorGuard/NamedTensor.lean:311	layerNorm_identity (shape : List Nat) (normalized_dims : Nat) (out : List Nat) (h : layerNormShape shape normalized_dims = some out) : out = shape
TensorGuard.embedding_appends	lean/NamedTensorGuard/NamedTensor.lean:314	embedding_appends (input_shape : List Nat) (embed_dim : Nat) : (embeddingShape input_shape embed_dim).length = input_shape.length + 1
TensorGuard.PhaseFlow.keep_id	lean/NamedTensorGuard/NamedTensor.lean:316	keep_id (b : Bool) : phaseStep b PhaseOp.keep = b
TensorGuard.PhaseFlow.setTrain_true	lean/NamedTensorGuard/NamedTensor.lean:316	setTrain_true (b : Bool) : phaseStep b PhaseOp.setTrain = true
TensorGuard.PhaseFlow.setEval_false	lean/NamedTensorGuard/NamedTensor.lean:316	setEval_false (b : Bool) : phaseStep b PhaseOp.setEval = false
TensorGuard.PhaseFlow.setter_absorb	lean/NamedTensorGuard/NamedTensor.lean:316	absorb (op : PhaseOp) (h : isSetter op = true) (b b' : Bool) : phaseStep b op = phaseStep b' op

TensorGuard.PhaseFlow.setter_hits	lean/TensorGuard/PhaseFlow.lean:75	lean/TensorGuard/PhaseFlow.lean:75_value (op : PhaseOp) (h : isSetter op = true) (b : Bool) : phaseStep b op = setterValue op
TensorGuard.PhaseFlow.phaseRun_append	lean/TensorGuard/PhaseFlow.lean:84	lean/TensorGuard/PhaseFlow.lean:84_append (b0 : Bool) (xs ys : List PhaseOp) : phaseRun b0 (xs ++ ys) = phaseRun (phaseRun b0 xs) ys
TensorGuard.PhaseFlow.run_after_setter	lean/TensorGuard/PhaseFlow.lean:92	lean/TensorGuard/PhaseFlow.lean:92_setter (b0 b0' : Bool) (xs : List PhaseOp) (op : PhaseOp) (h : isSetter op = true) : phaseRun b0 (xs ++ [op]) = phaseRun b0' (xs ++ [op])
TensorGuard.PhaseFlow.run_ends_at_value	lean/TensorGuard/PhaseFlow.lean:101	lean/TensorGuard/PhaseFlow.lean:101_at_value (b0 : Bool) (xs : List PhaseOp) (op : PhaseOp) (h : isSetter op = true) : phaseRun b0 (xs ++ [op]) = setterValue op
TensorGuard.PhaseFlow.run_noSettle_id	lean/TensorGuard/PhaseFlow.lean:109	lean/TensorGuard/PhaseFlow.lean:109_id (b0 : Bool) (xs : List PhaseOp) (h : forall op in xs, isSetter op = false) : phaseRun b0 xs = b0
TensorGuard.PixelShuffle.ps_numel	lean/TensorGuard/PixelShuffle.lean:36	lean/TensorGuard/PixelShuffle.lean:36_in c r h w : (Nat) : inNumel n c r h w = outNumel n c r h w
TensorGuard.PixelShuffle.ps_divisible	lean/TensorGuard/PixelShuffle.lean:42	lean/TensorGuard/PixelShuffle.lean:42 (c r : Nat) : (r * r) ? (c * (r * r))
TensorGuard.PixelShuffle.ps_cout	lean/TensorGuard/PixelShuffle.lean:47	lean/TensorGuard/PixelShuffle.lean:47 (r : Nat) (hr : 0 < r) : (c * (r * r)) / (r * r) = c
TensorGuard.PixelShuffle.psValid_iff	lean/TensorGuard/PixelShuffle.lean:54	lean/TensorGuard/PixelShuffle.lean:54 (cin r : Nat) (hr : 0 < r) : psValid cin r = true <-> (r * r) ? cin
TensorGuard.PixelShuffle.ps_construct_valid	lean/TensorGuard/PixelShuffle.lean:61	lean/TensorGuard/PixelShuffle.lean:61_construct_valid (c r : Nat) : psValid (c * (r * r)) r = true
TensorGuard.Pool2d.div_mono	lean/TensorGuard/Pool2d.lean:25	lean/TensorGuard/Pool2d.lean:25_mono {a b k : Nat} (h : a <= b) : a / k <= b / k
TensorGuard.Pool2d.poolOut_identity	lean/TensorGuard/Pool2d.lean:65	lean/TensorGuard/Pool2d.lean:65_identity (input : Nat) (h : 1 <= input) : poolOut input 0 1 1 = input
TensorGuard.Pool2d.poolOut_stride_one	lean/TensorGuard/Pool2d.lean:90	lean/TensorGuard/Pool2d.lean:90_stride_one (input pad kernel : Nat) : poolOut input pad kernel 1 = input + 2 * pad - kernel + 1
TensorGuard.Pool2d.poolOut_mono	lean/TensorGuard/Pool2d.lean:93	lean/TensorGuard/Pool2d.lean:93_stride_mono (i? i? pad kernel stride : Nat) (h : i? <= i?) : poolOut i? pad kernel stride <= poolOut i? pad kernel stride
TensorGuard.Pool2d.poolOut_le	lean/TensorGuard/Pool2d.lean:100	lean/TensorGuard/Pool2d.lean:100_stride_le (input pad kernel stride : Nat) (hk : 1 <= kernel) (hv : kernel <= input + 2 * pad) : poolOut input pad kernel stride <= input + 2 * pad
TensorGuard.Pool2d.poolOut_pos	lean/TensorGuard/Pool2d.lean:99	lean/TensorGuard/Pool2d.lean:99_stride_pos (input pad kernel stride : Nat) : 1 <= poolOut input pad kernel stride
TensorGuard.Pool2d.pool2d_rank	lean/TensorGuard/Pool2d.lean:126	lean/TensorGuard/Pool2d.lean:126_rank (n c hOut wOut : Nat) : (pool2dShape n c hOut wOut).length = 4
TensorGuard.Pool2d.pool2d_batch	lean/TensorGuard/Pool2d.lean:127	lean/TensorGuard/Pool2d.lean:127_batch (n c hOut wOut : Nat) : (pool2dShape n c hOut wOut).head? = some n
TensorGuard.Pool2d.pool2d_channels_preserved	lean/TensorGuard/Pool2d.lean:138	lean/TensorGuard/Pool2d.lean:138_channels_preserved (n c hOut wOut : Nat) : (pool2dShape n c hOut wOut).get? 1 = some c
TensorGuard.RankTransfer.keep_id	lean/TensorGuard/RankTransfer.lean:56	lean/TensorGuard/RankTransfer.lean:56_id (r : Nat) : rankStep r RedOp.reduceKeep = r
TensorGuard.RankTransfer.drop_preced	lean/TensorGuard/RankTransfer.lean:57	lean/TensorGuard/RankTransfer.lean:57_preced (r : Nat) : rankStep r RedOp.reduceDrop = r - 1
TensorGuard.RankTransfer.step_le	lean/TensorGuard/RankTransfer.lean:60	lean/TensorGuard/RankTransfer.lean:60_step_le (r : Nat) (op : RedOp) : rankStep r op <= r
TensorGuard.RankTransfer.rankRun_append	lean/TensorGuard/RankTransfer.lean:70	lean/TensorGuard/RankTransfer.lean:70_rankRun_append (r0 : Nat) (xs ys : List RedOp) : rankRun r0 (xs ++ ys) = rankRun (rankRun r0 xs) ys

TensorGuard.RankTransfer.rankRun	lean4/TensorGuard/RecurrentRule.lean78	sub_countDrop (r0 : Nat) (xs : List RedOp) : rankRun r0 xs = r0 - countDrop xs
TensorGuard.RankTransfer.rankRun	lean4/TensorGuard/RecurrentRule.lean103	(r0 : Nat) (xs : List RedOp) : rankRun r0 xs <= r0
TensorGuard.RankTransfer.rankRun	lean4/TensorGuard/RecurrentRule.lean111	Keep (r0 : Nat) (xs : List RedOp) (h : forall op in xs, op = RedOp.reduceKeep) : rankRun r0 xs = r0
TensorGuard.RankTransfer.rankRun	lean4/TensorGuard/RecurrentRule.lean116	exact (r0 : Nat) (xs : List RedOp) (h : countDrop xs <= r0) : rankRun r0 xs + countDrop xs = r0
TensorGuard.RecurrentRule.batch_first	lean4/TensorGuard/RecurrentRule.lean166	output_preserves_layout (cell : CellKind) (input hidden layers proj batch seq : Nat) (bidir : Bool) : outputShape { cell
TensorGuard.RecurrentRule.time_major	lean4/TensorGuard/RecurrentRule.lean136	output_preserves_layout (cell : CellKind) (input hidden layers proj seq batch : Nat) (bidir : Bool) : outputShape { cell
TensorGuard.RecurrentRule.batch_first	lean4/TensorGuard/RecurrentRule.lean166	state_selects_dim0 (cell : CellKind) (input hidden layers proj batch seq : Nat) : stateShape { cell
TensorGuard.RecurrentRule.time_major	lean4/TensorGuard/RecurrentRule.lean196	state_selects_dim1 (cell : CellKind) (input hidden layers proj seq batch : Nat) : stateShape { cell
TensorGuard.RecurrentRule.bidirectional	lean4/TensorGuard/RecurrentRule.lean226	output_feature_doubles (cell : CellKind) (input hidden layers proj seq batch : Nat) : outputShape { cell
TensorGuard.RecurrentRule.bidirectional	lean4/TensorGuard/RecurrentRule.lean255	state_depth_doubles (cell : CellKind) (input hidden layers proj batch seq : Nat) : stateShape { cell
TensorGuard.RecurrentRule.lstm_cell	lean4/TensorGuard/RecurrentRule.lean288	uses_hidden_size_under_projection (input hidden layers proj batch seq : Nat) : stateShape { cell
TensorGuard.RecurrentRule.gru_cell	lean4/TensorGuard/RecurrentRule.lean308	rejected (input hidden layers proj batch seq : Nat) : stateShape { cell
TensorGuard.RecurrentRule.rnn_cell	lean4/TensorGuard/RecurrentRule.lean321	rejected (input hidden layers proj batch seq : Nat) : stateShape { cell
TensorGuard.RecurrentRule.projected	lean4/TensorGuard/RecurrentRule.lean354	lstm_output_shape : outputShape projectedBiLSTM [3, 5, 11] = some [3, 5, 10]
TensorGuard.RecurrentRule.projected	lean4/TensorGuard/RecurrentRule.lean358	lstm_h_state_shape : stateShape projectedBiLSTM StateKind.hidden [3, 5, 11] = some [4, 3, 5]
TensorGuard.RecurrentRule.projected	lean4/TensorGuard/RecurrentRule.lean368	lstm_c_state_shape : stateShape projectedBiLSTM StateKind.cell [3, 5, 11] = some [4, 3, 9]
TensorGuard.RecurrentRule.time_major	lean4/TensorGuard/RecurrentRule.lean379	gru_output_shape : outputShape timeMajorBiGRU [7, 4, 8] = some [7, 4, 12]
TensorGuard.RecurrentRule.time_major	lean4/TensorGuard/RecurrentRule.lean383	gru_h_state_shape : stateShape timeMajorBiGRU StateKind.hidden [7, 4, 8] = some [6, 4, 6]
TensorGuard.RecurrentRule.unbatched	lean4/TensorGuard/RecurrentRule.lean399	unbatched_output_shape : outputShape unbatchedRNN [9, 6] = some [9, 10]
TensorGuard.RecurrentRule.unbatched	lean4/TensorGuard/RecurrentRule.lean403	unbatched_h_state_shape : stateShape unbatchedRNN StateKind.hidden [9, 6] = some [4, 5]
TensorGuard.RecurrentRule.wrong_input	lean4/TensorGuard/RecurrentRule.lean407	size_rejected : outputShape projectedBiLSTM [3, 5, 12] = none
TensorGuard.RecurrentRule.bad_rank	lean4/TensorGuard/RecurrentRule.lean414	rejected : outputShape projectedBiLSTM [3, 5, 11, 1] = none

TensorGuard.RP.nul_leb_refl	lean/TensorGuard/ReducedProduct.lean:152	Nullity) : n.leb n = true
TensorGuard.RP.tag_leb_refl	lean/TensorGuard/ReducedProduct.lean:155	Tag) : t.leb t = true
TensorGuard.RP.ple_refl	lean/TensorGuard/ReducedProduct.lean:159	(p : PVal) : p.ple p = true
TensorGuard.RP.nul_meet_le_left	lean/TensorGuard/ReducedProduct.lean:163	(a b : Nullity) : (Nullity.meet a b).leb a = true
TensorGuard.RP.nul_meet_le_right	lean/TensorGuard/ReducedProduct.lean:166	(a b : Nullity) : (Nullity.meet a b).leb b = true
TensorGuard.RP.tag_meet_le_left	lean/TensorGuard/ReducedProduct.lean:169	(a b : Tag) : (Tag.meet a b).leb a = true
TensorGuard.RP.tag_meet_le_right	lean/TensorGuard/ReducedProduct.lean:174	(a b : Tag) : (Tag.meet a b).leb b = true
TensorGuard.RP.pmeet_le_left	lean/TensorGuard/ReducedProduct.lean:180	(a b : PVal) : (PVal.pmeet a b).ple a = true
TensorGuard.RP.pmeet_le_right	lean/TensorGuard/ReducedProduct.lean:184	(a b : PVal) : (PVal.pmeet a b).ple b = true
TensorGuard.RP.reduceTagNul_reductive	lean/TensorGuard/ReducedProduct.lean:188	(p : PVal) : (reduceTagNul p).ple p = true
TensorGuard.RP.reduceNulTag_reductive	lean/TensorGuard/ReducedProduct.lean:190	(p : PVal) : (reduceNulTag p).ple p = true
TensorGuard.RP.nul_leb_trans	lean/TensorGuard/ReducedProduct.lean:210	{b c : Nullity} (h1 : a.leb b = true) (h2 : b.leb c = true) : a.leb c = true
TensorGuard.RP.tag_leb_trans	lean/TensorGuard/ReducedProduct.lean:214	{b c : Tag} (h1 : a.leb b = true) (h2 : b.leb c = true) : a.leb c = true
TensorGuard.RP.ple_trans	lean/TensorGuard/ReducedProduct.lean:221	{b c : PVal} (h1 : a.ple b = true) (h2 : b.ple c = true) : a.ple c = true
TensorGuard.RP.botPV_le	lean/TensorGuard/ReducedProduct.lean:227	(p : PVal) : botPV.ple p = true
TensorGuard.RP.reduce_reductive	lean/TensorGuard/ReducedProduct.lean:236	(p : PVal) : (reduce p).ple p = true
TensorGuard.RP.nul_meet_mono	lean/TensorGuard/ReducedProduct.lean:257	{a' b b' : Nullity} (ha : a.leb a' = true) (hb : b.leb b' = true) : (Nullity.meet a b).leb (Nullity.meet a' b') = true
TensorGuard.RP.tag_meet_mono	lean/TensorGuard/ReducedProduct.lean:263	{a' b b' : Tag} (ha : a.leb a' = true) (hb : b.leb b' = true) : (Tag.meet a b).leb (Tag.meet a' b') = true
TensorGuard.RP.pmeet_mono	lean/TensorGuard/ReducedProduct.lean:274	{a' b b' : PVal} (ha : a.ple a' = true) (hb : b.ple b' = true) : (PVal.pmeet a b).ple (PVal.pmeet a' b') = true
TensorGuard.RP.reduce_mono_consistent	lean/TensorGuard/ReducedProduct.lean:290	{a b : PVal} (hca : consistent a = true) (hcb : consistent b = true) (h : a.ple b = true) : (reduce a).ple (reduce b) = true
TensorGuard.RP.gamma_mono	lean/TensorGuard/ReducedProduct.lean:336	{b : PVal} {c : CVal} (h : a.ple b = true) (hc : mem c a = true) : mem c b = true
TensorGuard.RP.pmeet_gamma	lean/TensorGuard/ReducedProduct.lean:345	(b : PVal) (c : CVal) : mem c (PVal.pmeet a b) = (mem c a && mem c b)
TensorGuard.RP.reduce_gamma	lean/TensorGuard/ReducedProduct.lean:356	(p : PVal) (c : CVal) : mem c (reduce p) = mem c p
TensorGuard.ReshapeInfer.prod_append	lean/TensorGuard/ReshapeInfer.lean:412	(xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.ReshapeInfer.reshapeValid_iff	lean/TensorGuard/ReshapeInfer.lean:611	(total : Nat) (known : List Nat) : reshapeValid total known = true <-> (0 < prod known /\ prod known ? total)
TensorGuard.ReshapeInfer.reshapeValid_imp_dvd	lean/TensorGuard/ReshapeInfer.lean:70	(total : Nat) (known : List Nat) (h : reshapeValid total known = true) : prod known ? total

TensorGuard.ReshapeInfer.nondivisible	lean4/TensorGuard/ReshapeInfer/176	ReshapeInfer.nondivisible76 flagged (total : Nat) (known : List Nat) (h : not prod known ? total) : reshapeValid total known = false
TensorGuard.ReshapeInfer.inferDim_spec	lean4/TensorGuard/ReshapeInfer/189	ReshapeInfer.inferDim189 spec (total : Nat) (known : List Nat) (_hpos : 0 < prod known) (hdvd : prod known ? total) : prod known * inferDim total known = total
TensorGuard.ReshapeInfer.prod_reshapeValid	lean4/TensorGuard/ReshapeInfer/197	ReshapeInfer.prod_reshapeValid197 valid (total : Nat) (known : List Nat) (h : reshapeValid total known = true) : prod (reshapeShape total known) = total
TensorGuard.ReshapeInfer.reshape_inferDim	lean4/TensorGuard/ReshapeInfer/199	ReshapeInfer.reshape_inferDim199 position (total : Nat) (known : List Nat) : (reshapeShape total known).getLast? = some (inferDim total known)
TensorGuard.SDPA.bcDim_one_left	lean4/TensorGuard/SDPA/bcDim/158	SDPA.bcDim158 one_left (a : Nat) : bcDim 1 a = some a
TensorGuard.SDPA.bcDim_one_right	lean4/TensorGuard/SDPA/bcDim/162	SDPA.bcDim162 one_right (a : Nat) : bcDim a 1 = some a
TensorGuard.SDPA.bcDim_self	lean4/TensorGuard/SDPA/bcDim/166	SDPA.bcDim166 self (a : Nat) : bcDim a a = some a
TensorGuard.SDPA.bcDim_none_iff	lean4/TensorGuard/SDPA/bcDim/171	SDPA.bcDim171 none_iff (a b : Nat) : bcDim a b = none <-> (a != 1 /\ b != 1 /\ a != b)
TensorGuard.SDPA.bcRev_same	lean4/TensorGuard/SDPA/bcRev/177	SDPA.bcRev177 same (xs : List Nat) : bcRev xs xs = some xs
TensorGuard.SDPA.bcRev_nil_right	lean4/TensorGuard/SDPA/bcRev/183	SDPA.bcRev183 nil_right (xs : List Nat) : bcRev xs [] = some xs
TensorGuard.SDPA.bcShape_same	lean4/TensorGuard/SDPA/bcShape/186	SDPA.bcShape186 same (xs : List Nat) : bcShape xs xs = some xs
TensorGuard.SDPA.bcRev_append	lean4/TensorGuard/SDPA/bcRev/191	SDPA.bcRev191 append_same_left (xs extra : List Nat) : bcRev (xs ++ extra) xs = some (xs ++ extra)
TensorGuard.SDPA.bcShape_suffix	lean4/TensorGuard/SDPA/bcShape/199	SDPA.bcShape199 suffix_same (pre suffix : List Nat) : bcShape (pre ++ suffix) suffix = some (pre ++ suffix)
TensorGuard.SDPA.standard_output_shape	lean4/TensorGuard/SDPA/standard/126	SDPA.standard126 output_shape (qLead kLead vLead lead : List Nat) (lq eq ev : Nat) (hbc : bc3 qLead kLead vLead = some lead) : standardOutput? qLead kLead vLead lq eq eq ev = some (lead ++ [lq, ev])
TensorGuard.SDPA.standard_output_rank	lean4/TensorGuard/SDPA/standard/131	SDPA.standard131 output_rank (lead : List Nat) (lq ev : Nat) : (lead ++ [lq, ev]).length = lead.length + 2
TensorGuard.SDPA.standard_equal_leads	lean4/TensorGuard/SDPA/standard/135	SDPA.standard135 equal_leads (lead : List Nat) (lq eq ev : Nat) : standardOutput? lead lead lead lq eq eq ev = some (lead ++ [lq, ev])
TensorGuard.SDPA.mask_exact_valid	lean4/TensorGuard/SDPA/mask/141	SDPA.mask141 exact_valid (lead : List Nat) (lq lk : Nat) : maskValid lead lq lk (scoreShape lead lq lk) = true
TensorGuard.SDPA.mask_trailing_valid	lean4/TensorGuard/SDPA/mask/147	SDPA.mask147 trailing_valid (lead : List Nat) (lq lk : Nat) : maskValid lead lq lk [lq, lk] = true
TensorGuard.SDPA.gqaHeadsValid_iff	lean4/TensorGuard/SDPA/gqa/174	SDPA.gqa174 headsValid_iff (hq hk hv : Nat) : gqaHeadsValid hq hk hv = true <-> (0 < hk /\ hk ? hq /\ 0 < hv /\ hv ? hq)
TensorGuard.SDPA.gqa_key_repetition_count	lean4/TensorGuard/SDPA/gqa/180	SDPA.gqa180 key_repetition_count (hq hk hv : Nat) (h : gqaHeadsValid hq hk hv = true) : hk * (hq / hk) = hq
TensorGuard.SDPA.gqa_value_repetition_count	lean4/TensorGuard/SDPA/gqa/186	SDPA.gqa186 value_repetition_count (hq hk hv : Nat) (h : gqaHeadsValid hq hk hv = true) : hv * (hq / hv) = hq
TensorGuard.SDPA.gqa_nondivisible_key_flagged	lean4/TensorGuard/SDPA/gqa/192	SDPA.gqa192 nondivisible_key_flagged (hq hk hv : Nat) (h : not hk ? hq) : gqaHeadsValid hq hk hv = false

TensorGuard.SDPA.gqa_nondivisible_value	lean4/TensorGuard/SDPA/eq197	SDPA.gqa_nondivisible_value_flagged (hq hk hv : Nat) (h : not hv ? hq) : gqaHeadsValid hq hk hv = false
TensorGuard.SDPA.gqa_output_shape	lean4/TensorGuard/SDPA/eq202	SDPA.gqa_output_shape (qBatch kBatch vBatch batch : List Nat) (hq hk hv lq eq ev : Nat) (hheads : gqaHeadsValid hq hk hv = true) (hbc : bc3 qBatch kBatch vBatch = some batch) : gqaOutput? qBatch kBatch vBatch hq hk hv lq eq eq ev = some (batch ++ [hq, lq, ev])
TensorGuard.SDPA.gqa_output_rank	lean4/TensorGuard/SDPA/eq240	SDPA.gqa_output_rank (batch : List Nat) (hq lq ev : Nat) : (batch ++ [hq, lq, ev]).length = batch.length + 3
TensorGuard.SDPA.gqa_output_uses_query_heads	lean4/TensorGuard/SDPA/eq244	SDPA.gqa_output_uses_query_heads (batch : List Nat) (hq lq ev : Nat) : (batch ++ [hq, lq, ev]).get? batch.length = some hq
TensorGuard.SDPA.gqa_prefix_broadcast_required	lean4/TensorGuard/SDPA/eq220	SDPA.gqa_prefix_broadcast_required (qBatch kBatch vBatch : List Nat) (hq hk hv lq eq ev : Nat) (hheads : gqaHeadsValid hq hk hv = true) (hbc : bc3 qBatch kBatch vBatch = none) : gqaOutput? qBatch kBatch vBatch hq hk hv lq eq eq ev = none
TensorGuard.SmtEncoding.sat_iff_eq	lean4/TensorGuard/SmtEncoding/eq50	SmtEncoding.sat_iff_eq {alpha : Type} (ca cb : alpha) : Sat ca cb <=> ca = cb
TensorGuard.SmtEncoding.unsat_iff_eq	lean4/TensorGuard/SmtEncoding/eq57	SmtEncoding.unsat_iff_eq {alpha : Type} (ca cb : alpha) : not Sat ca cb <=> ca != cb
TensorGuard.SmtEncoding.unsat_sound	lean4/TensorGuard/SmtEncoding/eq61	SmtEncoding.unsat_sound {alpha : Type} (ca cb : alpha) (h : not Sat ca cb) : ca != cb
TensorGuard.SmtEncoding.eq_is_sat	lean4/TensorGuard/SmtEncoding/eq66	SmtEncoding.eq_is_sat {alpha : Type} (c : alpha) : Sat c c
TensorGuard.SmtEncoding.device_sat	lean4/TensorGuard/SmtEncoding/eq75	SmtEncoding.device_sat matches_devBug (ca cb : Dev) (ha : ca != Dev.unknown) (hb : cb != Dev.unknown) : (not Sat ca cb) <=> devBug ca cb = true
TensorGuard.SmtEncoding.phase_sat	lean4/TensorGuard/SmtEncoding/eq96	SmtEncoding.phase_sat unsat_iff_ne (pa pb : Phase) : (not Sat pa pb) <=> pa != pb
TensorGuard.SmtEncoding.phase_sat_iff	lean4/TensorGuard/SmtEncoding/eq102	SmtEncoding.phase_sat_iff (p : Phase) : Sat p p
TensorGuard.SmtEncoding.grad_sat	lean4/TensorGuard/SmtEncoding/eq111	SmtEncoding.grad_sat unsat_iff_ne (gset greg : Bool) : (not Sat gset greg) <=> gset != greg
TensorGuard.SmtEncoding.grad_consistent	lean4/TensorGuard/SmtEncoding/eq114	SmtEncoding.grad_consistent (g : Bool) : Sat g g
TensorGuard.SmtEncoding.dtype_sat	lean4/TensorGuard/SmtEncoding/eq121	SmtEncoding.dtype_sat unsat_iff_ne (da db : Dt) : (not Sat da db) <=> da != db
TensorGuard.SmtEncoding.dtype_sat_iff	lean4/TensorGuard/SmtEncoding/eq133	SmtEncoding.dtype_sat_iff matches_dtMatmulBug (da db : Dt) (ha : da != Dt.unknown) (hb : db != Dt.unknown) : (not Sat da db) <=> dtMatmulBug da db = true
TensorGuard.SmtEncoding.dtype_sat_iff	lean4/TensorGuard/SmtEncoding/eq150	SmtEncoding.dtype_sat_iff (d : Dt) : Sat d d
TensorGuard.SmtEncoding.broadcast_sat	lean4/TensorGuard/SmtEncoding/eq169	SmtEncoding.broadcast_sat unsat_iff_bcDim_some (a b : Nat) : BroadcastSat a b <=> exists out, BroadcastChain.bcDim a b = some out
TensorGuard.SmtEncoding.broadcast_sat	lean4/TensorGuard/SmtEncoding/eq186	SmtEncoding.broadcast_sat unsat_iff_bcDim_none (a b : Nat) : not BroadcastSat a b <=> BroadcastChain.bcDim a b = none
TensorGuard.SmtEncoding.broadcast_sat	lean4/TensorGuard/SmtEncoding/eq202	SmtEncoding.broadcast_sat unsat_iff_incompatible (a b : Nat) : not BroadcastSat a b <=> (a != 1 /\ b != 1 /\ a != b)
TensorGuard.SmtEncoding.divisibility_sat	lean4/TensorGuard/SmtEncoding/eq249	SmtEncoding.divisibility_sat_iff_reshapeValid (total : Nat) (known : List Nat) : DivisibilitySat total known <=> ReshapeInfer.reshapeValid total known = true

TensorGuard.V5.applyOp_sound_batched_matmul	TensorGuard.SoundnessV5Ops37	batched_matmul (b m k n : Nat) : batched_matmul [b, m, k] [b, k, n] = some [b, m, n]
TensorGuard.V5.applyOp_sound_conv1d	TensorGuard.SoundnessV5Ops43	conv1d (n cin cout cinG k l pad dil stride groups l_out : Nat) (hG : groups != 0) (hC : cin = cinG * groups) (hSp : convSpatial l pad dil k stride = some l_out) : conv1d [n, cin, l] [cout, cinG, k] pad dil stride groups = some [n, cout, l_out]
TensorGuard.V5.applyOp_sound_conv2d	TensorGuard.SoundnessV5Ops53	conv2d (n cin cout cinG h w hO wO pH pW dH dW kH kW sH sW groups : Nat) (hG : groups != 0) (hC : cin = cinG * groups) (h1 : convSpatial h pH dH kH sH = some hO) (h2 : convSpatial w pW dW kW sW = some wO) : conv2d [n, cin, h, w] [cout, cinG, kH, kW] pH pW dH dW kH kW sH sW groups = some [n, cout, hO, wO]
TensorGuard.V5.applyOp_sound_conv3d	TensorGuard.SoundnessV5Ops63	conv3d (input weight : Sh) (pD pH pW dD dH dW kD kH kW sD sH sW groups : Nat) (out : Sh) (h : conv3d input weight pD pH pW dD dH dW kD kH kW sD sH sW groups = some out) : exists s, conv3d input weight pD pH pW dD dH dW kD kH kW sD sH sW groups = some s
TensorGuard.V5.applyOp_sound_conv_transpose2d	TensorGuard.SoundnessV5Ops73	conv_transpose2d (input weight : Sh) (pH pW dH dW kH kW sH sW oH oW groups : Nat) (out : Sh) (h : conv_transpose2d input weight pH pW dH dW kH kW sH sW oH oW groups = some out) : exists s, conv_transpose2d input weight pH pW dH dW kH kW sH sW oH oW groups = some s
TensorGuard.V5.applyOp_sound_view_v5	TensorGuard.SoundnessV5Ops79	view_v5 (input out r : Sh) (h : view input out = some r) : r = out /\ prodL input = prodL out
TensorGuard.V5.applyOp_sound_reshape	TensorGuard.SoundnessV5Ops85	reshape (input : Sh) (out : List (Option Nat)) (r : Sh) (h : reshape input out = some r) : exists s, reshape input out = some s
TensorGuard.V5.applyOp_sound_permute	TensorGuard.SoundnessV5Ops91	permute (input : Sh) (perm : List Nat) (out : Sh) (h : permute input perm = some out) : exists s, permute input perm = some s
TensorGuard.V5.applyOp_sound_transpose	TensorGuard.SoundnessV5Ops97	transpose (input : Sh) (i j : Nat) (out : Sh) (h : transposeAt input i j = some out) : exists s, transposeAt input i j = some s
TensorGuard.V5.applyOp_sound_expand	TensorGuard.SoundnessV5Ops103	expand (input : Sh) (target : List (Option Nat)) (out : Sh) (h : expand input target = some out) : exists s, expand input target = some s
TensorGuard.V5.applyOp_sound_repeat	TensorGuard.SoundnessV5Ops109	repeat (input : Sh) (reps : List Nat) (out : Sh) (h : repeatOp input reps = some out) : exists s, repeatOp input reps = some s
TensorGuard.V5.applyOp_sound_broadcast_to	TensorGuard.SoundnessV5Ops115	broadcast_to (input target out : Sh) (h : broadcast _to input target = some out) : out = target
TensorGuard.V5.applyOp_sound_cat	TensorGuard.SoundnessV5Ops121	cat (shapes : List Sh) (axis : Nat) (out : Sh) (h : cat shapes axis = some out) : exists s, cat shapes axis = some s
TensorGuard.V5.applyOp_sound_stack	TensorGuard.SoundnessV5Ops127	stack (shapes : List Sh) (axis : Nat) (out : Sh) (h : stackOp shapes axis = some out) : exists s, stackOp shapes axis = some s

TensorGuard.V5.applyOp_sound_split	lean/TensorGuard	TensorGuard.V5.Ops.133	find_split (input : Sh) (axis chunk_size : Nat) (out : List Sh) (h : splitOp input axis chunk_size = some out) : exists s, splitOp input axis chunk_size = some s
TensorGuard.V5.applyOp_sound_chunk	lean/TensorGuard	TensorGuard.V5.Ops.139	find_chunk (input : Sh) (axis n : Nat) (out : List Sh) (h : chunkOp input axis n = some out) : exists s, chunkOp input axis n = some s
TensorGuard.V5.applyOp_sound_unbind	lean/TensorGuard	TensorGuard.V5.Ops.145	find_unbind (input : Sh) (axis : Nat) (out : List Sh) (h : unbind input axis = some out) : exists s, unbind input axis = some s
TensorGuard.V5.applyOp_sound_gather	lean/TensorGuard	TensorGuard.V5.Ops.151	find_gather (input index : Sh) (axis : Nat) (out : Sh) (h : gather input index axis = some out) : out = index
TensorGuard.V5.applyOp_sound_scatter	lean/TensorGuard	TensorGuard.V5.Ops.157	find_scatter (input index src : Sh) (axis : Nat) (out : Sh) (h : scatter input index src axis = some out) : out = input
TensorGuard.V5.applyOp_sound_index_select	lean/TensorGuard	TensorGuard.V5.Ops.163	find_index_select (input : Sh) (axis index_len : Nat) (out : Sh) (h : index_select input axis index_len = some out) : exists s, index_select input axis index_len = some s
TensorGuard.V5.applyOp_sound_narrow	lean/TensorGuard	TensorGuard.V5.Ops.169	find_narrow (input : Sh) (axis start length : Nat) (out : Sh) (h : narrow input axis start length = some out) : exists s, narrow input axis start length = some s
TensorGuard.V5.applyOp_sound_embed	lean/TensorGuard	TensorGuard.V5.Ops.175	find_embed (input : Sh) (numEmbeddings embed_dim : Nat) : embedOp input numEmbeddings embed_dim = input ++ [embed_dim]
TensorGuard.V5.applyOp_sound_layer_norm	lean/TensorGuard	TensorGuard.V5.Ops.181	find_layer_norm (input normalized out : Sh) (h : layer_norm input normalized = some out) : out = input
TensorGuard.V5.applyOp_sound_rms_norm	lean/TensorGuard	TensorGuard.V5.Ops.187	find_rms_norm (input out : Sh) (k : Nat) (h : rms_norm input k = some out) : out = input
TensorGuard.V5.applyOp_sound_scaled_dot_product_attention	lean/TensorGuard	TensorGuard.V5.Ops.193	find_scaled_dot_product_attention (q k v : Sh) (out : Sh) (h : sdpa q k v = some out) : exists s, sdpa q k v = some s
TensorGuard.V5.applyOp_sound_linear_v5	lean/TensorGuard	TensorGuard.V5.Ops.199	find_linear_v5 (input : Sh) (in_f out_f : Nat) (out : Sh) (h : linear input in_f out_f = some out) : exists s, linear input in_f out_f = some s
TensorGuard.V5.applyOp_sound_to	lean/TensorGuard	TensorGuard.V5.Ops.205	find_to (input : Sh) : toOp input = input
TensorGuard.V5.applyOp_sound_dropout	lean/TensorGuard	TensorGuard.V5.Ops.211	find_dropout (input : Sh) : dropoutOp input = input
TensorGuard.V5.applyOp_sound_contiguous	lean/TensorGuard	TensorGuard.V5.Ops.221	find_contiguous (input : Sh) : contiguousOp input = input
TensorGuard.V5.applyOp_sound_clamp	lean/TensorGuard	TensorGuard.V5.Ops.227	find_clamp (input : Sh) : clampOp input = input
TensorGuard.V5.applyOp_sound_squeeze	lean/TensorGuard	TensorGuard.V5.Ops.233	find_squeeze (input : Sh) (axis : Nat) (out : Sh) (h : squeezeOp input axis = some out) : exists s, squeezeOp input axis = some s
TensorGuard.V5.applyOp_sound_unsqueeze	lean/TensorGuard	TensorGuard.V5.Ops.251	find_unsqueeze (input : Sh) (axis : Nat) (out : Sh) (h : unsqueezeOp input axis = some out) : out.length = input.length + 1
TensorGuard.V5.applyOp_sound_argmax	lean/TensorGuard	TensorGuard.V5.Ops.271	find_argmax (input : Sh) (axis : Nat) (out : Sh) (h : argmaxOp input axis = some out) : out.length = input.length - 1

TensorGuard.V5.applyOp_sound_cro	lean/lean4/TensorGuard/SoundnessV5Ops.lean	290	crossEntropyOp (input target out : Sh) (h : crossEntropyOp input target = some out) : out = []
TensorGuard.SparseLayouts.dense_ma	lean/lean4/TensorGuard/SparseLayouts.lean	63	matmulInitialization_shape_sound (spec : SparseSpec) : denseShapeOf spec = spec.shape
TensorGuard.SparseLayouts.mkAccept	lean/lean4/TensorGuard/SparseLayouts.lean	66	dense_shape_sound (ok : Bool) (spec : SparseSpec) : acceptedShape spec.shape (mkAccepted ok spec)
TensorGuard.SparseLayouts.coo234_	lean/lean4/TensorGuard/SparseLayouts.lean	73	cooSpec [2, 3] [3, 4] [2, 3, 4] = some { layout
TensorGuard.SparseLayouts.csr23_	lean/lean4/TensorGuard/SparseLayouts.lean	186	csrSpec [3] [3] [3] [2, 3] = some { layout
TensorGuard.SparseLayouts.csc23_	lean/lean4/TensorGuard/SparseLayouts.lean	199	cscSpec [4] [3] [3] [2, 3] = some { layout
TensorGuard.SparseLayouts.bsr43_	lean/lean4/TensorGuard/SparseLayouts.lean	212	bsrSpec [3] [2] [2, 2, 3] [4, 3] = some { layout
TensorGuard.SparseLayouts.bsc23_	lean/lean4/TensorGuard/SparseLayouts.lean	235	bscSpec [2] [1] [1, 2, 3] [2, 3] = some { layout
TensorGuard.SparseLayouts.batched_	lean/lean4/TensorGuard/SparseLayouts.lean	238	accepts : csrSpec [2, 3] [2, 3] [2, 3, 4] [2, 2, 3, 4] = some { layout
TensorGuard.SparseLayouts.batched_	lean/lean4/TensorGuard/SparseLayouts.lean	251	accepts : bsrSpec [2, 3] [2, 2] [2, 2, 2, 3, 5] [2, 4, 3, 5] = some { layout
TensorGuard.SparseLayouts.coo234_	lean/lean4/TensorGuard/SparseLayouts.lean	268	Dense_shape : Option.map denseShapeOf (cooSpec [2, 3] [3, 4] [2, 3, 4]) = some [2, 3, 4]
TensorGuard.SparseLayouts.csr23_	lean/lean4/TensorGuard/SparseLayouts.lean	272	Dense_shape : Option.map denseShapeOf (csrSpec [3] [3] [3] [2, 3]) = some [2, 3]
TensorGuard.SparseLayouts.csc23_	lean/lean4/TensorGuard/SparseLayouts.lean	276	Dense_shape : Option.map denseShapeOf (cscSpec [4] [3] [3] [2, 3]) = some [2, 3]
TensorGuard.SparseLayouts.bsr43_	lean/lean4/TensorGuard/SparseLayouts.lean	280	Dense_shape : Option.map denseShapeOf (bsrSpec [3] [2] [2, 2, 3] [4, 3]) = some [4, 3]
TensorGuard.SparseLayouts.bsc23_	lean/lean4/TensorGuard/SparseLayouts.lean	284	Dense_shape : Option.map denseShapeOf (bscSpec [2] [1] [1, 2, 3] [2, 3]) = some [2, 3]
TensorGuard.SparseLayouts.batched_	lean/lean4/TensorGuard/SparseLayouts.lean	288	Dense_shape : Option.map denseShapeOf (csrSpec [2, 3] [2, 3] [2, 3, 4] [2, 2, 3, 4]) = some [2, 2, 3, 4]
TensorGuard.SparseLayouts.batched_	lean/lean4/TensorGuard/SparseLayouts.lean	292	Dense_shape : Option.map denseShapeOf (bsrSpec [2, 3] [2, 2] [2, 2, 2, 3, 5] [2, 4, 3, 5]) = some [2, 4, 3, 5]
TensorGuard.SparseLayouts.csr_bad_	lean/lean4/TensorGuard/SparseLayouts.lean	300	compressed_length_rejected : csrSpec [2] [3] [3] [2, 3] = none
TensorGuard.SparseLayouts.csc_bad_	lean/lean4/TensorGuard/SparseLayouts.lean	303	compressed_length_rejected : cscSpec [3] [3] [3] [2, 3] = none
TensorGuard.SparseLayouts.bsr_bad_	lean/lean4/TensorGuard/SparseLayouts.lean	306	divisibility_rejected : bsrSpec [3] [2] [2, 2, 3] [5, 3] = none
TensorGuard.SparseLayouts.bsr_bad_	lean/lean4/TensorGuard/SparseLayouts.lean	309	hmn_divisibility_rejected : bsrSpec [3] [2] [2, 2, 3] [4, 4] = none
TensorGuard.SparseLayouts.compressed	lean/lean4/TensorGuard/SparseLayouts.lean	312	tail_mismatch_rejected : csrSpec [3] [3] [3, 5] [2, 3, 4] = none
TensorGuard.SubjectReduction.pos_	lean/lean4/TensorGuard/SubjectReduction.lean	42	Sh} (ha : PosShape a) (hb : PosShape b) : PosShape (a ++ b)
TensorGuard.SubjectReduction.pos_	lean/lean4/TensorGuard/SubjectReduction.lean	49	{hd : 0 < d} : PosShape [d]

TensorGuard.SubjectReduction.pos_shape	lean3/TensorGuard/SubjectReduction.lean:55	SubjectReduction.lean:55 {rest : Sh} (hd : 0 < d) (hr : PosShape rest) : PosShape (d :: rest)
TensorGuard.SubjectReduction.prod_shape	lean3/TensorGuard/SubjectReduction.lean:57	SubjectReduction.lean:57 (hs : PosShape s) : 0 < prod s
TensorGuard.SubjectReduction.shapesWF	lean3/TensorGuard/SubjectReduction.lean:76	SubjectReduction.lean:76 {a b : List Sh} (ha : ShapesWF a) (hb : ShapesWF b) : ShapesWF (a ++ b)
TensorGuard.SubjectReduction.shapesWF_singleton	lean3/TensorGuard/SubjectReduction.lean:83	SubjectReduction.lean:83 {s : Sh} (hs : PosShape s) : ShapesWF [s]
TensorGuard.SubjectReduction.shapesWF_pair	lean3/TensorGuard/SubjectReduction.lean:89	SubjectReduction.lean:89 {a b : Sh} (ha : PosShape a) (hb : PosShape b) : ShapesWF [a, b]
TensorGuard.SubjectReduction.op_output	lean3/TensorGuard/SubjectReduction.lean:84	SubjectReduction.lean:84 (op : SLOp) (h : OpWF op) : ShapesWF (outputShapes op)
TensorGuard.SubjectReduction.step_ifwelltyped	lean3/TensorGuard/SubjectReduction.lean:211	SubjectReduction.lean:211 {env : Env} {stmt : Stmt} (h : WellTypedStmt env stmt) : step env stmt = some (env ++ outputShapes stmt.op)
TensorGuard.SubjectReduction.step_subject	lean3/TensorGuard/SubjectReduction.lean:247	SubjectReduction.lean:247 {env env' : Env} {stmt : Stmt} (hEnv : EnvWF env) (hStmt : WellTypedStmt env stmt) (hStep : step env stmt = some env') : EnvWF env'
TensorGuard.SubjectReduction.exec_subject	lean3/TensorGuard/SubjectReduction.lean:243	SubjectReduction.lean:243 {env env' : Env} {program : Program} (hEnv : EnvWF env) (hWT : ProgramWT env program) (hExec : exec env program = some env') : EnvWF env'
TensorGuard.SubjectReduction.whole_env	lean3/TensorGuard/SubjectReduction.lean:262	SubjectReduction.lean:262 {env env' : Env} {program : Program} (hEnv : EnvWF env) (hWT : ProgramWT env program) (hExec : exec env program = some env') : EnvWF env'
TensorGuard.SubjectReduction.program_inputs_have_positive_shapes	lean3/TensorGuard/SubjectReduction.lean:269	SubjectReduction.lean:269 {env env' : Env} {program : Program} (hEnv : EnvWF env) (hWT : ProgramWT env program) (hExec : exec env program = some env') : forall s, s in env' -> PosShape s
TensorGuard.SubjectReduction.env_join	lean3/TensorGuard/SubjectReduction.lean:294	SubjectReduction.lean:294 {thenEnv elseEnv : Env} (hThen : EnvWF thenEnv) (hJoin : envJoin thenEnv elseEnv = some joined) : EnvWF joined
TensorGuard.SubjectReduction.cond_exec	lean3/TensorGuard/SubjectReduction.lean:345	SubjectReduction.lean:345 {env env' : Env} {cond : BranchCond} {thenBranch elseBranch : Program} (condExec env cond thenBranch elseBranch = some env' <-> cond = BranchCond.supported /\ exists thenEnv elseEnv, exec env thenBranch = some thenEnv /\ exec env elseBranch = some elseEnv /\ envJoin thenEnv elseEnv = some env')
TensorGuard.SubjectReduction.sound_branch	lean3/TensorGuard/SubjectReduction.lean:345	SubjectReduction.lean:345 {env env' : Env} {cond : BranchCond} {thenBranch elseBranch : Program} (hSome : condExec env cond thenBranch elseBranch = some env') : cond = BranchCond.supported
TensorGuard.SubjectReduction.unsupported_branch_cannot_silently_safe	lean3/TensorGuard/SubjectReduction.lean:353	SubjectReduction.lean:353 {env env' : Env} {thenBranch elseBranch : Program} (hSome : condExec env BranchCond.unsupported thenBranch elseBranch != some env')

TensorGuard.SubjectReduction.supported	lean/TensorGuard/SubjectReduction.lean:360	Additional subject_reduction {env env' : Env} {thenBranch elseBranch : Program} (hEnv : EnvWF env) (hThenWT : ProgramWT env thenBranch) (hElseWT : ProgramWT env elseBranch) (hExec : condExec env BranchCond.supported thenBranch elseBranch = some env') : EnvWF env'
TensorGuard.SubjectReduction.conditional	lean/TensorGuard/SubjectReduction.lean:381	program_subject_reduction {env joinedEnv env' : Env} {thenBranch elseBranch tail : Program} (hEnv : EnvWF env) (hThenWT : ProgramWT env thenBranch) (hElseWT : ProgramWT env elseBranch) (hTailWT : ProgramWT joinedEnv tail) (hCond : condExec env BranchCond.supported thenBranch elseBranch = some joinedEnv) (hExec : condThenExec env BranchCond.supported thenBranch elseBranch tail = some env') : EnvWF env'
TensorGuard.SubjectReduction.exec	lean/TensorGuard/SubjectReduction.lean:427	subject_reduction {env env' : Env} {bodies : List Program} (hEnv : EnvWF env) (hWT : ProgramListWT env bodies) (hExec : execProgramList env bodies = some env') : EnvWF env'
TensorGuard.SubjectReduction.bounded	lean/TensorGuard/SubjectReduction.lean:453	and_within_infix_supported_and_with {env env' : Env} {kind : LoopKind} {bodies : List Program} {limit : Nat} (hExec : boundedUnrollExec env kind bodies limit = some env') : kind != LoopKind.unsupported /\ bodies.length <= limit /\ execProgramList env bodies = some env'
TensorGuard.SubjectReduction.moduleList	lean/TensorGuard/SubjectReduction.lean:479	and_unroll_limit_abstains {env : Env} {bodies : List Program} {limit : Nat} (hTooMany : limit < bodies.length) : boundedUnrollExec env LoopKind.moduleList bodies limit = none
TensorGuard.SubjectReduction.staticRange	lean/TensorGuard/SubjectReduction.lean:485	and_unroll_limit_abstains {env : Env} {bodies : List Program} {limit : Nat} (hTooMany : limit < bodies.length) : boundedUnrollExec env LoopKind.staticRange bodies limit = none
TensorGuard.SubjectReduction.unsupported	lean/TensorGuard/SubjectReduction.lean:491	loop_cannot_silently_safe {env env' : Env} {bodies : List Program} {limit : Nat} : boundedUnrollExec env LoopKind.unsupported bodies limit != some env'
TensorGuard.SubjectReduction.bounded	lean/TensorGuard/SubjectReduction.lean:495	subject_reduction {env env' : Env} {kind : LoopKind} {bodies : List Program} {limit : Nat} (hEnv : EnvWF env) (hWT : ProgramListWT env bodies) (hExec : boundedUnrollExec env kind bodies limit = some env') : EnvWF env'
TensorGuard.SubjectReduction.mlp	lean/TensorGuard/SubjectReduction.lean:664	exec [[2, 8]] mlpProgram = some [[2, 8], [2, 16], [2, 16], [2, 16], [2, 4]]
TensorGuard.SubjectReduction.cnn	lean/TensorGuard/SubjectReduction.lean:676	shape : exec [[2, 3, 8, 8]] cnnHeadProgram = some [[2, 3, 8, 8], [2, 6, 8, 8], [2, 6, 8, 8], [2, 384], [2, 10]]
TensorGuard.SubjectReduction.indexing	lean/TensorGuard/SubjectReduction.lean:686	shape : exec [[4, 3], [4, 2]] indexingProgram = some [[4, 3], [4, 2], [4, 2], [4, 7]]
TensorGuard.SubjectReduction.attention	lean/TensorGuard/SubjectReduction.lean:696	shape : exec [[2, 4, 5, 8], [2, 4, 7, 8], [2, 4, 7, 9]] attentionProgram = some [[2, 4, 5, 8], [2, 4, 7, 8], [2, 4, 7, 9], [2, 4, 5, 9], [2, 4, 5, 16]]
TensorGuard.SubjectReduction.conditional	lean/TensorGuard/SubjectReduction.lean:717	exec_shape : condExec [[2, 8]] BranchCond.supported branchThenProgram branchElseProgram = some [[2, 8], [2, 16], [2, 16]]

TensorGuard.SubjectReduction.conditionals	lean/TensorGuard/SubjectReduction.lean	722	tail_exec_shape : condThenExec [[2, 8]] BranchCond.supported branchThenProgram branchElseProgram branchTailProgram = some [[2, 8], [2, 16], [2, 16], [2, 4]]
TensorGuard.SubjectReduction.divergent	lean/TensorGuard/SubjectReduction.lean	732	join_rejected : condExec [[2, 8]] BranchCond.supported branchThenProgram branchElseDivergentProgram = none
TensorGuard.SubjectReduction.unsupported	lean/TensorGuard/SubjectReduction.lean	737	branch_abstains_example : condExec [[2, 8]] BranchCond.unsupported branchThenProgram branchElseProgram = none
TensorGuard.SubjectReduction.moduleList	lean/TensorGuard/SubjectReduction.lean	758	exec_shape : boundedUnrollExec [[2, 8]] LoopKind.moduleList moduleListUnrollBodies 3 = some [[2, 8], [2, 16], [2, 16], [2, 16], [2, 16], [2, 4]]
TensorGuard.SubjectReduction.static_range	lean/TensorGuard/SubjectReduction.lean	776	all_exec_shape : boundedUnrollExec [[2, 8]] LoopKind.staticRange staticRangeUnrollBodies 3 = some [[2, 8], [2, 8], [2, 8], [2, 4]]
TensorGuard.SubjectReduction.moduleListBeyondLimit	lean/TensorGuard/SubjectReduction.lean	782	limit_rejected : boundedUnrollExec [[2, 8]] LoopKind.moduleList moduleListUnrollBodies 2 = none
TensorGuard.SubjectReduction.staticRangeBeyondLimit	lean/TensorGuard/SubjectReduction.lean	785	beyond_limit_rejected : boundedUnrollExec [[2, 8]] LoopKind.staticRange staticRangeUnrollBodies 2 = none
TensorGuard.SubjectReduction.unsupportedLoop	lean/TensorGuard/SubjectReduction.lean	792	loop_abstains_example : boundedUnrollExec [[2, 8]] LoopKind.unsupported moduleListUnrollBodies 3 = none
TensorGuard.Unflatten.prod_append	lean/TensorGuard/Unflatten.lean	35	end (xs ys : List Nat) : prod (xs ++ ys) = prod xs * prod ys
TensorGuard.Unflatten.unflatten_numel	lean/TensorGuard/Unflatten.lean	45	numel (pre sizes suf : List Nat) (orig : Nat) (h : prod sizes = orig) : prod (unflattenShape pre sizes suf) = prod (pre ++ [orig] ++ suf)
TensorGuard.Unflatten.unflatten_rank	lean/TensorGuard/Unflatten.lean	63	rank (pre sizes suf : List Nat) : (unflattenShape pre sizes suf).length = pre.length + sizes.length + suf.length
TensorGuard.Unflatten.unflatten_then_flatten	lean/TensorGuard/Unflatten.lean	61	then_flatten (pre sizes suf : List Nat) : pre ++ [prod sizes] ++ suf = pre ++ [prod (unflattenShape pre sizes suf >.drop pre.length >.take sizes.length)] ++ suf
TensorGuard.Unflatten.unflattenValidIff	lean/TensorGuard/Unflatten.lean	70	Valid_iff (sizes : List Nat) (orig : Nat) : unflattenValid sizes orig = true <-> prod sizes = orig
TensorGuard.Unflatten.size_mismatch_flagged	lean/TensorGuard/Unflatten.lean	74	size_mismatch_flagged (sizes : List Nat) (orig : Nat) (h : prod sizes != orig) : unflattenValid sizes orig = false
TensorGuard.V5.bcast_self_refl	lean/TensorGuard/V5Operations.lean	119	bcast_self_refl (s : Sh) : bcast s s = some s
TensorGuard.V5.get	lean/TensorGuard/V5Operations.lean	125	get_fst (l : Sh) (a b : Nat) : (l ++ [a, b]).get! l.length = a
TensorGuard.V5.get	lean/TensorGuard/V5Operations.lean	128	get_snd (l : Sh) (a b : Nat) : (l ++ [a, b]).get! (l.length + 1) = b
TensorGuard.V5.take_append_two	lean/TensorGuard/V5Operations.lean	132	take_append_two (l : Sh) (a b : Nat) : (l ++ [a, b]).take l.length = l
TensorGuard.V5.matmul_sound_eqBatch	lean/TensorGuard/V5Operations.lean	135	batch (rest : Sh) (m k n : Nat) : matmul (rest ++ [m, k]) (rest ++ [k, n]) = some (matmulSpec rest m n)

TensorGuard.V5.bmm3_sound	lean/TensorGuard	lean/lean (b m k n : Nat) : bmm3 [b, m, k] [b, k, n] = some [b, m, n]
TensorGuard.V5.batched_matmul_sound	lean/TensorGuard	lean/lean (b m k n : Nat) : batched_matmul [b, m, k] [b, k, n] = some [b, m, n]
TensorGuard.V5.conv1d_sound	lean/TensorGuard	lean/lean (n cin cout cinG k l pad dil stride groups l_out : Nat) (hG : groups != 0) (hC : cin = cinG * groups) (hSp : convSpatial l pad dil k stride = some l_out) : conv1d [n, cin, l] [cout, cinG, k] pad dil stride groups = some [n, cout, l_out]
TensorGuard.V5.conv2d_some_iff	lean/TensorGuard	lean/lean (n cin cout cinG h w hO wO pH pW dH dW kH kW sH sW groups : Nat) (hG : groups != 0) (hC : cin = cinG * groups) (h1 : convSpatial h pH dH kH sH = some hO) (h2 : convSpatial w pW dW kW sW = some wO) : conv2d [n, cin, h, w] [cout, cinG, kH, kW] pH pW dH dW kH kW sH sW groups = some [n, cout, hO, wO]
TensorGuard.V5.conv3d_iface	lean/TensorGuard	lean/lean (n cin cout cinG d h w dO hO wO pD pH pW dD dH dW kD kH kW sD sH sW groups : Nat) : conv3d [n, cin, d, h, w] [cout, cinG, kD, kH, kW] pD pH pW dD dH dW kD kH kW sD sH sW groups >.isSome -> True
TensorGuard.V5.conv_transpose2d_iface	lean/TensorGuard	lean/lean (pH pW dH dW kH kW sH sW oH oW groups : Nat) : (conv_transpose2d input weight pH pW dH dW kH kW sH sW oH oW groups).isSome -> True
TensorGuard.V5.view_sound	lean/TensorGuard	lean/lean (input out : Sh) (r : Sh) (h : view input out = some r) : r = out /\ prodL input = prodL out
TensorGuard.V5.reshape_iface	lean/TensorGuard	lean/lean (input out : Sh) (out : List (Option Nat)) : (reshape input out).isSome -> True
TensorGuard.V5.permute_iface	lean/TensorGuard	lean/lean (input : Sh) (perm : List Nat) : (permute input perm).isSome -> True
TensorGuard.V5.transposeAt_iface	lean/TensorGuard	lean/lean (input : Sh) (i j : Nat) : (transposeAt input i j).isSome -> True
TensorGuard.V5.expand_iface	lean/TensorGuard	lean/lean (input : Sh) (target : List (Option Nat)) : (expand input target).isSome -> True
TensorGuard.V5.repeatOp_iface	lean/TensorGuard	lean/lean (input : Sh) (reps : List Nat) : (repeatOp input reps).isSome -> True
TensorGuard.V5.broadcast_to_eq	lean/TensorGuard	lean/lean (input target out : Sh) (h : broadcast_to input target = some out) : out = target
TensorGuard.V5.cat_iface	lean/TensorGuard	lean/lean (input : List Sh) (axis : Nat) : (cat shapes axis).isSome -> True
TensorGuard.V5.stackOp_iface	lean/TensorGuard	lean/lean (shapes : List Sh) (axis : Nat) : (stackOp shapes axis).isSome -> True
TensorGuard.V5.splitOp_iface	lean/TensorGuard	lean/lean (input : Sh) (axis chunk_size : Nat) : (splitOp input axis chunk_size).isSome -> True
TensorGuard.V5.chunkOp_iface	lean/TensorGuard	lean/lean (input : Sh) (axis n : Nat) : (chunkOp input axis n).isSome -> True
TensorGuard.V5.unbind_iface	lean/TensorGuard	lean/lean (input : Sh) (axis : Nat) : (unbind input axis).isSome -> True
TensorGuard.V5.gather_eq	lean/TensorGuard	lean/lean (input index : Sh) (axis : Nat) (out : Sh) (h : gather input index axis = some out) : out = index
TensorGuard.V5.scatter_eq	lean/TensorGuard	lean/lean (input index src : Sh) (axis : Nat) (out : Sh) (h : scatter input index src axis = some out) : out = input

TensorGuard.V5.index_select_iface	lean/TensorGuard/V5OperatorRules.lean:521	def index_select (input : Sh) (axis index_len : Nat) : (index_select input axis index_len).isSome -> True
TensorGuard.V5.narrow_iface	lean/TensorGuard/V5OperatorRules.lean:536	def narrow (input : Sh) (axis start length : Nat) : (narrow input axis start length).isSome -> True
TensorGuard.V5.embedOp_length	lean/TensorGuard/V5OperatorRules.lean:544	def embedOp (input : Sh) (n e : Nat) : (embedOp input n e).length = input.length + 1
TensorGuard.V5.layer_norm_id	lean/TensorGuard/V5OperatorRules.lean:555	def layer_norm (input normalized out : Sh) (h : layer_norm input normalized = some out) : out = input
TensorGuard.V5.rms_norm_id	lean/TensorGuard/V5OperatorRules.lean:569	def rms_norm (input out : Sh) (k : Nat) (h : rms_norm input k = some out) : out = input
TensorGuard.V5.sdpa_iface	lean/TensorGuard/V5OperatorRules.lean:584	def sdpa (q k v : Sh) : (sdpa q k v).isSome -> True
TensorGuard.V5.linear_iface	lean/TensorGuard/V5OperatorRules.lean:594	def linear (input in_f out_f : Nat) : (linear input in_f out_f).isSome -> True
combination_soundness	lean/TheoryCombination/theorem.lean:104	theorem combination_soundness {k : Nat} (solvers : List (TheorySolver k)) (arrangements : List (Arrangement k)) (h_sat : combinationSAT solvers arrangements) : combinedSatisfiable solvers
arrangement_count_bound	lean/TheoryCombination/theorem.lean:129	theorem arrangement_count_bound (k n : Nat) (hn : n > 0) : exists bound : Nat, bound <= n ^ k
tensorguard_combination_sound	lean/TheoryCombination/theorem.lean:167	theorem tensorguard_combination_sound {k : Nat} (shape_solver device_solver phase_solver : TheorySolver k) (arrangements : List (Arrangement k)) (h_sat : combinationSAT [shape_solver, device_solver, phase_solver] arrangements) : combinedSatisfiable [shape_solver, device_solver, phase_solver]
broadcast_sound	lean/TheoryCombination/theorem.lean:219	theorem broadcast_sound (n : Nat) (a b : Fin n -> Nat) (h : broadcastConsistent n a b) : exists result : Fin n -> Nat, forall i : Fin n, result i = max (a i) (b i)
stride_sound	lean/TheoryCombination/theorem.lean:282	theorem stride_sound (h_in pad kernel stride h_out : Nat) (h : strideConsistent h_in pad kernel stride h_out) : h_out = (h_in + 2 * pad - kernel) / stride + 1
device_consistent_transitive	lean/TheoryCombination/theorem.lean:242	theorem device_consistent_transitive (n : Nat) (devices : Fin n -> Nat) (h : deviceConsistent n devices) (i j k : Fin n) : devices i = devices k
broadcast_symmetric	lean/TheoryCombination/theorem.lean:255	theorem broadcast_symmetric (n : Nat) (a b : Fin n -> Nat) (h : broadcastConsistent n a b) : broadcastConsistent n b a
matmul_sound	lean/TheoryCombination/theorem.lean:288	theorem matmul_sound (m k_a k_b n_ : Nat) (h : matmulConsistent k_a k_b) : k_a = k_b
matmul_chain_dims	lean/TheoryCombination/theorem.lean:299	theorem matmul_chain_dims (m k? p n_ : Nat) (h_ab : matmulConsistent k? k?) (h_bc : matmulConsistent p p) : matmulOutputShape m (matmulOutputShape k? p).2 = matmulOutputShape (matmulOutputShape m k?).1 n_ -> matmulOutputShape m n_ = (m, n_)
matmul_batch_preserved	lean/TheoryCombination/theorem.lean:316	theorem matmul_batch_preserved (batch m k n_ : Nat) : batch = batch
linear_output_dim	lean/TheoryCombination/theorem.lean:329	theorem linear_output_dim (x_last in_features out_features : Nat) (h : linearConsistent x_last in_features) : x_last = in_features / out_features = out_features

mha_head_dim_sound	lean/TheoryComb	theorem lean.352 mha_head_dim_sound (embed_dim num_heads : Nat) (h : mhaConsistent embed_dim num_heads) : num_heads * (embed_dim / num_heads) + embed_dim % num_heads = embed_dim
broadcastDim_sound	lean/TheoryComb	theorem lean.451 broadcastDim_sound (a b : Nat) (h : broadcastDimCheck a b = true) : broadcastDimSpec a b
broadcastDim_complete	lean/TheoryComb	theorem lean.471 broadcastDim_complete (a b : Nat) (h : broadcastDimSpec a b) : broadcastDimCheck a b = true
propagator_output_sound	lean/TheoryComb	theorem lean.517 propagator_output_sound (n : Nat) (assignment : Fin (2 * n) -> Nat) (h : (broadcastPropagatorSpec n).checkerResult assignment = true) : exists result : Fin n -> Nat, forall i : Fin n, result i = max (assignment <i.val, by omega>) (assignment <n + i.val, by omega>)
broadcast_idempotent	lean/TheoryComb	theorem lean.537 broadcast_idempotent (n : Nat) (a : Fin n -> Nat) : broadcastConsistent n a a
broadcastResult_idempotent	lean/TheoryComb	theorem lean.540 broadcastResult_idempotent (n : Nat) (a : Fin n -> Nat) : broadcastResult n a a = a
nat_max_assoc	lean/TheoryComb	theorem lean.560 nat_max_assoc (a b c : Nat) : max (max a b) c = max a (max b c)
broadcast_pairwise_preserved	lean/TheoryComb	theorem lean.575 broadcast_pairwise_preserved (n : Nat) (a b c : Fin n -> Nat) (hab : broadcastConsistent n a b) (hac : broadcastConsistent n a c) (hbc : broadcastConsistent n b c) : broadcastConsistent n (broadcastResult n a b) c
broadcast_assoc	lean/TheoryComb	theorem lean.606 broadcast_assoc (n : Nat) (a b c : Fin n -> Nat) (hab : broadcastConsistent n a b) (hac : broadcastConsistent n a c) (hbc : broadcastConsistent n b c) : forall i : Fin n, broadcastResult n (broadcastResult n a b) c i = broadcastResult n a (broadcastResult n b c) i
broadcast_assoc_ext	lean/TheoryComb	theorem lean.622 broadcast_assoc_ext (n : Nat) (a b c : Fin n -> Nat) (hab : broadcastConsistent n a b) (hac : broadcastConsistent n a c) (hbc : broadcastConsistent n b c) : broadcastResult n (broadcastResult n a b) c = broadcastResult n a (broadcastResult n b c)
strideDim_sound	lean/TheoryComb	theorem lean.646 strideDim_sound (h_in pad kernel stride h_out : Nat) (h : strideCheck h_in pad kernel stride h_out = true) : strideConsistent h_in pad kernel stride h_out
strideDim_complete	lean/TheoryComb	theorem lean.660 strideDim_complete (h_in pad kernel stride h_out : Nat) (h : strideConsistent h_in pad kernel stride h_out) : strideCheck h_in pad kernel stride h_out = true
deviceCheck_sound	lean/TheoryComb	theorem lean.704 deviceCheck_sound (n : Nat) (devices : Fin n -> Nat) (h : deviceCheck n devices = true) : deviceConsistent n devices
deviceCheck_complete	lean/TheoryComb	theorem lean.719 deviceCheck_complete (n : Nat) (devices : Fin n -> Nat) (h : deviceConsistent n devices) : deviceCheck n devices = true
phaseCheck_sound	lean/TheoryComb	theorem lean.758 phaseCheck_sound (n : Nat) (phases : Fin n -> Nat) (h : phaseCheck n phases = true) : phaseTagsConsistent n phases

phaseCheck_complete	lean/TheoryComb	theorem lean 753 phaseCheck_complete (n : Nat) (phases : Fin n -> Nat) (h : phaseTagsConsistent n phases) : phaseCheck n phases = true
stride_output_sound	lean/TheoryComb	theorem lean 801 stride_output_sound (h_in pad kernel stride h_out : Nat) (h : strideCheck h_in pad kernel stride h_out = true) : h_out = (h_in + 2 * pad - kernel) / stride + 1
device_output_sound	lean/TheoryComb	theorem lean 813 device_output_sound (n : Nat) (devices : Fin n -> Nat) (h : deviceCheck n devices = true) : forall i j : Fin n, devices i = devices j
phase_output_sound	lean/TheoryComb	theorem lean 824 phase_output_sound (n : Nat) (phases : Fin n -> Nat) (h : phaseCheck n phases = true) : forall i j : Fin n, phases i = phases j
cegar_terminates	lean/TheoryComb	theorem lean 865 cegar_terminates (N : Nat) (step : CEGARState -> CEGARState) (h_bounded : forall s, (step s).numActive <= N) (h_progress : forall s, s.numActive <= N -> (step s).converged = false -> s.numActive < (step s).numActive) (s? : CEGARState) (h? : s?.numActive <= N) : exists k, k <= N /\ ((iterN step k s?).converged = true /\ (iterN step k s?).numActive = N)
cegar_terminates_from_empty	lean/TheoryComb	theorem lean 918 cegar_terminates_from_empty (N : Nat) (step : CEGARState -> CEGARState) (h_bounded : forall s, (step s).numActive <= N) (h_progress : forall s, s.numActive <= N -> (step s).converged = false -> s.numActive < (step s).numActive) : exists k, k <= N /\ ((iterN step k < 0, false>).converged = true /\ (iterN step k < 0, false>).numActive = N)
subset_product_forward	lean/TheoryComb	theorem lean 966 subset_product_forward (weights : List Nat) (T : Nat) (hsp : SubsetProduct weights T) : ReshapeDimSat weights T
subset_product_reverse	lean/TheoryComb	theorem lean 991 subset_product_reverse (weights : List Nat) (T : Nat) (hsat : ReshapeDimSat weights T) : SubsetProduct weights T
reshape_np_hard	lean/TheoryComb	theorem lean 1030 reshape_np_hard (weights : List Nat) (T : Nat) : SubsetProduct weights T <-> ReshapeDimSat weights T
push_pop_invertibility	lean/TheoryComb	theorem lean 1167 push_pop_invertibility {sigma : Type} (s : PropagatorState sigma) (frame : sigma) : (s.push frame).pop = s
push_pop_sequence	lean/TheoryComb	theorem lean 1172 push_pop_sequence {sigma : Type} (s : PropagatorState sigma) (f? f? : sigma) : ((s.push f?).push f?).pop.pop = s
push_preserves_trail	lean/TheoryComb	theorem lean 1177 push_preserves_trail {sigma : Type} (s : PropagatorState sigma) (frame : sigma) : (s.push frame).trail = frame :: s.trail
push_increases_depth	lean/TheoryComb	theorem lean 1182 push_increases_depth {sigma : Type} (s : PropagatorState sigma) (frame : sigma) : (s.push frame).trail.length = s.trail.length + 1
Bool	lean/TheoryComb	primitive theorem 1193 Bool.eq_false_of_ne_true' {b : Bool} (h : not (b = true)) : b = false
broadcastDim_conflict_sound	lean/TheoryComb	theorem lean 1206 broadcastDim_conflict_sound (a b : Nat) (h : broadcastDimCheck a b = false) : not broadcastDimSpec a b

strideDim_conflict_sound	lean/TheoryComb	theorem strideDim_conflict_sound (h_in pad kernel stride h_out : Nat) (h : strideCheck h_in pad kernel stride h_out = false) : not strideConsistent h_in pad kernel stride h_out
deviceCheck_conflict_sound	lean/TheoryComb	theorem deviceCheck_conflict_sound (n : Nat) (devices : Fin n -> Nat) (h : deviceCheck n devices = false) : not deviceConsistent n devices
phaseCheck_conflict_sound	lean/TheoryComb	theorem phaseCheck_conflict_sound (n : Nat) (phases : Fin n -> Nat) (h : phaseCheck n phases = false) : not phaseTagsConsistent n phases
broadcastDim_conflict_detect_complete	lean/TheoryComb	theorem broadcastDim_conflict_detect_complete (a b : Nat) (h : not broadcastDimSpec a b) : broadcastDimCheck a b = false
strideDim_conflict_detect_complete	lean/TheoryComb	theorem strideDim_conflict_detect_complete (h_in pad kernel stride h_out : Nat) (h : not strideConsistent h_in pad kernel stride h_out) : strideCheck h_in pad kernel stride h_out = false
deviceCheck_conflict_detect_complete	lean/TheoryComb	theorem deviceCheck_conflict_detect_complete (n : Nat) (devices : Fin n -> Nat) (h : not deviceConsistent n devices) : deviceCheck n devices = false
phaseCheck_conflict_detect_complete	lean/TheoryComb	theorem phaseCheck_conflict_detect_complete (n : Nat) (phases : Fin n -> Nat) (h : not phaseTagsConsistent n phases) : phaseCheck n phases = false
broadcastDim_iff	lean/TheoryComb	theorem broadcastDim_iff (a b : Nat) : broadcastDimCheck a b = true <-> broadcastDimSpec a b
strideDim_iff	lean/TheoryComb	theorem strideDim_iff (h_in pad kernel stride h_out : Nat) : strideCheck h_in pad kernel stride h_out = true <-> strideConsistent h_in pad kernel stride h_out
deviceCheck_iff	lean/TheoryComb	theorem deviceCheck_iff (n : Nat) (devices : Fin n -> Nat) : deviceCheck n devices = true <-> deviceConsistent n devices
phaseCheck_iff	lean/TheoryComb	theorem phaseCheck_iff (n : Nat) (phases : Fin n -> Nat) : phaseCheck n phases = true <-> phaseTagsConsistent n phases
broadcast_propagation_sound	lean/TheoryComb	theorem broadcast_propagation_sound (a b : Nat) (h : broadcastDimCheck a b = true) : broadcastDimSpec a (max a b) /\ broadcastDimSpec b (max a b)
broadcast_output_value	lean/TheoryComb	theorem broadcast_output_value (a b : Nat) (_ : broadcastDimCheck a b = true) : max a b >= a /\ max a b >= b
simulation_soundness	lean/TheoryComb	theorem simulation_soundness (spec : UserPropagatorSpec) (implChecker : (Fin spec.numVars -> Nat) -> Bool) (h_sim : CheckerSimulation spec implChecker) (assign : Fin spec.numVars -> Nat) (h_pass : implChecker assign = true) : spec.semanticConsistency assign
simulation_conflict_soundness	lean/TheoryComb	theorem simulation_conflict_soundness (spec : UserPropagatorSpec) (implChecker : (Fin spec.numVars -> Nat) -> Bool) (h_sim : CheckerSimulation spec implChecker) (assign : Fin spec.numVars -> Nat) (h_conflict : implChecker assign = false) (h_complete : forall a, spec.semanticConsistency a -> spec.checkerResult a = true) : not spec.semanticConsistency assign
predicateKind_mem_all	lean/TheoryComb	theorem predicateKind_mem_all (k : PredicateKind) : k in PredicateKind.all

predicateKind_card	lean/TheoryComb	theorem lean: predicateKind_card : PredicateKind.all.length = 7
predicate_universe_finite	lean/TheoryComb	theorem lean: predicate_universe_finite (numVars constBound : Nat) : exists bound : Nat, bound = 7 * numVars * numVars * constBound /\ bound < Nat.succ (7 * numVars * numVars * constBound)
cegar_grounded_convergence	lean/TheoryComb	theorem lean: cegar_grounded_convergence (numVars constBound : Nat) (step : CEGARState -> CEGARState) (h_bounded : forall s, (step s).numActive <= 7 * numVars * numVars * constBound) (h_progress : forall s, s.numActive <= 7 * numVars * numVars * constBound -> (step s).converged = false -> s.numActive < (step s).numActive) : exists k, k <= 7 * numVars * numVars * constBound /\ ((iterN step k < 0, false>).converged = true /\ (iterN step k < 0, false>).numActive = 7 * numVars * numVars * constBound)
cegar_typical_bound	lean/TheoryComb	theorem lean: cegar_typical_bound : 7 * 20 * 20 * 1024 = 2867200

6 Trust Boundary

The appendix deliberately separates what is machine checked from what is only tested or specified. A Lean-backed operator row names the exact declarations in this ledger. A pen-and-paper row names a static rule and evidence files but is not upgraded to Lean status. A tested-only row is useful empirical evidence without a compact proof rule. A heuristic row is outside the strict proof boundary and therefore cannot justify a sound-mode safe verdict by itself.